

C++ 知识点大全

一份从基础到进阶、覆盖 C++98 ~ C++23 的系统性知识点复习文档。

每个知识点都采用「文字描述 + 代码示例」的形式讲解，并标注适用标准版本。

关于本文档

- 覆盖范围：**核心语法、函数、指针与内存、STL 容器、面向对象、模板泛型、异常、标准库、多线程、现代 C++ (11/14/17/20/23)、工程实践与面试高频题，共十二大部分。
- 讲解格式：**每个知识点包含【概念】（文字描述）、【要点】（提炼总结）、【示例】（可编译运行的代码）、【易错点/注意】（常见坑）。
- 版本标注：**特性后标注 (C++11)、(C++14)、(C++17)、(C++20)、(C++23) 表示引入该特性的最低标准；未标注的为基础内容 (C++98 即可用)。
- 编译建议：**代码示例默认使用 C++17 或更高标准编译，如 `g++ -std=c++17 main.cpp -o main`。

目录

- 第一部分：基础语法与核心概念
- 第二部分：函数进阶
- 第三部分：指针、引用与内存管理
- 第四部分：字符串、数组与 STL 容器
- 第五部分：类与对象（面向对象编程）
- 第六部分：模板与泛型编程
- 第七部分：异常处理
- 第八部分：标准库进阶
- 第九部分：并发与多线程
- 第十部分：现代 C++ 特性 (C++11/14/17/20/23)
- 第十一部分：预处理、命名空间与工程实践
- 第十二部分：常见陷阱、性能优化与面试高频题

学习路线建议

- 考前速刷：**第一部分 → 第二部分 → 第三部分 → 第五部分 → 第四部分 → 第七部分 → 第十二部分。
- 系统学习：**按目录顺序通读，代码全部手敲一遍。
- 面试冲刺：**重点看第三部分（内存与智能指针）、第五部分（类与多态）、第十部分（现代特性）、第十二部分（手写题）。

阅读约定

- 代码块中的 `//` 注释为讲解注释，帮助理解每一行关键代码。
- 复杂度对比表使用平均时间复杂度。
- 示例优先使用现代 C++ 写法，同时给出旧写法对比以应对考试与旧编译器环境。

第一部分：基础语法与核心概念

一、语言简介与编译流程

C++ 语言简介

概念：C++ 是一门通用编程语言，由 Bjarne Stroustrup 于 1979 年在 C 语言基础上发展而来，支持面向过程、面向对象、泛型与函数式等多种编程范式。它在提供底层控制能力（指针、手动内存管理）的同时，通过 RAII、模板、STL 等机制提供了高层次抽象，广泛应用于系统软件、游戏、金融、嵌入式等领域。

要点：

- C++ 是编译型、静态强类型语言，类型在编译期确定。
- 三大经典特性：封装、继承、多态；现代 C++ 强调"零开销抽象"。
- 语言标准演进：C++98 → C++03 → C++11（现代 C++ 起点）→ C++14 → C++17 → C++20 → C++23。
- 完全兼容 C 语言的大部分语法（但风格与最佳实践差异很大）。

示例：

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v{5, 2, 8, 1, 9};           // 初始化列表 (C++11)
    std::sort(v.begin(), v.end());              // 标准库算法
    for (int x : v)                             // 范围 for (C++11)
        std::cout << x << ' ';
    std::cout << '\n';
}
```

易错点/注意：C++ 允许书写 C 风格代码，但现代项目更推荐使用标准库容器与 RAII 管理资源，避免裸 new/delete。

编译与运行全流程

概念：从源代码到可执行文件要经历四个阶段——预处理、编译、汇编、链接。预处理展开宏与头文件，编译把预处理后的代码翻译成汇编代码，汇编把汇编代码转换为目标文件（机器码），链接把多个目标文件与库合并为最终可执行文件。

要点：

- 预处理：处理 `#include`、`#define`、`#if` 等指令，输出纯 C++ 文本。
- 编译：语法检查与优化，生成汇编代码（`.s` 文件）。
- 汇编：生成目标文件（`.o` / `.obj`），内部已是机器码但符号地址未定。
- 链接：解析符号引用，合并库与目标文件，生成可执行文件。
- 用 `g++` 可单独观察每个阶段。

示例：

```
// main.cpp — 供编译演示的源文件
#include <iostream>
int main() {
    std::cout << "Hello, build!\n";
    return 0;
}
```

```
# 一步到位：编译并链接
g++ -std=c++17 -Wall -wextra -O2 main.cpp -o main

# 分阶段观察：
g++ -E main.cpp -o main.i      # 1. 预处理（展开宏与头文件）
g++ -S main.i -o main.s        # 2. 编译（生成汇编代码）
g++ -C main.s -o main.o        # 3. 汇编（生成目标文件）
g++ main.o -o main             # 4. 链接（生成可执行文件）
```

易错点/注意：语法错误在编译阶段报出，而“未定义引用（undefined reference）”属于链接阶段错误，通常是因为只声明未定义函数，或忘记链接对应库（如数学库需要 `-lm`）。

二、基本程序结构

基本程序结构（main、include、注释）

概念：一个可执行 C++ 程序必须有且仅有一个 `main` 函数作为入口。`#include` 把头文件内容插入当前文件，注释用于给代码添加说明，编译器会忽略注释。

要点：

- `main` 的标准签名是 `int main()` 或 `int main(int argc, char* argv[])`，返回值 0 表示成功。
- `#include <...>` 用于标准库头文件，`#include "..."` 优先搜索当前目录，常用于自定义头文件。
- 单行注释 `//`，多行注释 `/* ... */`（不可嵌套）。
- 命名空间 `std`：标准库名字都在其中，可用 `using namespace std;`（教学方便，工程不推荐）。

示例：

```
#include <iostream>    // 引入标准输入输出流
#include <string>       // 引入字符串

/* 这是一个多行注释
   主函数：程序入口 */
int main() {
    std::string name = "Colin";
    std::cout << "Hello, " << name << "!\n"; // 单行注释
    return 0;    // 0 表示正常退出
}
```

易错点/注意：`/* ... */` 注释不能嵌套；把 `main` 写成 `void main()` 不符合标准，应使用 `int main()`。

三、基本数据类型

基本数据类型与范围

概念：基本（内建）类型分为整型、字符型、布尔型与浮点型。它们占用的字节数与取值范围可通过 `sizeof` 运算符和 `<limits>` 头文件查询。标准只规定“最小宽度”与相对大小关系，具体大小由实现决定。

要点：

- 整型：short (≥16位)、int (≥16位, 通常32位)、long (≥32位)、long long (≥64位)。
- 字符型：char (1字节, 通常8位), wchar_t / char16_t / char32_t 用于宽字符。
- 布尔型：bool 只取 true / false, 占 1 字节。
- 浮点型：float (单精度)、double (双精度)、long double (扩展精度)。
- sizeof(类型) 返回字节数, <limits> 提供 min()、max()、lowest() 等静态成员。

示例:

```
#include <iostream>
#include <limits>

int main() {
    std::cout << "short 字节数: " << sizeof(short) << '\n';
    std::cout << "int 字节数: " << sizeof(int) << '\n';
    std::cout << "long 字节数: " << sizeof(long) << '\n';
    std::cout << "long long 字节数: " << sizeof(long long) << '\n';
    std::cout << "char 字节数: " << sizeof(char) << '\n';
    std::cout << "bool 字节数: " << sizeof(bool) << '\n';
    std::cout << "float 字节数: " << sizeof(float) << '\n';
    std::cout << "double 字节数: " << sizeof(double) << '\n';
    std::cout << "long double 字节数: " << sizeof(long double) << '\n';

    std::cout << "int 范围: [" << std::numeric_limits<int>::min()
                << ", " << std::numeric_limits<int>::max() << "]\n";
    std::cout << "unsigned int 范围: [0, "
                << std::numeric_limits<unsigned int>::max() << "]\n";
    std::cout << "double 位数: " << std::numeric_limits<double>::digits << '\n';
    std::cout << "float 精度(epsilon): "
                << std::numeric_limits<float>::epsilon() << '\n';
}
```

易错点/注意: sizeof 不是函数而是运算符, 编译期求值; 不要假设 int 一定是 32 位, 跨平台时可用 <cstdint> 中的 int32_t、int64_t 等定宽类型。

有符号与无符号

概念: 整型默认是有符号 (signed) 的, 用 unsigned 关键字可声明无符号类型, 其所有位都用来表示数值, 因此范围是非负且上限更大。有符号与无符号混用时会触发类型提升, 容易产生意外结果。

要点:

- unsigned int 可简写为 unsigned; char 是否有符号由实现决定。
- 无符号数溢出是“模 2^n 回绕” (合法), 有符号数溢出是未定义行为。
- 有符号与无符号运算时, 有符号数会被转换为无符号数 (类型提升)。
- 无符号数减到 0 以下会回绕成极大值, 循环条件容易出错。

示例:

```
#include <iostream>
#include <limits>

int main() {
    unsigned int a = 0;
    a = a - 1; // 回绕为 4294967295 (32 位下)
    std::cout << "0 - 1 = " << a << '\n';
}
```

```

unsigned int big = std::numeric_limits<unsigned int>::max();
big = big + 1;           // 回绕为 0
std::cout << "max + 1 = " << big << '\n';

int x = -1;
unsigned int y = 1;
// -1 被转换为极大的 unsigned, 比较结果为 true
if (x < y)
    std::cout << "有符号 -1 被提升为无符号后比较\n";
}

```

易错点/注意： `for (unsigned i = 10; i >= 0; --i)` 是死循环，因为 `i` 到 0 后再减变为极大值仍满足 `>= 0`；应改用 `int` 或改写循环条件。

四、字面量与初始化

字面量与转义字符

概念：字面量是直接写在代码里的常量值，如 `42`、`3.14`、`'a'`、`"hello"`、`true`。后缀可以指定字面量的精确类型，转义字符用于在字符或字符串中表示无法直接书写的字符。

要点：

- 整型后缀：`u/U` (unsigned)、`l/L` (long)、`ll/LL` (long long)，可组合如 `42ULL`。
- 浮点后缀：`f/F` (float)、`l/L` (long double)；无后缀的浮点是 `double`。
- 字符字面量 `'a'` 类型是 `char`，字符串字面量 `"ab"` 类型是 `const char[N]`。
- 转义：`\n`、`\t`、`\\`、`\'`、`\"`、`\0`；`\x` 后跟十六进制，`\u/\U` 表示 Unicode 码点。
- 二进制字面量 `0b1010` 与数字分隔符 `1'000'000` 是 (C++14)。

示例：

```

#include <iostream>

int main() {
    int a = 0b1010;           // C++14 二进制字面量，等于 10
    int b = 1'000'000;        // C++14 数字分隔符，等于 1000000
    unsigned long long c = 42ULL; // unsigned long long 后缀
    float f = 3.14f;          // float 后缀
    double d = 3.14;          // 默认 double
    char ch = 'A';            // 字符字面量
    char tab = '\t';          // 转义：制表符
    const char* s = "line1\nline2"; // 字符串含换行转义

    std::cout << a << " " << b << " " << c << " "
              << f << " " << d << " " << ch << tab << s << '\n';
}

```

易错点/注意： `'\0'` 是空字符（值 0），字符串末尾由编译器自动追加 `'\0'`；`'\x41'` 等于 `'A'`，但要小心 `\x` 会尽量多地吞并后续十六进制字符。

变量的初始化

概念：初始化是给变量赋予初始值的过程，与"赋值"不同。C++ 提供默认初始化、拷贝初始化、直接初始化、值初始化与列表初始化等多种形式，推荐统一使用列表初始化 `{}` 以杜绝窄化与"最令人烦恼的解析"。

要点：

- 默认初始化：`int x;` 内置类型在局部作用域内是未定义值（未初始化）。
- 拷贝初始化：`int x = 5;`；直接初始化：`int x(5);`。
- 列表初始化：`int x{5};`，发生窄化转换会报错。
- 值初始化：`int x{};` 等价于初始化为 0。
- 未初始化变量的值是垃圾值，读取属未定义行为，是常见 bug 来源。

示例：

```
#include <iostream>
#include <vector>

int main() {
    int a = 5;           // 拷贝初始化
    int b(6);           // 直接初始化
    int c{7};           // 列表初始化 (C++11)
    int d{};            // 值初始化, d == 0
    int e;              // 默认初始化: 局部变量值不确定!

    // 窄化转换在列表初始化下会编译报错
    // int bad{3.14}; // 错误: double 到 int 是窄化

    std::vector<int> v{1, 2, 3, 4}; // 容器用列表初始化
    std::cout << a << b << c << d << '\n';
}
```

示例（零初始化）：

```
#include <iostream>

int g;                // 全局变量: 默认零初始化
static int sg;        // 静态存储期: 默认零初始化

int main() {
    static int ls;     // 局部 static: 默认零初始化
    int local;        // 普通局部变量: 未初始化 (值不确定)
    std::cout << g << ' ' << sg << ' ' << ls << '\n'; // 输出 0 0 0
    // std::cout << local; // 未定义行为, 不要读取
}
```

易错点/注意：`int x();` 是"函数声明"而非变量定义（最令人烦恼的解析），应写 `int x{};` 或 `int x;`。未初始化的内置变量不要读取；全局/静态变量会被零初始化，局部普通变量不会。

五、const 与 constexpr

const 与 constexpr

概念： `const` 表示"只读"，对象一旦初始化就不能修改；`constexpr` 表示"编译期常量"，其值在编译期即可确定。两者可以组合，但语义不同：`const` 不一定是编译期常量，`constexpr` 则强制要求编译期可求值。

要点：

- `const int N = 10;` 是运行期只读（若用字面量初始化也常被当作编译期常量）。
- `constexpr int M = 20;` 一定是编译期常量，可用于数组长度、模板参数。
- 指针的 `const` 分两种：`const int* p`（指向只读的指针）与 `int* const p`（只读指针）。
- `constexpr` 函数在 (C++11) 引入，(C++14) 放宽为可包含循环与局部变量。
- 与 `#define` 相比，`const` 常量有类型、有作用域、可被调试器识别。

示例：

```
#include <iostream>
#include <array>

constexpr int square(int x) { // constexpr 函数 (C++11)
    return x * x;
}

int main() {
    const int a = 10;          // 只读变量
    constexpr int b = 20;      // 编译期常量
    // a = 11;                 // 错误: const 不可修改

    const int* p1 = &a;        // 指向常量的指针，不能通过 p1 改值
    int val = 5;
    int* const p2 = &val;      // 常指针，p2 不能改指向
    *p2 = 6;                   // 可以改所指向的值

    int arr[square(3)];         // 编译期求值，等于 9
    std::array<int, square(4)> ar{}; // 16 个元素
    std::cout << square(b) << ' ' << *p2 << ' ' << sizeof(arr)/sizeof(int) <<
    '\n';
}
```

易错点/注意： `const int n = 10;` 用作数组长度是编译器扩展，标准 C++ 中数组长度要求常量表达式，此时应使用 `constexpr` 或 `const int`（用字面量初始化时间为常量表达式）。`constexpr` 变量必须用常量表达式初始化。

六、存储说明符

存储说明符 (static/extern/mutable/volatile/thread_local)

概念： 存储说明符控制变量的存储期、链接性与访问方式。`static` 影响内部链接或局部生命周期，`extern` 声明外部链接，`mutable` 允许在 `const` 成员函数中修改成员，`volatile` 阻止编译器优化对该变量的访问，`thread_local` 让每个线程拥有独立副本。

要点：

- 局部 `static`：只初始化一次，生命周期贯穿整个程序，作用域仍局限于所在块。
- 文件级 `static`：内部链接，仅本翻译单元可见。
- `extern`：声明变量在其他翻译单元定义，用于跨文件共享。

- `mutable`：只能用于类成员，配合 `const` 成员函数使用。
- `volatile`：告诉编译器每次读写都直接访问内存，不缓存到寄存器。
- `thread_local` (C++11)：每个线程独立副本，常与 `static/extern` 组合。

示例：

```
#include <iostream>
#include <thread>

int counter = 0;           // 外部链接（可用 extern 声明于别处）
static int file_only = 1; // 内部链接，仅本文件可见

void count() {
    static int calls = 0;    // 局部 static，只初始化一次
    ++calls;
    std::cout << "被调用 " << calls << " 次\n";
}

int main() {
    count();
    count();

    thread_local int per_thread = 0; // 每个线程独立
    per_thread = 42;
    std::thread t([&] {
        per_thread = 7;           // 修改的是线程 t 自己的副本
        std::cout << "子线程 per_thread = " << per_thread << '\n';
    });
    t.join();
    std::cout << "主线程 per_thread = " << per_thread << '\n';
}
```

示例 (`extern` 与 `mutable`)：

```
#include <iostream>

// ---- file1.cpp (另一个翻译单元) ----
// int shared = 42;    // 在此处定义

// ---- 本文件 (file2.cpp) ----
extern int shared;      // 声明：在别处定义，链接期解析

struct Cache {
    mutable int accessCount = 0; // const 成员函数也能修改
    int data = 1;
    int get() const {
        ++accessCount;           // mutable 允许在 const 函数中自增
        return data;
    }
};

int main() {
    Cache c;
    c.get();
    c.get();
    std::cout << "访问次数 = " << c.accessCount << '\n';
}
```

```
// std::cout << shared << '\n'; // 需与 file1.cpp 一起链接
}
```

易错点/注意：局部 `static` 的初始化在首次执行到该语句时进行，且是线程安全的（C++11 起保证）；`volatile` 不等于“线程安全”，多线程同步应使用 `std::atomic`。

七、auto 与 decltype

auto 与 decltype

概念：`auto` 让编译器从初始化表达式推导变量类型，`decltype` 获取表达式的声明类型。两者规则不同：`auto` 会丢弃顶层 `const` 与引用，而 `decltype` 会完整保留；`decltype(auto)` (C++14) 则让 `auto` 按照 `decltype` 的规则推导。

要点：

- `auto` 必须由初始化表达式推导，不能单独声明 `auto x;`。
- `auto` 丢弃顶层 `const` 和引用：`const int& r = x; auto a = r;` 得到 `int`。
- 想保留引用/`const` 需显式写 `auto&`、`const auto&`。
- `decltype(表达式)` 不真正求值表达式，只取类型。
- `decltype(auto)` 保留引用与顶层 `const`，常用于转发返回类型。

示例：

```
#include <iostream>
#include <vector>

int main() {
    int x = 1;
    const int& r = x;

    auto a = r;           // a 是 int (丢弃 const 与引用)
    auto& b = r;           // b 是 const int&
    const auto c = r;      // c 是 const int

    decltype(r) d = r;    // d 是 const int& (保留)

    auto&& e = x;           // 万能引用, e 是 int&
    decltype(auto) f = r;  // C++14: f 是 const int&

    std::vector<int> v{1, 2, 3};
    for (auto it = v.begin(); it != v.end(); ++it) // 迭代器推导
        std::cout << *it << ' ';
    std::cout << '\n';
}
```

易错点/注意：`auto` 用于返回类型时也会丢弃引用，返回容器元素的引用需写 `auto&`；`decltype(auto)` 不能省略括号区分 `decltype(x)` 与 `decltype((x))`（后者得到引用类型）。

八、类型别名

类型别名 (typedef 与 using)

概念：类型别名给已有类型起一个新名字，提高可读性。`using` (C++11) 语法更直观，且唯一支持"模板别名"，因此在现代 C++ 中优先使用 `using`。

要点：

- `typedef` 是 C 遗留语法，声明顺序易混淆（尤其函数指针）。
- `using` 采用"名字 = 类型"的直观形式。
- 模板别名只能用 `using`：`template<class T> using Vec = std::vector<T>;`。
- 别名不产生新类型，原类型与别名完全等价。

示例：

```
#include <iostream>
#include <vector>
#include <string>

// typedef 写法
typedef unsigned long ulong;
typedef int (*FuncPtr)(int, int);

// using 写法 (C++11)
using ulong2 = unsigned long;
using FuncPtr2 = int (*)(int, int);
template<class T> using Vec = std::vector<T>; // 模板别名

int add(int a, int b) { return a + b; }

int main() {
    ulong x = 100UL;
    ulong2 y = 200UL;
    FuncPtr fp = add;           // 函数指针别名
    Vec<int> v{1, 2, 3};        // 等价于 std::vector<int>

    std::cout << x + y << " " << fp(3, 4) << " " << v.size() << '\n';
}
```

易错点/注意：`typedef` 与 `using` 都只是别名，不是新类型，二者混用不会造成重载区分；模板别名不能通过 `typedef` 实现。

九、类型转换

类型转换

概念：类型转换把一个值从一种类型变为另一种。C++ 提供四类显式转换运算符，各自语义明确、便于查找，应取代容易隐藏错误的 C 风格转换。隐式转换在需要时会自动发生，但窄化可能丢数据。

要点：

- `static_cast`：编译期检查的普通转换（数值、指针向上转型、`void*` 还原）。
- `dynamic_cast`：运行期安全的向下转型，仅用于多态类型，失败返回空指针（或抛异常）。
- `const_cast`：去除或添加 `const/volatile`，是唯一能改 `cv` 限定符的转换。
- `reinterpret_cast`：位模式重新解释，最危险，仅限底层操作。
- C 风格 `(T)x`：几乎什么都能转，不区分语义，强烈不推荐。
- 隐式转换与窄化：`double` 到 `int` 会截断，列表初始化下会报错。

示例：

```
#include <iostream>

struct Base { virtual ~Base() {} };
struct Derived : Base { int v = 1; };

int main() {
    double d = 3.99;
    int a = static_cast<int>(d);    // 截断为 3
    std::cout << "static_cast: " << a << '\n';

    const int c = 10;
    int* p = const_cast<int*>(&c); // 去掉 const (不要修改原 const 对象)
    *p = 20;                       // 未定义行为: 修改 const 对象

    int i = 0x1234;
    char* bytes = reinterpret_cast<char*>(&i); // 按字节查看

    Derived der;
    Base* b = &der;                  // 隐式向上转型
    Derived* dd = dynamic_cast<Derived*>(b); // 安全的向下转型
    if (dd) std::cout << "dynamic_cast 成功, v=" << dd->v << '\n';

    int x = (int)d;                  // C 风格转换 (不推荐)
    std::cout << x << ' ' << (int)bytes[0] << '\n';
}
```

对比表：

转换方式	用途	运行期开销	安全性
static_cast	普通类型转换、向上转型	无	高（编译期检查）
dynamic_cast	多态向下转型	有（RTTI）	高（运行期检查）
const_cast	增删 const/volatile	无	中（易误用）
reinterpret_cast	位级重解释	无	低（依赖实现）
C 风格 (T)x	混合上述全部	视情况	低（语义不清）

易错点/注意：用 `const_cast` 去掉 `const` 后修改一个原本 `const` 的对象是未定义行为；
`dynamic_cast` 要求基类至少有一个虚函数（多态类型），否则编译报错。

十、运算符与优先级

运算符与优先级

概念：运算符按优先级与结合性参与表达式求值。算术、关系、逻辑、位、赋值、自增减、三目、逗号与成员访问共同构成 C++ 的运算符体系。短路求值是逻辑运算符的重要特性。

要点：

- 算术：`+` `-` `*` `/` `%`，`/` 对整数是整除，`%` 只能用于整数。
- 关系：`==` `!=` `<` `<=` `>` `>=`；逻辑：`&&` `||` `!`。
- 位：`&` `|` `^` `~` `<<` `>>`；赋值：`=` `+=` `-=` `*=` `/=` `%=` 等。

- 自增减：`++x`（先加后用）与 `x++`（先用后加）。
- 三目：`条件 ? 表达式1 : 表达式2`；逗号：从左到右求值，返回最右值。
- 短路：`&&` 左边为假、`||` 左边为真时，右边不求值。
- 优先级从高到低：`::` > 后缀 `() [] -> . ++ --` > 一元 `! ~ ++ -- - + * &` > 乘除 > 加减 > 移位 > 关系 > 相等 > 位与 > 位异或 > 位或 > 逻辑与 > 逻辑或 > 三目 > 赋值 > 逗号。

示例：

```
#include <iostream>

int main() {
    int a = 5, b = 2;
    std::cout << "整除: " << a / b << " 取余: " << a % b << '\n'; // 2 与 1

    int x = 1;
    int y = x++ + ++x;    // 顺序未定义，不要依赖！应拆开写
    std::cout << "y = " << y << '\n';

    int* p = nullptr;
    // 短路求值: p 为空时右边不会解引用
    if (p != nullptr && *p > 0)
        std::cout << "不会执行\n";

    int z = (a > b) ? 100 : 200;    // 三目, z = 100
    int w = (z = 1, z + 2);        // 逗号: 先 z=1, 再取 z+2, w=3
    std::cout << "z=" << z << " w=" << w << '\n';
}
```

易错点/注意：`x++ + ++x` 之类的表达式在 C++ 中求值顺序未定义，属未定义行为；赋值运算符是右结合，`a = b = c` 等价于 `a = (b = c)`。

位运算专题

概念：位运算直接操作整数的二进制位，速度极快，常用于掩码、标志位、状态压缩与底层编程。移位、按位与或异或、取反是基本操作，`std::bitset` 提供了类型安全的位集合。

要点：

- `&` 按位与：常用于取指定位（掩码）；`|` 按位或：用于置位。
- `^` 按位异或：相同为 0、相异为 1，可用于交换与翻转。
- `~` 按位取反；`<<` 左移（低位补 0）；`>>` 右移（无符号补 0，有符号行为由实现定义）。
- 常见技巧：`x & (x-1)` 清零最低位的 1；`x & -x` 取出最低位的 1。
- 左移右移超出位数是未定义行为；`1 << n` 中 1 是 int，需注意溢出。
- `std::bitset<N>` 提供 `set/reset/flip/test/count` 等成员。

示例：

```
#include <iostream>
#include <bitset>

int main() {
    unsigned int flags = 0b0000;
    const unsigned int READ = 1 << 0; // 0001
    const unsigned int WRITE = 1 << 1; // 0010
    const unsigned int EXEC = 1 << 2; // 0100
}
```

```

flags |= READ | WRITE;           // 置位：开启读和写
flags &= ~WRITE;                 // 清位：关闭写
bool canRead = flags & READ;     // 测试某位是否置位

int x = 0b101100;
int lowest = x & -x;             // 取出最低位的 1
int cleared = x & (x - 1);       // 清零最低位的 1

std::bitset<8> bs(0b1010);
bs.flip(0);                      // 翻转第 0 位
std::cout << "canRead=" << canRead
          << " lowest=" << std::bitset<8>(lowest)
          << " cleared=" << std::bitset<8>(cleared)
          << " bs=" << bs << '\n';
}

```

易错点/注意： `1 << 31` 在 32 位 int 中产生符号位问题，应使用 `1U << 31` 或 `1ULL << n`；右移负数结果是实现定义的，避免依赖。

十一、控制流

控制流

概念：控制流语句决定程序的执行顺序，包括分支（if/else、switch）、循环（while/do-while/for、范围 for）与跳转（break/continue/goto）。合理使用这些语句可以表达任意逻辑，但要避免“意大利面代码”。

要点：

- if/else 支持任意条件；else 与最近的未匹配 if 配对。
- switch 的 case 是常量表达式，匹配后从该处贯穿执行，需 break；fallthrough (C++17) 显式标注贯穿意图。
- while 先判断后执行，do-while 至少执行一次。
- for 三段式；范围 for for（元素：容器）遍历容器或数组。
- break 跳出本层循环/switch；continue 跳过本次循环剩余部分。
- if（初始化；条件）是 (C++17) 语法，把临时变量作用域限定在 if 内。

示例：

```

#include <iostream>
#include <vector>

int main() {
    // switch 与 fallthrough
    int level = 2;
    switch (level) {
        case 1:
            std::cout << "基础\n";
            break;
        case 2:
            std::cout << "进阶\n";
            [[fallthrough]]; // C++17 显式贯穿
        case 3:
            std::cout << "高级\n";
            break;
        default:
    }
}

```

```

        std::cout << "未知\n";
    }

    // 范围 for 与 continue
    std::vector<int> v{1, 2, 3, 4, 5};
    for (int n : v) {
        if (n % 2 == 0) continue;    // 跳过偶数
        std::cout << n << ' ';
    }
    std::cout << '\n';

    // C++17 if 初始化语句
    if (int found = 42; found > 0)
        std::cout << "found = " << found << '\n';
}

```

示例 (while/do-while/goto) :

```

#include <iostream>

int main() {
    int n = 0;
    while (n < 3) {                // 先判断后执行
        std::cout << "while " << n << '\n';
        ++n;
    }

    do {                          // 至少执行一次
        std::cout << "do-while " << n << '\n';
    } while (n < 3);

    for (int i = 0; i < 3; ++i) {
        if (i == 1) continue;    // 跳过 i==1 本次循环
        std::cout << "for " << i << '\n';
    }

    int k = 0;
loop:                             // goto 标签
    std::cout << "goto " << k << '\n';
    if (++k < 2) goto loop;    // 少用，易造成混乱
}

```

易错点/注意： `switch` 忘记 `break` 会贯穿到下一个 `case`; `continue` 用在 `while` 中要确保更新循环变量，否则可能死循环。 `goto` 尽量少用，且不能跳过带初始化的变量定义。

十二、标准输入输出

标准输入输出 (cin/cout 与格式化)

概念： C++ 使用流对象 `cin`、`cout`、`cerr`、`clog` 完成标准输入输出。通过操纵符（如 `std::setw`、`std::setprecision`）与 `std::format` (C++20) 可控制格式。 `getline` 用于整行读取。

要点：

- `cout` 标准输出、`cin` 标准输入、`cerr` 无缓冲错误输出、`clog` 缓冲日志输出。

- `<iomanip>` 提供 `setw`、`setfill`、`setprecision`、`fixed`、`hex` 等操纵符。
- `std::boolalpha` 让 `bool` 输出为 `true/false`；`std::showbase` 显示进制前缀。
- `getline(cin, s)` 读一整行（含空格，不含换行），常与 `cin >>` 混用出问题。
- `std::format` (C++20) 提供类似 Python 的格式化，比 `printf` 更安全。

示例：

```
#include <iostream>
#include <iomanip>
#include <string>
// #include <format>    // C++20

int main() {
    double pi = 3.14159265358979;

    std::cout << "默认: " << pi << '\n';
    std::cout << std::fixed << std::setprecision(2)
              << "保留两位: " << pi << '\n';

    std::cout << "十六进制: " << std::hex << std::showbase << 255
              << std::dec << '\n';

    std::cout << std::boolalpha << "布尔: " << true << '\n';
    std::cout << std::setw(10) << std::setfill('*') << 42 << '\n';

    std::cout << "输入姓名: ";
    std::string name;
    std::getline(std::cin, name);    // 读取整行
    std::cout << "你好, " << name << '\n';
}
```

易错点/注意：`cin >> n` 读取数字后会残留换行符在缓冲区，紧接着的 `getline` 会读到空行；应在两者之间调用 `cin.ignore()` 清掉残留字符。`setw` 只对下一个输出生效，之后需重新设置。

十三、struct、union、enum

struct、union、enum 与 enum class

概念：`struct` 与 `class` 几乎相同（仅默认访问权限不同），用于聚合数据；`union` 让多个成员共享同一块内存；`enum` 是传统枚举（作用域泄漏），`enum class` (C++11) 是强类型枚举（作用域限定、禁止隐式转 `int`）。

要点：

- `struct` 成员默认 `public`，`class` 默认 `private`。
- `union` 所有成员共用存储，同一时刻只有一个成员有意义，大小等于最大成员。
- 传统 `enum` 枚举值会泄漏到外层作用域，且可隐式转为 `int`。
- `enum class` 需用 `枚举名::值` 访问，不能隐式转 `int`，可指定底层类型：`uint8_t`。
- 位域 `int x : 3` 让成员只占指定位数，常与 `union` 组合解析协议。

示例：

```
#include <iostream>
#include <cstdint>

struct Point { int x; int y; };    // 默认 public
```

```

union Value {                                // 共享内存
    int i;
    float f;
    char c[4];
};

enum Color { RED, GREEN, BLUE };             // 传统枚举, 值泄漏
enum class Status : uint8_t { OK = 0, WARN, ERROR }; // 强类型枚举

struct Flags {                                // 位域
    unsigned int read : 1;
    unsigned int write : 1;
    unsigned int exec : 1;
    unsigned int : 5;                         // 未命名位域, 占位填充
};

int main() {
    Point p{1, 2};
    Value v;
    v.i = 0x41424344;                         // 写入 int, 读取 char 数组

    Color c = RED;                            // 传统枚举直接用
    Status s = Status::OK;                    // 必须限定作用域
    // int n = s;                             // 错误: 不能隐式转 int
    int n = static_cast<int>(s);              // 需显式转换

    Flags fl{};
    fl.read = 1; fl.write = 0;
    std::cout << p.x << ' ' << v.c[0] << ' ' << c << ' ' << n
                << " read=" << fl.read << '\n';
}

```

易错点/注意: `union` 读取"非当前活跃成员"是未定义行为 (`char` 数组等例外); 传统枚举值容易与外部标识符冲突, 优先用 `enum class`。

十四、内存对齐

内存对齐 (alignof/alignas)

概念: 内存对齐指数据在内存中的地址必须是某个值的整数倍。CPU 按对齐地址访问更快, 某些平台要求强制对齐。 `alignof` (C++11) 查询类型的对齐要求, `alignas` (C++11) 指定变量的对齐值。

要点:

- 每种类型有对齐要求, 通常等于其大小 (但不超过硬件上限)。
- `struct` 大小会因对齐填充而大于成员大小之和。
- 排列成员时把大的放前面可减少填充、节约空间。
- `alignas(16)` 可强制 16 字节对齐, 常用于 SIMD/原子操作。
- `alignof(T)` 返回 `std::size_t` 类型的对齐字节数。

示例:

```

#include <iostream>

struct Bad {    // 顺序差: char 后跟 int 会填充 3 字节
    char c;     // 偏移 0

```

```

    int i;        // 偏移 4 (前面填充 3 字节)
};               // 大小 8

struct Good {    // 顺序好: 无填充
    int i;        // 偏移 0
    char c;       // 偏移 4
};               // 大小仍 8 (末尾填充 3 字节保持对齐)

struct alignas(16) Aligned { char c; }; // 强制 16 字节对齐

int main() {
    std::cout << "sizeof(Bad) = " << sizeof(Bad) << '\n';
    std::cout << "sizeof(Good) = " << sizeof(Good) << '\n';
    std::cout << "alignof(double) = " << alignof(double) << '\n';
    std::cout << "alignof(Aligned) = " << alignof(Aligned)
                << " sizeof = " << sizeof(Aligned) << '\n';
}

```

示例 (offsetof 观察填充) :

```

#include <iostream>
#include <cstdint>

struct S { char c; int i; };

int main() {
    std::cout << "c 偏移: " << offsetof(S, c) << '\n'; // 0
    std::cout << "i 偏移: " << offsetof(S, i) << '\n'; // 4, 中间有 3 字节填充
    std::cout << "sizeof(S) = " << sizeof(S) << '\n'; // 8
}

```

易错点/注意: 结构体总大小必须是对齐要求的整数倍, 所以末尾也可能填充; `alignas` 指定的值必须是合法的对齐值 (2 的幂, 且不小于自然对齐), 否则编译报错。

十五、作用域与生命周期

作用域与生命周期

概念: 作用域决定名字在何处可见, 生命周期决定对象何时创建与销毁。C++ 有块作用域、文件作用域、全局作用域等, 存储期分为自动、静态、线程与动态四种。名字遮蔽指内层作用域的同名变量隐藏外层变量。

要点:

- 块作用域: `{ }` 内的变量, 离开块即销毁 (自动存储期)。
- 文件/全局作用域: 定义在所有函数外的变量, 程序启动创建、结束销毁 (静态存储期)。
- 静态存储期: `static` 与全局变量, 未显式初始化时被零初始化。
- 动态存储期: `new` 创建的对象, 需手动 `delete` 释放 (或使用智能指针)。
- 名字遮蔽: 内层声明与外层同名, 内层优先, 外层被隐藏。
- 可用作用域解析运算符 `::` 访问被遮蔽的全局变量。

示例:

```

#include <iostream>

int global = 100;           // 全局作用域, 静态存储期

```

```

void func() {
    static int s = 0;          // 静态存储期，只初始化一次
    ++s;
    std::cout << "static s = " << s << '\n';
}

int main() {
    int global = 1;            // 遮蔽全局变量
    std::cout << "局部 global = " << global << '\n';
    std::cout << "全局 global = " << ::global << '\n'; // 访问全局

    func();
    func();

    {
        int block = 5;         // 块作用域，离开即销毁
        std::cout << "块内 block = " << block << '\n';
    }
    // std::cout << block;      // 错误: block 已超出作用域

    int* p = new int(10);       // 动态存储期
    std::cout << "动态 *p = " << *p << '\n';
    delete p;                   // 手动释放
}

```

易错点/注意：局部变量离开作用域后其内存可能被复用，返回局部变量地址是悬垂指针；动态分配不释放会造成内存泄漏，优先用 `std::unique_ptr` / `std::shared_ptr`。

本部分小结

易忘点	说明
有符号与无符号混用	有符号会提升为无符号，比较结果可能出乎意料
无符号循环 <code>i >= 0</code>	无符号数永不小于 0，是死循环
局部未初始化变量	值是垃圾值，读取是未定义行为
<code>int x();</code>	是函数声明，不是变量定义
列表初始化 <code>{}</code>	杜绝窄化，优先使用
auto 丢引用/const	需显式写 <code>auto&</code> 、 <code>const auto&</code>
<code>1 << 31</code>	溢出/符号问题，用 <code>1U << 31</code>
switch 忘 break	贯穿执行，或显式 <code>[[fallthrough]]</code>
cin >> 后 getline	残留换行符，需 <code>cin.ignore()</code>
union 读非活跃成员	未定义行为
动态内存忘 delete	内存泄漏，用智能指针

清单总结：优先现代 C++ 写法（列表初始化、范围 for、enum class、using 别名、强类型转换）；区分 const 与 constexpr；警惕有符号/无符号与窄化；理解编译四阶段与四种存储期。

第二部分：函数进阶

一、函数基础

函数基础（声明/定义/调用/返回值/void/main 参数）

概念：函数是把一段可复用逻辑封装成命名单元的基本机制，由返回类型、函数名、参数列表与函数体组成。声明（原型）只给出签名，定义提供实现，调用负责执行函数体。

要点：

- 声明只写签名：`int add(int a, int b);`，定义包含函数体。
- 使用函数前必须先声明或定义（先定义后调用即可）。
- 返回类型为 `void` 表示无返回值；返回值通过 `return` 表达式返回。
- `main` 的标准签名是 `int main(int argc, char* argv[])`，`argc` 是命令行参数个数，`argv` 是指向参数字符串的指针数组。
- 普通函数不能嵌套定义（lambda 除外）。

示例：

```
#include <iostream>
#include <string>

int add(int a, int b);           // 声明（原型）

int add(int a, int b) {         // 定义
    return a + b;
}

void greet(const std::string& name) { // void: 无返回值
    std::cout << "Hello, " << name << "!\n";
}

int main(int argc, char* argv[]) {
    std::cout << "参数个数: " << argc << '\n';
    for (int i = 0; i < argc; ++i) // argv[0] 是程序名
        std::cout << "argv[" << i << "] = " << argv[i] << '\n';
    std::cout << add(2, 3) << '\n'; // 调用
    greet("Colin");
    return 0;
}
```

示例（返回类型推导与尾置返回类型）：

```

#include <iostream>
#include <type_traits>

auto add(double a, double b) { return a + b; }           // C++14 返回类型推导
auto mul(int a, int b) -> int { return a * b; }          // 尾置返回类型

int main() {
    auto s = add(1.5, 2.5);           // s 推导为 double
    std::cout << s << ' ' << mul(3, 4) << '\n';
    std::cout << std::boolalpha
                << std::is_same_v<decltype(s), double> << '\n'; // true
}

```

易错点/注意：声明与定义的签名（返回类型、参数）必须完全一致，否则要么构成重载、要么链接时报“未定义引用”；一个程序只能有一个 `main` 入口。

二、参数传递

参数传递（按值/按引用/按 const 引用/按指针）

概念：传参方式决定函数对实参的读写能力与拷贝开销。按值会拷贝实参，按引用/指针直接操作实参，按 const 引用则在不拷贝的前提下保证只读。

要点：

- 按值：拷贝实参，函数内修改不影响调用者。
- 按引用 `int&`：直接绑定实参，修改会外传，且避免拷贝。
- 按 const 引用 `const int&`：只读、零拷贝，适合大型对象。
- 按指针 `int*`：可传空值（需判空），也能修改实参。
- 临时对象不能绑定到非 const 左值引用，但可绑定到 const 引用并延长其生命周期。

示例：

```

#include <iostream>
#include <string>

void byValue(int x) { x = 100; }           // 不影响调用者
void byRef(int& x) { x = 100; }            // 影响调用者
void byConstRef(const std::string& s) {    // 只读、零拷贝
    std::cout << s << '\n';
}

void byPtr(int* p) { if (p) *p = 200; }    // 可空，需判空

int main() {
    int a = 1, b = 2, c = 3;
    byValue(a);    std::cout << "a=" << a << '\n'; // 1
    byRef(b);      std::cout << "b=" << b << '\n'; // 100
    byPtr(&c);     std::cout << "c=" << c << '\n'; // 200
    byPtr(nullptr); // 安全
    byConstRef(std::string("hello"));      // const 引用绑定临时对象
}

```

选择建议对比表：

方式	是否拷贝	能否修改实参	能否传空	适用场景
按值	是	否	—	小型基本类型
按引用	否	是	否	需要修改实参
const 引用	否	否	否	只读大型对象/临时对象
按指针	否（仅指针拷贝）	是	是	可选参数、输出参数

示例（数组退化为指针）：

```
#include <iostream>

void printArr(const int arr[], int n) {    // arr 实为 const int*
    for (int i = 0; i < n; ++i)
        std::cout << arr[i] << ' ';
    std::cout << '\n';
}

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    printArr(a, 5);        // 数组名退化为首元素指针，需额外传长度
}
```

易错点/注意：临时对象不能绑定到 `int&`（非 `const` 左值引用）；`const` 引用可以绑定临时对象并延长其生命周期，常用于接收字面量。

三、默认参数

默认参数

概念：默认参数为参数提供缺省值，调用时可以省略。默认值必须从右向左连续给出，且只能在声明与定义其中一处出现。

要点：

- 默认参数只能从右往左省略，右侧不能出现"无默认值的参数"。
- 声明与定义都写默认值会重复定义报错，通常只写在声明处。
- 默认参数与函数重载可能产生二义性调用。
- 默认值在编译期绑定，可以是常量表达式。

示例：

```
#include <iostream>

void show(int a, int b = 10, int c = 20); // 只在声明处写默认值

void show(int a, int b, int c) {        // 定义处不重复
    std::cout << a << ' ' << b << ' ' << c << '\n';
}

int main() {
    show(1);                // 1 10 20
    show(1, 2);             // 1 2 20
    show(1, 2, 3);          // 1 2 3
}
```

```
// show(1, , 3);    // 错误：不能跳过中间参数
}
```

示例（默认参数与重载冲突）：

```
#include <iostream>

void f(int a)          { std::cout << "f(int)\n"; }
void f(int a, int b = 0) { std::cout << "f(int,int=0)\n"; }

int main() {
    // f(1);    // 二义性：两个重载都能匹配 f(1)
    f(1, 2);    // 明确匹配双参版本
}
```

易错点/注意： `void f(int a, int b = 0, int c)` 非法（`c` 在 `b` 右侧却没有默认值）；若在声明和定义两处都写默认值会报“重复默认实参”。

四、函数重载

函数重载

概念：同一作用域内函数名相同但参数列表不同即为重载。编译器根据实参的类型与数量选择最佳匹配；仅返回类型不同不构成重载。

要点：

- 重载要求参数个数或类型不同，返回类型不算重载依据。
- 匹配优先级：精确匹配 > 提升 > 标准转换 > 用户定义转换。
- 当多个候选同样“好”时产生二义性，编译报错。
- `const` 与非 `const` 成员函数可构成重载（按对象是否 `const` 选择）。

示例：

```
#include <iostream>

void print(int x)          { std::cout << "int: " << x << '\n'; }
void print(double x)       { std::cout << "double: " << x << '\n'; }
void print(int a, int b)   { std::cout << "两个 int: " << a << ' ' << b << '\n'; }

struct S {
    void f() { std::cout << "非 const 版本\n"; }
    void f() const { std::cout << "const 版本\n"; }    // const 重载
};

int main() {
    print(1);          // int
    print(1.0);        // double
    print(1, 2);       // 两个 int
    print('A');        // char 提升为 int → int

    S s;
    s.f();             // 非 const
    const S cs;
    cs.f();            // const
}
```

```
}
```

二义性示例：

```
void g(long) {}
void g(double) {}
// g(1); // 二义性: int→long 与 int→double 都是"标准转换", 无优劣之分
```

示例（引用重载与右值引用重载）：

```
#include <iostream>

void f(int& x) { std::cout << "左值引用\n"; }
void f(const int& x) { std::cout << "const 左值引用\n"; }
void f(int&& x) { std::cout << "右值引用\n"; }

int main() {
    int a = 1;
    const int b = 2;
    f(a); // 左值引用
    f(b); // const 左值引用
    f(3); // 右值引用
}
```

易错点/注意：`print('A')` 会因整型提升选中 `int` 版本；字面量 `0`、`NULL`、`nullptr` 在重载下可能匹配到不同的函数，混用易产生意外。

五、inline 函数

inline 函数

概念：`inline` 建议编译器把函数体在调用处展开，从而省去函数调用开销。它适合短小、频繁调用的函数；`inline` 变量 (C++17) 让头文件中的全局变量在所有翻译单元共享同一实体。

要点：

- `inline` 是建议而非强制，编译器可自行决定是否内联。
- 现代编译器会自动内联简单函数，显式 `inline` 更多用于"头文件中定义函数"时避免重复定义。
- 定义在头文件且被多个翻译单元包含的函数必须 `inline`。
- `inline` 变量 (C++17)：头文件里的 `inline` 变量只存在一份实体，避免多文件重复定义。
- 递归函数不适合内联。

示例：

```
#include <iostream>

inline int square(int x) { return x * x; } // 短小函数适合内联

// ---- header.hpp ----
// inline int shared = 42; // C++17: 头文件中的 inline 变量, 全局唯一

int main() {
    int v = 5;
    std::cout << square(v) << '\n'; // 25
}
```

易错点/注意：若把普通（非 inline、非 static）函数定义放在头文件并被多个 `.cpp` 包含，链接时会报“多重定义”；`inline` 与 ODR 配合是头文件放定义的正确方式。

六、函数指针与回调

函数指针与回调

概念：函数指针存储函数的地址，可把函数作为参数传递，实现回调机制。成员函数指针需要对象才能调用。`std::function` 是更通用、类型安全的替代。

要点：

- 普通函数指针类型：返回类型 (*指针名)(参数列表)；函数名会退化为指针。
- 用 `using/typedef` 简化函数指针类型，提高可读性。
- 成员函数指针：返回类型 (类::*名)(参数)，需通过 对象.*指针 或 对象指针->*指针 调用。
- C 的 `qsort` 用函数指针比较器；C++ 的 `std::sort` 用模板/函数对象，类型安全且更快。

示例：

```
#include <iostream>
#include <cstdlib>
#include <algorithm>
#include <vector>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

using BinaryOp = int (*)(int, int);           // using 简化函数指针

int apply(BinaryOp op, int x, int y) { return op(x, y); } // 回调

int cmp(const void* a, const void* b) {       // qsort 比较器
    return *(const int*)a - *(const int*)b;
}

struct Widget {
    int value = 0;
    void set(int v) { value = v; }
};

int main() {
    BinaryOp op = add;
    std::cout << apply(op, 3, 4) << '\n';      // 7
    std::cout << apply(sub, 3, 4) << '\n';     // -1

    int arr[] = {3, 1, 2};
    std::qsort(arr, 3, sizeof(int), cmp);      // C 风格
    std::vector<int> v{3, 1, 2};
    std::sort(v.begin(), v.end());             // C++ 风格
    std::cout << arr[0] << ' ' << v[0] << '\n'; // 1 1

    void (Widget::*setter)(int) = &Widget::set; // 成员函数指针
    Widget w;
    (w.*setter)(42);                           // 通过对象调用
    std::cout << "w.value=" << w.value << '\n'; // 42
}
```

示例（函数指针数组与回调）：

```
#include <iostream>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

using Op = int (*)(int, int);

int main() {
    Op ops[] = { add, sub, mul };           // 函数指针数组
    int a = 6, b = 3;
    for (Op op : ops)
        std::cout << op(a, b) << ' ';    // 9 3 18
    std::cout << '\n';
}
```

易错点/注意：成员函数指针不是普通指针，不能用 `int (*)(int)` 存储；取成员函数地址必须写成 `&类名::函数名`；`qsort` 的比较器不能直接使用 lambda（除非无捕获）。

七、lambda 表达式

lambda 表达式

概念：lambda 是匿名函数对象，完整语法为 `[捕获](参数) mutable -> 返回类型 { 函数体 }`。它可捕获外部变量，广泛用于算法、回调与一次性逻辑。

要点：

- 捕获列表：`=` 按值捕获、`&` 按引用捕获、`this` 捕获 `this` 指针、`*this` (C++17) 捕获对象副本。
- `mutable` 允许修改按值捕获的副本；`->` 返回类型 可省略让编译器推导。
- 泛型 lambda (C++14)：参数用 `auto` 声明。
- 立即调用 lambda：定义后紧跟 `()` 立即执行。
- lambda 本质是编译器生成的、重载了 `operator()` 的闭包对象。

示例：

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    int base = 10;
    auto add = [base](int x) { return base + x; };           // 值捕获
    auto addRef = [&base](int x) { return base + x; };       // 引用捕获
    auto generic = [](auto a, auto b) { return a + b; };     // C++14 泛型 lambda

    base = 100;
    std::cout << add(1) << ' ' << addRef(1) << '\n';         // 11 101

    auto counter = [n = 0]() mutable { return ++n; };       // mutable 修改捕获副本
    std::cout << counter() << ' ' << counter() << '\n';       // 1 2

    int total = [](int x, int y) { return x * y; }(6, 7);   // 立即调用
    std::cout << "立即调用 = " << total << '\n';             // 42
}
```

```
std::vector<int> v{5, 3, 8, 1};
std::sort(v.begin(), v.end(),
          [](int a, int b) { return a > b; }); // 降序
for (int x : v) std::cout << x << ' '; // 8 5 3 1
std::cout << '\n';
}
```

示例 (捕获 this 与 *this) :

```
#include <iostream>

struct Counter {
    int n = 0;
    auto makeLambda() {
        // 捕获 this: lambda 通过指针访问成员
        return [this]() { return ++n; };
    }
    auto makeCopyLambda() {
        // C++17 捕获 *this: 拷贝当前对象, 修改不影响原对象
        return [*this]() mutable { return ++n; };
    }
};

int main() {
    Counter c;
    auto f = c.makeLambda();
    f(); f();
    std::cout << "原对象 n = " << c.n << '\n'; // 2

    auto g = c.makeCopyLambda(); // 拷贝时 n=2
    g(); // 只改副本
    std::cout << "拷贝后原对象 n = " << c.n << '\n'; // 仍为 2
}
```

易错点/注意: 按值捕获的变量默认只读, 需 `mutable` 才能修改副本; 按引用捕获需保证被捕获对象的生命周期覆盖 lambda 的调用时刻; `[=]` 在旧规则下会隐式捕获 `this`, 注意对象生命周期。

八、std::function 与 std::bind

std::function 与 std::bind、占位符

概念: `std::function` 是可存储任意可调用对象 (函数、函数对象、lambda) 的类型擦除包装器; `std::bind` 可绑定部分参数或调整参数顺序, 配合 `placeholders` 占位符使用。

要点:

- `std::function<返回类型(参数)>` 可容纳函数指针、lambda、仿函数。
- `std::bind(可调用对象, 参数...)` 返回新的可调用对象。
- `placeholders::_1`、`_2` 表示调用时的第 1、2 个实参。
- 现代 C++ 更推荐用 lambda 替代 `std::bind` (更清晰、可内联、无额外开销)。

示例:

```
#include <iostream>
#include <functional>
```

```

int add(int a, int b) { return a + b; }

struct Multiplier {
    int factor;
    int operator()(int x) const { return x * factor; } // 仿函数
};

int main() {
    std::function<int(int, int)> f = add;
    std::cout << "function: " << f(3, 4) << '\n'; // 7

    f = [](int a, int b) { return a - b; }; // 存储 lambda
    std::cout << "lambda: " << f(3, 4) << '\n'; // -1

    using namespace std::placeholders; // _1, _2 ...
    auto add10 = std::bind(add, _1, 10); // 绑定第二参数为 10
    std::cout << "bind: " << add10(5) << '\n'; // 15

    auto swapArg = std::bind(add, _2, _1); // 交换参数顺序
    std::cout << "交换: " << swapArg(1, 2) << '\n'; // 3

    Multiplier m{3};
    std::function<int(int)> g = m; // 存储仿函数
    std::cout << "仿函数: " << g(4) << '\n'; // 12
}

```

示例（绑定成员函数与引用）：

```

#include <iostream>
#include <functional>

struct Adder {
    int base = 0;
    int add(int x) { base += x; return base; }
};

int main() {
    using namespace std::placeholders;

    Adder a;
    // 绑定成员函数：第二个实参传入对象指针
    auto addTo = std::bind(&Adder::add, &a, _1);
    std::cout << addTo(5) << '\n'; // 5
    std::cout << addTo(5) << '\n'; // 10

    int n = 1;
    auto inc = [](int& x) { ++x; };
    auto bound = std::bind(inc, std::ref(n)); // std::ref 按引用绑定
    bound();
    std::cout << "n = " << n << '\n'; // 2
}

```

易错点/注意： `std::function` 有间接调用（虚派发）开销，性能敏感处慎用；`std::bind` 的占位符可读性差，优先用 `lambda`；`std::bind` 绑定引用需配合 `std::ref`/`std::cref`。

九、递归与尾递归

递归与尾递归

概念：递归是函数直接或间接调用自身，需要基线条件（终止条件）与递归步骤。尾递归是递归调用出现在函数最后一步返回表达式中的形式，编译器可将其优化为迭代以避免栈溢出。

要点：

- 递归必须包含基线条件，否则无限递归。
- 阶乘、斐波那契是经典递归示例。
- 递归开销：每次调用占用栈帧，深度过大导致栈溢出（stack overflow）。
- 尾递归优化 (TCO) 并非 C++ 标准强制保证，取决于编译器与优化级别。
- 大量线性递归可用迭代替代，效率更高。

示例：

```
#include <iostream>

int factorial(int n) {                // 普通递归
    if (n <= 1) return 1;             // 基线条件
    return n * factorial(n - 1);
}

int fib(int n) {                      // 指数级递归，效率低
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

int factorialTail(int n, int acc = 1) { // 尾递归形式
    if (n <= 1) return acc;
    return factorialTail(n - 1, acc * n); // 尾调用
}

int fibIter(int n) {                  // 迭代替代
    int a = 0, b = 1;
    for (int i = 0; i < n; ++i) { int t = a + b; a = b; b = t; }
    return a;
}

int main() {
    std::cout << factorial(5) << '\n';    // 120
    std::cout << fib(10) << '\n';         // 55
    std::cout << factorialTail(5) << '\n'; // 120
    std::cout << fibIter(10) << '\n';     // 55
}
```

示例（记忆化与递归二分查找）：

```
#include <iostream>
#include <unordered_map>

long long fibMemo(int n, std::unordered_map<int, long long>& memo) {
    if (n <= 1) return n;
    if (memo.count(n)) return memo[n]; // 命中缓存
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}
```

```

}

int binarySearch(const int arr[], int l, int r, int target) {
    if (l > r) return -1; // 基线: 未找到
    int mid = l + (r - l) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] > target) return binarySearch(arr, l, mid - 1, target);
    return binarySearch(arr, mid + 1, r, target);
}

int main() {
    std::unordered_map<int, long long> memo;
    std::cout << fibMemo(40, memo) << '\n'; // 102334155
    int arr[] = {1, 3, 5, 7, 9, 11};
    std::cout << binarySearch(arr, 0, 5, 7) << '\n'; // 3
}

```

易错点/注意: 递归缺少或写错终止条件会无限递归直至栈溢出; 朴素斐波那契是指数复杂度 $O(2^n)$, 应用记忆化或迭代; 尾递归优化需开启优化选项 (如 `-O2`) 才可能生效。

十、拷贝省略与返回值优化

拷贝省略与返回值优化 (RVO/NRVO)

概念: 编译器可以省略函数返回值的拷贝, 直接在调用者的目标存储位置构造对象。RVO (返回匿名临时对象) 与 NRVO (返回具名对象) 是两种形式; (C++17) 起部分场景的拷贝省略成为强制。

要点:

- RVO: `return T(...)` 返回匿名临时对象, 直接构造到目标位置。
- NRVO: 返回局部具名对象, 编译器可优化但非强制。
- (C++17) 强制拷贝省略: 返回 prvalue 时保证不调用拷贝/移动构造函数。
- 现代 C++ 应放心按值返回, 不必手动优化; 切勿返回局部对象的引用或指针。

示例:

```

#include <iostream>

struct Big {
    int data[100];
    Big() { std::cout << "构造\n"; }
    Big(const Big&) { std::cout << "拷贝构造\n"; }
    Big& operator=(const Big&) { std::cout << "拷贝赋值\n"; return *this; }
};

Big makeRVO() {
    return Big(); // RVO: 返回临时对象
}

Big makeNRVO() {
    Big b; // 具名对象
    return b; // NRVO: 可能被省略
}

int main() {
    Big a = makeRVO(); // 理想: 只打印一次"构造"
    Big b = makeNRVO(); // NRVO 生效时同样只打印一次
}

```

```
(void)a; (void)b;
}
```

示例（移动语义配合）：

```
#include <iostream>

struct Big {
    int* p = nullptr;
    Big() { p = new int[100]; std::cout << "构造\n"; }
    Big(const Big&) { p = new int[100]; std::cout << "拷贝构造\n"; }
    Big(Big&& o) noexcept : p(o.p) { o.p = nullptr; std::cout << "移动构造\n"; }
    ~Big() { delete[] p; }
};

Big make() {
    Big b;
    b.p[0] = 42;
    return b;           // NRVO 或移动构造，避免深拷贝
}

int main() {
    Big a = make();     // 理想：只构造一次
}
```

易错点/注意：不要返回局部对象的引用或指针（悬垂引用）；NRVO 不是强制的，析构/拷贝有副作用时要留意；(C++17) 之前的拷贝省略是可选优化，之后部分成为强制。

十一、C 风格可变参数

C 风格可变参数 ()

概念：通过 `<cstdarg>` 中的 `va_list`、`va_start`、`va_arg`、`va_end` 宏可实现参数个数不定的函数。它是 C 遗留机制，类型不安全，现代 C++ 通常用可变参数模板或 `std::initializer_list` 替代。

要点：

- 必须至少有一个具名参数来确定可变参数的起始位置。
- `va_start` 初始化，`va_arg` 逐个取出，`va_end` 清理。
- 类型不安全：调用方需另行告知参数的类型与个数（如 `printf` 的格式串）。
- 默认实参提升：`float` 提升为 `double`、`char` 提升为 `int`。

示例：

```
#include <iostream>
#include <cstdarg>

int sum(int count, ...) {           // count 告知参数个数
    va_list args;
    va_start(args, count);          // 从 count 之后开始
    int total = 0;
    for (int i = 0; i < count; ++i)
        total += va_arg(args, int); // 按 int 逐个取出
    va_end(args);                    // 清理
    return total;
}
```

```

}

int main() {
    std::cout << sum(3, 1, 2, 3) << '\n';           // 6
    std::cout << sum(5, 1, 2, 3, 4, 5) << '\n';      // 15
}

```

缺点：

- 类型不安全，无编译期检查，类型不匹配会读到错误数据。
- 无法自动获知参数个数与类型，必须靠约定。
- 参数发生默认实参提升，取错类型会出错。

示例（模拟 printf 的格式解析）：

```

#include <iostream>
#include <cstdarg>

void myprintf(const char* fmt, ...) {
    va_list args;
    va_start(args, fmt);
    for (const char* p = fmt; *p; ++p) {
        if (*p == '%' && *(p + 1) == 'd') { // 仅支持 %d
            std::cout << va_arg(args, int);
            ++p; // 跳过 'd'
        } else {
            std::cout << *p;
        }
    }
    va_end(args);
}

int main() {
    myprintf("值1=%d, 值2=%d\n", 10, 20); // 值1=10, 值2=20
}

```

易错点/注意： float 实参会提升为 double，此时 va_arg(..., float) 是错误用法；漏掉 va_end 是未定义行为；可变参数是运行期解析，无法获得编译期类型安全。

十二、函数对象/仿函数

函数对象/仿函数 (operator())

概念： 函数对象（仿函数）是重载了 operator() 的类的实例，可以像函数一样调用，同时通过成员变量携带状态，比函数指针更灵活。

要点：

- 重载 operator() 使对象可调用。
- 可携带状态：系数、计数器等成员变量。
- 相比函数指针：类型安全、可被内联、可有状态。
- 标准库算法（如 for_each、sort）广泛接受函数对象。

示例：

```

#include <iostream>
#include <vector>

```

```
#include <algorithm>

struct Add {
    int n;
    explicit Add(int v) : n(v) {}
    int operator()(int x) const { return x + n; } // 重载 operator()
};

struct Count {
    int total = 0;
    void operator()(int x) { total += x; } // 有状态仿函数
};

int main() {
    Add add5(5);
    std::cout << add5(10) << '\n'; // 15

    std::vector<int> v{1, 2, 3, 4};
    Count c = std::for_each(v.begin(), v.end(), Count{});
    std::cout << "总和 = " << c.total << '\n'; // 10
}
```

示例（仿函数用于排序）：

```
#include <iostream>
#include <vector>
#include <algorithm>

struct Desc {
    bool operator()(int a, int b) const { return a > b; } // 降序比较
};

int main() {
    std::vector<int> v{5, 1, 4, 2, 3};
    std::sort(v.begin(), v.end(), Desc()); // 函数对象
    for (int x : v) std::cout << x << ' '; // 5 4 3 2 1
    std::cout << '\n';

    // lambda 等价写法
    std::sort(v.begin(), v.end(), [](int a, int b) { return a < b; });
    for (int x : v) std::cout << x << ' '; // 1 2 3 4 5
    std::cout << '\n';
}
```

易错点/注意：函数对象按值传给算法时状态可能被复制，`for_each` 返回的副本才包含最终状态；`lambda` 本质就是编译器生成的函数对象，两者能力等价。

十三、函数重载决议与参数匹配进阶

函数重载决议与参数匹配进阶

概念：重载决议在多个候选函数中选出最佳匹配。编译器按“精确匹配 → 提升 → 标准转换 → 用户定义转换”的优先级排序；`explicit` 构造函数禁止隐式转换，从而影响决议结果。

要点：

- 隐式转换序列分为三个等级：精确匹配、提升、标准转换。

- 用户定义转换（构造函数/转换函数）优先级最低。
- `explicit` 构造函数禁止隐式调用，避免意外的隐式转换。
- 多个候选不分伯仲时产生二义性，编译报错。

示例：

```
#include <iostream>

void f(int)    { std::cout << "f(int)\n"; }
void f(double) { std::cout << "f(double)\n"; }
void g(long)   { std::cout << "g(long)\n"; }
void g(double) { std::cout << "g(double)\n"; }

struct Widget {
    explicit Widget(int) {}    // explicit: 禁止隐式转换
};

void h(Widget) { std::cout << "h(Widget)\n"; }

int main() {
    f('A');           // char 提升为 int → f(int)
    f(1.0f);          // float 提升为 double → f(double)
    // g(1);           // 二义性: int→long 与 int→double 同级标准转换

    Widget w(1);
    h(w);             // 精确匹配
    // h(1);           // 错误: explicit 禁止隐式构造 Widget
}
```

易错点/注意：整型到浮点与整型到另一整型同属"标准转换"，会二义性；`explicit` 只禁止隐式转换，显式写法 `Widget(1)` 依然合法。

本部分小结

易忘点	说明
声明与定义默认参数	只能出现一次，且从右向左
仅返回类型不同	不构成重载
临时对象绑定引用	只能绑定 <code>const</code> 左值引用
头文件中函数定义	必须 <code>inline</code> ，否则多重定义
成员函数指针	需 <code>&类::函数</code> ，通过对象或指针调用
值捕获默认只读	需 <code>mutable</code> 才能改副本
递归无基线	栈溢出；斐波那契应用迭代/记忆化
返回局部对象引用	悬垂引用，应返回值并依赖 RVO
<code>va_arg</code> 类型	<code>float</code> 已提升为 <code>double</code> ，取错类型出错
<code>explicit</code> 构造	禁止隐式转换，重载决议中避免意外匹配

清单总结：优先现代 C++（按 const 引用传递大型对象、lambda 替代函数指针与 bind、按值返回依赖 RVO）；理解重载决议的转换等级；警惕默认参数与重载的二义性；避免 C 风格可变参数。

第三部分：指针、引用与内存管理

指针基础

指针基础

概念：指针是一种专门用来存放"内存地址"的变量。通过地址可以直接读写该地址处的对象，这是 C++ 强大的底层能力，也是内存错误的常见来源。C++11 引入 `nullptr` 作为空指针字面量，取代有歧义的 `NULL` 与 `0`。

要点：

- 定义：`int* p;` 声明一个指向 `int` 的指针；`int a = 10; int* p = &a;` 用 `&` 取地址初始化。
- 解引用：`*p` 表示"p 指向的那个对象"，可读也可写。
- 指针大小：在给定平台上所有对象指针大小相同（64 位平台通常为 8 字节），与所指类型无关。
- `nullptr` (C++11)：类型为 `std::nullptr_t`，可安全转换为任意指针类型，不参与整型重载歧义。
- `NULL` 通常被定义为 `0`，`0` 是整型常量，参与函数重载时可能误匹配到整型重载版本。

示例：

```
#include <iostream>

int main() {
    int a = 10;           // 普通变量
    int* p = &a;          // & 取地址，p 保存 a 的地址
    std::cout << "a 的值: " << a << "\n";
    std::cout << "a 的地址: " << &a << "\n";
    std::cout << "p 保存的地址: " << p << "\n";
    std::cout << "*p 解引用: " << *p << "\n";    // 通过指针读 a

    *p = 20;              // 通过指针写 a
    std::cout << "修改后 a = " << a << "\n";    // 输出 20

    int* q = nullptr;     // 空指针（推荐写法）
    if (q == nullptr) { std::cout << "q 是空指针\n"; }

    std::cout << "sizeof(int*) = " << sizeof(int*) << "\n"; // 8 (64位)
    return 0;
}
```

易错点/注意：

- `int* p, q;` 只有 `p` 是指针，`q` 是普通 `int`；想要两个指针应写成 `int *p, *q;`。
- 解引用空指针或未初始化指针是未定义行为，程序可能崩溃。
- 未初始化的指针（如 `int* p;`）保存的是垃圾地址，称为"野指针"来源之一。

指针与数组

指针与数组

概念：数组名在大多数表达式中会"退化"为指向首元素的指针（decay）。因此指针与数组关系密切：指针算术、下标访问与数组下标访问在语义上等价。数组名本身不是可修改的左值，不能整体赋值或自增。

要点：

- 数组名退化为首元素地址：`int arr[5];` 中 `arr` 等价于 `&arr[0]`。
- 指针运算：`p + n` 移动的是 `n * sizeof(元素类型)` 个字节，而非 `n` 个字节。
- 下标等价：`arr[i]` 等价于 `*(arr + i)`，也等价于 `*(i + arr)` 和 `i[arr]`。
- 例外情况：`sizeof(arr)` 得到整个数组大小、`&arr` 得到指向"整个数组"的指针，此时不退化。

示例：

```
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* p = arr;                // 数组名退化为首元素指针

    std::cout << "p[2] = " << p[2] << "\n";        // 30
    std::cout << "*(p+2) = " << *(p + 2) << "\n";    // 30
    std::cout << "*(arr+2) = " << *(arr + 2) << "\n"; // 30
    std::cout << "2[arr] = " << 2[arr] << "\n";      // 30（等价写法）

    for (int* it = arr; it != arr + 5; ++it) {      // 指针遍历
        std::cout << *it << ' ';
    }
    std::cout << "\nsizeof(arr) = " << sizeof(arr) << "\n"; // 20（整数组）
    std::cout << "sizeof(p) = " << sizeof(p) << "\n";      // 8（指针）
    return 0;
}
```

易错点/注意：

- `int* p = arr;` 后 `sizeof(p)` 是指针大小，不是数组大小，`sizeof(arr)` 才是整个数组。
- 把数组当参数传给函数时退化为指针，函数内 `sizeof` 只能得到指针大小，必须额外传长度。

多级指针、指针数组、数组指针、函数指针数组

多级指针

概念：指针也可以指向指针，形成多级指针。`int** pp` 指向 `int*`，`int*** ppp` 指向 `int**`。多级指针常见于需要"修改指针本身"的函数参数，以及字符串数组等场景。

要点：

- `int** pp = &p;` 中 `*pp` 是 `p` 本身，`**pp` 是 `p` 指向的 `int`。
- 通过二级指针可以在函数内修改外部的一级指针（指向新地址）。
- 层级越深越难读，超过三级指针通常意味着设计需要重构。

示例：

```
#include <iostream>

void newTarget(int** pp) {    // 通过二级指针修改外部一级指针
```

```

static int x = 100;
*pp = &x;           // 修改外部指针指向
}

int main() {
    int a = 5;
    int* p = &a;
    int** pp = &p;    // 二级指针

    std::cout << "**pp = " << **pp << "\n"; // 5
    newTarget(&p);    // 传入 p 的地址
    std::cout << "*p = " << *p << "\n";    // 100
    return 0;
}

```

易错点/注意：解引用层级必须与指针层级对应：`pp` 是地址、`*pp` 是一级指针、`**pp` 才是值；写错一层就会类型不匹配或崩溃。

指针数组

概念：指针数组是"元素为指针的数组"，即 `T* arr[N]`。常用于存放多个字符串（`char* arr[]`）或一组对象的指针。

要点：

- 定义形式：`int* arr[3]`；先与 `[3]` 结合成数组，元素类型为 `int*`。
- 字符串指针数组：`const char* names[] = {"A", "B", "C"};` 每个元素指向字符串字面量。
- 适合管理"多个独立对象"的句柄，本身只存指针，不拥有对象。

示例：

```

#include <iostream>

int main() {
    int a = 1, b = 2, c = 3;
    int* ptrs[3] = {&a, &b, &c};    // 指针数组：元素是指针
    for (int i = 0; i < 3; ++i) {
        std::cout << *ptrs[i] << ' ';    // 解引用每个指针
    }
    std::cout << "\n";

    const char* names[] = {"Alice", "Bob", "Cindy"}; // 字符串指针数组
    std::cout << names[1] << "\n";    // Bob
    return 0;
}

```

易错点/注意：区分"指针数组" `int* a[3]`（数组，元素是指针）与"数组指针" `int (*p)[3]`（一个指针，指向数组）。

数组指针

概念：数组指针是指向"整个数组"的指针，声明为 `T (*p)[N]`。括号保证 `p` 先与 `*` 结合。它指向的是数组整体，常用于二维数组的行遍历。

要点：

- 定义：`int (*p)[3]`；表示 `p` 指向"包含 3 个 `int` 的数组"。

- 用法: `int a[2][3]; int (*p)[3] = a;` 此时 `p` 指向第一行, `p+1` 指向第二行 (步长为 3 个 `int`)。
- `&arr` (对一维数组取地址) 得到的也是数组指针。

示例:

```
#include <iostream>

int main() {
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int (*p)[3] = a;           // p 指向"3个int"的数组

    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 3; ++j) {
            std::cout << p[i][j] << ' '; // 等价于 a[i][j]
        }
    }
    std::cout << "\n";

    int arr[4] = {1, 2, 3, 4};
    int (*q)[4] = &arr;       // 指向整个一维数组
    std::cout << "(*q)[2] = " << (*q)[2] << "\n"; // 3
    return 0;
}
```

易错点/注意: `int *p[3]` 与 `int (*p)[3]` 含义完全不同, 括号位置决定结合顺序, 务必记牢。

函数指针数组

概念: 函数指针用于保存函数地址, 函数指针数组则把多个"同签名函数"组织成数组, 便于按索引调度 (如表驱动、回调分发)。C++11 后可用 `std::function` 或 `using` 别名降低复杂度。

要点:

- 单个函数指针: `int (*fp)(int, int);`, 赋值 `fp = add;`, 调用 `fp(1, 2)` 或 `(*fp)(1, 2)`。
- 函数指针数组: `int (*ops[4])(int, int);` 元素是指向函数的指针。
- 用 `using Op = int (*)(int, int);` 可让写法更清晰: `Op ops[4] = {...};`。

示例:

```
#include <iostream>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

using Op = int (*)(int, int); // 函数指针类型别名

int main() {
    Op ops[3] = {add, sub, mul}; // 函数指针数组
    int x = 6, y = 2;
    for (int i = 0; i < 3; ++i) {
        std::cout << ops[i](x, y) << ' '; // 8 4 12
    }
    std::cout << "\n";
    return 0;
}
```

```
}
```

易错点/注意：函数指针的返回类型与参数列表必须完全匹配；`std::function<int(int,int)>` 可接受函数、lambda、可调用对象，是更现代的替代。

const 与指针

const 与指针

概念：`const` 与指针组合有三种形式，区别在于“`const` 修饰的是指针本身还是指针指向的对象”。判别技巧：从右往左读，`const` 修饰它左边最近的类型；若 `const` 在最左，则修饰其右边的类型。

要点：

- `const T* p`（或 `T const* p`）：指向常量，不能通过 `p` 修改对象，但 `p` 可指向别处。
- `T* const p`：指针本身是常量，不能改指向，但可以通过 `p` 修改对象。
- `const T* const p`：两者都不能改。
- 判别技巧：看 `*` 与 `const` 的相对位置——`const` 在 `*` 左边修饰所指对象，在 `*` 右边修饰指针本身。

示例：

```
#include <iostream>

int main() {
    int a = 10, b = 20;

    const int* p = &a;    // 指向常量的指针
    // *p = 30;           // 错误：不能改所指对象
    p = &b;               // 正确：指针可改指向
    std::cout << "*p = " << *p << "\n";

    int* const q = &a;    // 常量指针
    *q = 30;              // 正确：可改所指对象
    // q = &b;            // 错误：不能改指向
    std::cout << "a = " << a << "\n";

    const int* const r = &a; // 指向常量的常量指针
    // *r = 40;            // 错误
    // r = &b;             // 错误
    return 0;
}
```

易错点/注意：

- 不可把 `const T*` 隐式转换为 `T*`（丢失 `const`），但 `T*` 可以转为 `const T*`。
- 顶层 `const`（修饰指针本身）在函数重载/类型推导中与底层 `const` 规则不同，需区分。

引用

左值引用

概念：引用（左值引用）是已有对象的别名，声明为 `T& r = obj;`。引用必须在定义时初始化，一旦绑定终生不可更换对象；对引用的操作等价于对原对象的操作。

要点：

- 引用必须在声明时初始化，不存在"空引用"。
- 引用不是对象，不占独立存储（编译器通常用指针实现，但语义上是别名）。
- 引用不可重新绑定；`int& r = a; r = b;` 是把 `b` 的值赋给 `a`，而非让 `r` 绑定 `b`。
- 常量引用 `const T&` 可以绑定右值（临时对象），延长其生命周期，常用于函数参数。

示例：

```
#include <iostream>

int main() {
    int a = 10;
    int& r = a;           // r 是 a 的别名
    r = 20;
    std::cout << "a = " << a << "\n"; // 20

    int b = 5;
    r = b;                // 这是赋值，不是重新绑定
    std::cout << "a = " << a << "\n"; // 5 (a 被改, r 仍绑定 a)

    const int& cr = 42;    // const 引用绑定临时量
    std::cout << "cr = " << cr << "\n";
    return 0;
}
```

易错点/注意：返回局部变量的引用会造成"悬垂引用"；用 `const T&` 作参数可避免拷贝并接受右值。

引用与指针对比

概念：引用与指针都能间接访问对象，但语义不同：引用是"别名"，指针是"保存地址的变量"。两者有各自的适用场景与限制。

要点：

- 引用必须初始化、不能为空、不可重新绑定；指针可为空、可重新指向。
- 引用解引用无需 `*`，指针需要。
- 引用没有"引用的引用"，指针有"指针的指针"。
- 引用的 `sizeof` 得到被引用对象大小，指针的 `sizeof` 是地址大小。

示例：

```
#include <iostream>

int main() {
    int a = 10;
    int& r = a;           // 引用：别名
    int* p = &a;          // 指针：地址

    r += 5;                // 等价于 a += 5
    *p += 5;               // 通过指针修改
    std::cout << "a = " << a << "\n"; // 20
}
```

```

int b = 1;
p = &b;           // 指针可以重新指向
// r = b;         // 引用不能重新绑定，这是赋值
std::cout << "sizeof(r) = " << sizeof(r) << "\n"; // 4
std::cout << "sizeof(p) = " << sizeof(p) << "\n"; // 8
return 0;
}

```

易错点/注意：需要"可能为空"或"需要换对象"时用指针；需要"必然绑定某对象、语法更简洁"时用引用。

引用作为返回值

概念：函数可以返回引用，从而让调用方直接操作函数内部引用的对象，避免拷贝。典型场景是重载 `operator[]`、返回容器元素、链式调用（如 `cout`）。必须保证返回的引用指向的对象在函数返回后仍然存活。

要点：

- 返回静态变量、全局变量、传入的引用参数、`*this` 等生命周期足够长的对象是安全的。
- 严禁返回局部变量或临时对象的引用（悬垂引用）。
- 返回引用可实现左值链式调用：`obj.f().g()` 或 `a[i] = 3`。

示例：

```

#include <iostream>

int& maxRef(int& x, int& y) { // 返回引用
    return x > y ? x : y;
}

int arr[3] = {1, 2, 3};
int& at(int i) { return arr[i]; } // 返回数组元素引用

int main() {
    int a = 5, b = 9;
    maxRef(a, b) = 100;           // 直接修改较大的那个
    std::cout << "b = " << b << "\n"; // 100

    at(1) = 20;                   // 修改 arr[1]
    std::cout << "arr[1] = " << arr[1] << "\n"; // 20
    return 0;
}

```

易错点/注意：

```

int& bad() { int x = 1; return x; } // 错误：返回局部变量引用，悬垂！

```

悬垂引用在 `-O2` 下可能"看似正常"，属未定义行为，难以排查。

悬垂引用场景

概念：悬垂引用指引用绑定的对象已被销毁，引用本身变成"指向已释放内存"的别名。解引用它会触发未定义行为。认识这些场景是写出安全代码的关键。

要点：

- 返回局部变量/局部对象的引用。
- 引用绑定的容器元素在容器扩容或元素被删除后失效。
- 临时对象的非 `const` 引用在语句结束后失效（`const T&` 会延长生命周期）。
- 迭代器/引用因容器操作失效后继续使用，本质是同类问题。

示例：

```
#include <iostream>
#include <string>
#include <vector>

const std::string& pick(bool flag) {
    static std::string a = "A";    // 静态对象生命周期贯穿整个程序
    std::string b = "B";          // 局部对象，函数结束即销毁
    return flag ? a : b;          // 返回 b 时悬垂！
}

int main() {
    const std::string& r = pick(false); // 悬垂引用
    // std::cout << r << "\n";      // 未定义行为，切勿使用
    return 0;
}
```

易错点/注意：`const T&` 绑定函数返回的临时对象可延长其生命周期，但绑定"函数返回的引用"不延长；只有直接绑定纯右值才延长。

new/delete 与 new[]/delete[]

new/delete 与 new[]/delete[]

概念：`new` 在堆上动态分配内存并调用构造函数，`delete` 调用析构函数并释放内存；数组版本 `new[]` / `delete[]` 必须配对。C++ 还提供 `nothrow new` 和 `placement new` 等变体。

要点：

- `new T` 分配单个对象并返回 `T*`；`new T[n]` 分配数组，返回首元素指针。
- `delete p` 释放单个对象；`delete[] p` 释放数组，两者不可混用。
- `new` 失败默认抛 `std::bad_alloc`；`new (std::nothrow)` 失败返回 `nullptr`。
- `placement new`：`new (地址) T(...)` 在已分配的内存上构造对象，不分配内存，需手动调用析构。
- 现代 C++ 应优先用智能指针和容器替代裸 `new/delete`。

示例：

```
#include <iostream>
#include <new>

struct Widget {
    int v;
```

```

widget(int x) : v(x) { std::cout << "构造 " << v << "\n"; }
~widget() { std::cout << "析构 " << v << "\n"; }
};

int main() {
    widget* p = new widget(1);    // 单个对象
    delete p;                    // 必须 delete

    widget* arr = new widget[3]{widget(1), widget(2), widget(3)}; // 数组
    delete[] arr;                // 必须 delete[]

    widget* q = new (std::nothrow) widget(9); // nothrow 版
    if (q) { delete q; }           // 失败返回 nullptr，需判空

    // placement new: 在已有缓冲区上构造
    alignas(widget) unsigned char buf[sizeof(widget)];
    widget* pw = new (buf) widget(7); // 在 buf 上构造
    pw->~widget();                    // 手动析构，不能 delete
    return 0;
}

```

易错点/注意：

- new 与 delete[] 混用、new[] 与 delete 混用都是未定义行为。
- placement new 构造的对象不能 delete（内存不是 new 来的），只能显式析构。
- 忘记 delete 造成内存泄漏；delete 两次造成重复释放。

内存分区模型

内存分区模型

概念：C++ 程序运行时内存通常划分为几个区域：栈、堆、全局/静态区、常量区、代码区。理解每个区域的生命周期与存储内容，有助于判断对象何时创建、何时销毁、能否返回指针/引用。

要点：

- 栈：局部变量、函数参数，由编译器自动分配释放，速度快、空间有限。
- 堆：new/malloc 动态分配，程序员手动（或由智能指针）释放，生命周期可跨越函数。
- 全局/静态区：全局变量、static 变量，程序启动分配、结束释放。
- 常量区：字符串字面量等常量，只读，不可修改。
- 代码区：存放编译后的机器指令。

示例：

```

#include <iostream>

int g = 1;           // 全局/静态区
static int sg = 2;   // 全局/静态区
const char* msg = "hi"; // msg 在静态区，指向常量区的 "hi"

int* make() {
    static int s = 3; // 静态局部：函数结束仍存活
    int local = 4;    // 栈：函数结束销毁
    int* heap = new int(5); // 堆：手动释放
    return heap;      // 安全（堆）
    // return &local; // 危险（栈）
}

```

```

int main() {
    int a = 6;           // 栈
    int* hp = make();    // hp 指向堆
    std::cout << *hp << "\n";
    delete hp;          // 释放堆内存

    const char* s1 = "hi"; // 常量区（只读）
    // s1[0] = 'H';        // 错误：修改常量区是未定义行为
    return 0;
}

```

易错点/注意：栈空间较小（默认几 MB），递归过深或大数组放栈上可能栈溢出；堆空间大但需手动管理、易泄漏。字符串字面量不可写，应使用 `const char*` 指向。

内存错误专题

内存错误专题

概念：C++ 内存错误是隐蔽且危险的 bug 来源，主要包括内存泄漏、悬垂指针、野指针、重复释放、缓冲区溢出。理解每种错误的成因与表现，配合检测工具，是工程化的必备素养。

要点：

- 内存泄漏：`new` 后未 `delete`，内存只增不减；程序长期运行耗尽内存。
- 悬垂指针：指向已释放内存的指针，解引用是未定义行为。
- 野指针：从未初始化、或已越界的指针，值不可预测。
- 重复释放：同一指针 `delete` 两次，通常崩溃。
- 缓冲区溢出：越界读写数组，破坏相邻内存，可能被利用为安全漏洞。
- 检测手段：`-fsanitize=address` (AddressSanitizer)、`valgrind`、静态分析、智能指针。

示例：

```

#include <iostream>

int main() {
    // 1. 内存泄漏
    int* leak = new int(1); // 未 delete, 泄漏
    delete leak;           // 修复：释放

    // 2. 悬垂指针
    int* p = new int(2);
    delete p;              // 内存已释放
    // *p = 3;             // 危险：p 已悬垂

    // 3. 重复释放
    int* q = new int(4);
    delete q;
    // delete q;           // 危险：重复释放

    // 4. 缓冲区溢出
    int arr[3] = {0, 0, 0};
    // arr[3] = 1;         // 危险：越界写（第4个元素不存在）
    return 0;
}

```

易错点/注意：内存错误未必立即崩溃，可能表现为偶发、不可复现的问题。开启 ASan：编译加 `-fsanitize=address -g`，可精确报告越界、泄漏、释放后使用等位置。

RAII 思想

RAII 思想

概念：RAII (Resource Acquisition Is Initialization，资源获取即初始化) 把资源的生命周期绑定到对象的生命周期：构造时获取资源，析构时自动释放资源。无论函数正常返回还是抛异常，栈对象析构都会执行，从而避免资源泄漏。

要点：

- 核心：用对象管理资源，资源获取在构造、释放在析构。
- 适用于内存、文件句柄、互斥锁、网络连接等一切"需要成对获取/释放"的资源。
- 异常安全：作用域结束（含异常展开）自动调用析构，不依赖手动清理。
- 标准库体现：`std::unique_ptr`、`std::shared_ptr`、`std::lock_guard`、`std::fstream` 都是 RAII。

示例：

```
#include <iostream>
#include <fstream>
#include <mutex>

// 自定义 RAII 锁管理
class LockGuard {
    std::mutex& m_;
public:
    explicit LockGuard(std::mutex& m) : m_(m) { m_.lock(); }
    ~LockGuard() { m_.unlock(); }    // 析构自动解锁
};

int main() {
    std::mutex mtx;
    {
        LockGuard g(mtx);            // 构造时加锁
        // 无论这里是否抛异常，g 析构时都会解锁
    }

    std::ifstream f("data.txt");    // 构造时打开
    if (f) { /* 读取 */ }           // f 析构时自动关闭文件
    return 0;
}
```

易错点/注意：RAII 对象应避免手动 `release/unlock` 后再依赖析构，否则可能重复释放；把资源封装进类时要禁止或正确实现拷贝（或用 `= delete`）。

智能指针总览

智能指针总览

概念：智能指针是对裸指针的 RAII 封装，负责在合适时机自动释放堆内存，避免手动 `delete` 带来的泄漏与悬垂问题。C++11 引入 `unique_ptr`、`shared_ptr`、`weak_ptr`，替代已被废弃的 `auto_ptr`。

要点：

- 为什么需要：手动管理易泄漏、易重复释放、异常不安全；智能指针自动管理所有权。
- `unique_ptr`：独占所有权，不可拷贝、可移动，零额外开销。
- `shared_ptr`：共享所有权，引用计数归零时释放，有控制块开销。
- `weak_ptr`：不拥有、只观察，解决 `shared_ptr` 循环引用。
- `auto_ptr` (C++98)：已废弃 (C++11 起)，拷贝语义有陷阱，勿再使用。

示例：

```
#include <memory>
#include <iostream>

int main() {
    std::unique_ptr<int> u = std::make_unique<int>(1);    // 独占
    std::shared_ptr<int> s = std::make_shared<int>(2);    // 共享
    std::weak_ptr<int> w = s;                            // 观察 s，不增加计数

    std::cout << "u = " << *u << ", s = " << *s << "\n";
    std::cout << "s.use_count() = " << s.use_count() << "\n"; // 1
    std::cout << "w.expired() = " << w.expired() << "\n";    // false
    return 0;
}
```

易错点/注意：不要在函数参数中同时出现多个智能指针裸 `new` 表达式（如 `f(shared_ptr<T>(new T), g())` 有求值顺序风险）；优先用 `make_unique` / `make_shared`。

智能指针四者对比表

概念：下表汇总四种指针（含已废弃的 `auto_ptr`）的所有权、拷贝/移动、开销与典型用途，便于快速选择。

要点：

类型	所有权	可拷贝	可移动	额外开销	典型用途
<code>unique_ptr</code>	独占	否	是	几乎无	局部资源、工厂返回值
<code>shared_ptr</code>	共享	是	是	控制块（引用计数）	多对象共享同一资源
<code>weak_ptr</code>	不拥有	是	是	依赖 <code>shared_ptr</code>	观察、打破循环引用
<code>auto_ptr</code>	转移式独占	拷贝即转移	否	无	已废弃，勿用

示例：

```

#include <memory>
#include <iostream>

int main() {
    auto u1 = std::make_unique<int>(1);
    auto u2 = std::move(u1);          // 移动, u1 变为空
    std::cout << "u1 == nullptr: " << (u1 == nullptr) << "\n"; // 1

    auto s1 = std::make_shared<int>(2);
    auto s2 = s1;                    // 拷贝, 计数 +1
    std::cout << "s1.use_count() = " << s1.use_count() << "\n"; // 2
    return 0;
}

```

易错点/注意: `unique_ptr` 的"独占"由 `= delete` 拷贝构造保证; 复制会编译失败, 传递所有权需显式 `std::move`。

unique_ptr

unique_ptr

概念: `unique_ptr` (C++11) 是独占所有权的智能指针, 同一时刻只有一个 `unique_ptr` 拥有某资源。它不可拷贝、可移动, 开销接近裸指针, 是默认的首选智能指针。

要点:

- 创建: C++14 起用 `std::make_unique<T>(args...)`, C++11 用 `unique_ptr<T>(new T(...))`。
- 独占语义: 拷贝构造/拷贝赋值被 `= delete`, 只能 `std::move` 转移所有权。
- 数组形式: `std::unique_ptr<T[]>`, 支持 `operator[]`。
- 自定义删除器: `std::unique_ptr<T, Deleter>`, 适合管理非 `delete` 释放的资源 (如 `fclose`)。
- 自动释放: 离开作用域或 `reset()` 时析构, `release()` 放弃所有权并返回裸指针。

示例:

```

#include <memory>
#include <iostream>
#include <cstdio>

struct FileDeleter {                // 自定义删除器
    void operator()(std::FILE* f) const {
        if (f) { std::fclose(f); std::cout << "文件已关闭\n"; }
    }
};

int main() {
    auto u1 = std::make_unique<int>(42);    // 推荐: 异常安全
    auto u2 = std::move(u1);                // 转移所有权
    std::cout << "u1 空? " << (u1 == nullptr) << ", *u2 = " << *u2 << "\n";

    auto arr = std::make_unique<int[]>(3); // 数组形式
    arr[0] = 7;
    std::cout << "arr[0] = " << arr[0] << "\n";

    std::unique_ptr<std::FILE, FileDeleter> fp(std::fopen("t.txt", "w"));
}

```

```

    if (fp) { std::fputs("hi", fp.get()); } // get() 取裸指针
    return 0;                             // fp 析构自动 fclose
}

```

易错点/注意:

- `unique_ptr` 不支持拷贝, 作为函数返回值时可隐式移动, 直接 `return make_unique<T>()` 即可。
- 数组形式与单对象形式别混用删除器; `release()` 后必须由调用者负责释放, 否则泄漏。

shared_ptr

shared_ptr

概念: `shared_ptr` (C++11) 通过引用计数实现共享所有权: 每拷贝一次计数加一, 每个 `shared_ptr` 析构时计数减一, 计数归零时释放资源。适合多个对象共享同一资源的场景。

要点:

- 引用计数原理: 控制块保存"强引用计数""弱引用计数"与删除器等。
- `make_shared` 优点: 一次分配同时容纳对象和控制块, 更高效、异常安全。
- 别名构造: `shared_ptr<T>(other, ptr)` 共享 `other` 的控制块但指向 `ptr`, 可用于指向子对象。
- 线程安全性: 引用计数增减是线程安全的, 但被管理的对象本身不保证线程安全, 需自行加锁。
- `reset()` 释放当前引用, `use_count()` 返回强引用计数。

示例:

```

#include <memory>
#include <iostream>

struct Node { int v; Node* next = nullptr; };

int main() {
    auto s1 = std::make_shared<int>(10); // 一次分配
    auto s2 = s1;                        // 计数 2
    std::cout << "use_count = " << s1.use_count() << "\n"; // 2
    s2.reset();                          // 计数回到 1
    std::cout << "use_count = " << s1.use_count() << "\n"; // 1

    // 别名构造: 共享 Node 对象, 但指向其成员 v
    auto node = std::make_shared<Node>();
    node->v = 99;
    std::shared_ptr<int> alias(node, &node->v); // 共享 node 的所有权
    std::cout << "*alias = " << *alias << ", use_count = "
              << node.use_count() << "\n";    // 2

    return 0;
}

```

易错点/注意:

- 不要从同一裸指针构造多个独立 `shared_ptr`, 否则会产生多个控制块、多次释放。
- `shared_ptr` 存在控制块开销与线程安全计数开销, 无共享需求时优先 `unique_ptr`。

weak_ptr

weak_ptr

概念： `weak_ptr` (C++11) 是对 `shared_ptr` 所管理对象的"弱引用"：它不增加引用计数、不拥有资源，只用于观察。典型用途是观察者模式，以及打破 `shared_ptr` 互相引用造成的循环引用。

要点：

- 创建：只能用 `shared_ptr` 或另一 `weak_ptr` 构造，不能直接指向裸指针。
- `lock()`：返回一个临时 `shared_ptr`；对象已释放则返回空，用于安全访问。
- `expired()`：判断被观察对象是否已销毁。
- 观察者模式：被观察者持有 `weak_ptr` 指向观察者，避免延长其生命。
- 循环引用：两个 `shared_ptr` 互指会导致计数永不为零，把一方改为 `weak_ptr` 即可破解。

示例：

```
#include <memory>
#include <iostream>

struct B; // 前置声明

struct A {
    std::shared_ptr<B> b; // 强引用
    ~A() { std::cout << "A 析构\n"; }
};

struct B {
    std::weak_ptr<A> a; // 弱引用：打破循环
    ~B() { std::cout << "B 析构\n"; }
};

int main() {
    auto a = std::make_shared<A>();
    auto b = std::make_shared<B>();
    a->b = b; // A 强引用 B
    b->a = a; // B 弱引用 A (不增加计数)

    std::cout << "a.use_count = " << a.use_count() << "\n"; // 1
    std::cout << "b.use_count = " << b.use_count() << "\n"; // 2

    // 作用域结束：a、b 局部 shared_ptr 释放，A/B 均能被正确析构
    return 0;
}
```

易错点/注意： `weak_ptr` 不能直接解引用，必须先 `lock()` 得到 `shared_ptr` 再访问；`lock()` 之后对象可能被其他线程释放，需在持有 `shared_ptr` 期间使用。

enable_shared_from_this

enable_shared_from_this

概念： 当类内部需要"获取指向自身的安全 `shared_ptr`"时（如回调中注册自己），应继承 `std::enable_shared_from_this<T>` 并调用 `shared_from_this()`。直接基于裸 `this` 构造 `shared_ptr` 会产生多个控制块，导致重复释放。

要点：

- 为什么需要：对象已被某个 `shared_ptr` 管理，内部想再拿一个共享它的 `shared_ptr`。
- 裸 `this` 的坑：`shared_ptr<T>(this)` 会为同一对象新建独立控制块，计数不共享，多次释放。
- 用法：`class T : public std::enable_shared_from_this<T>`，内部调用 `shared_from_this()`。
- 前提：对象必须已被 `shared_ptr` 管理，否则 `shared_from_this()` 抛 `std::bad_weak_ptr`。

示例：

```
#include <memory>
#include <iostream>

struct Widget : std::enable_shared_from_this<Widget> {
    std::shared_ptr<Widget> self() {
        return shared_from_this(); // 安全：共享现有控制块
    }
};

int main() {
    auto w = std::make_shared<Widget>();
    auto w2 = w->self(); // 与 w 共享同一控制块
    std::cout << "w.use_count = " << w.use_count() << "\n"; // 2

    // Widget bad; // 未由 shared_ptr 管理
    // auto x = bad.self(); // 抛 std::bad_weak_ptr
    return 0;
}
```

易错点/注意：不要在构造函数/析构函数中调用 `shared_from_this()`（此时 `enable_shared_from_this` 尚未初始化或已失效）；务必确保对象由 `shared_ptr` 持有。

右值引用与移动语义

右值引用与移动语义

概念：C++11 引入右值引用 `T&&` 与移动语义，把“即将销毁的对象”的资源直接“偷”过来，避免深拷贝。配合 `std::move` 与移动构造/移动赋值，可大幅提升含堆资源对象的性能。

要点：

- 值类别：左值（有名字、可取地址）、纯右值（字面量、临时值）、将亡值（xvalue，`std::move` 的结果）；右值 = 纯右值 + 将亡值。
- `std::move` 本质是 `static_cast<T&&>`，只做类型转换，不移动任何东西。
- 移动构造/移动赋值接收 `T&&`，通常把源对象的资源指针置空，源进入“可析构”状态。
- `noexcept` 对移动的影响：移动构造标 `noexcept` 才能让 `vector` 扩容时优先移动而非拷贝。

示例：

```
#include <iostream>
#include <utility>
#include <cstring>

class Buffer {
    char* data_;
    size_t size_;
public:
    explicit Buffer(size_t n) : data_(new char[n]), size_(n) {
```

```

        std::cout << "构造\n";
    }
    ~Buffer() { delete[] data_; }

    Buffer(const Buffer& o) : data_(new char[o.size_]), size_(o.size_) {
        std::memcpy(data_, o.data_, size_);
        std::cout << "拷贝构造\n";
    }
    Buffer(Buffer&& o) noexcept // 移动构造
        : data_(o.data_), size_(o.size_) {
        o.data_ = nullptr; // 源置空
        o.size_ = 0;
        std::cout << "移动构造\n";
    }
    Buffer& operator=(Buffer&& o) noexcept { // 移动赋值
        if (this != &o) {
            delete[] data_;
            data_ = o.data_; size_ = o.size_;
            o.data_ = nullptr; o.size_ = 0;
        }
        std::cout << "移动赋值\n";
        return *this;
    }
};

int main() {
    Buffer a(10);
    Buffer b = std::move(a); // 触发移动构造, a 被"掏空"
    Buffer c(20);
    c = std::move(b); // 触发移动赋值
    return 0;
}

```

易错点/注意:

- 被移动的对象仍可析构、可重新赋值，但不应再依赖其旧值。
- `std::move` 不保证一定移动：若目标类型没有移动语义，会退化为拷贝。
- 移动后源对象的成员应保持"有效但未指定"状态。

万能引用与完美转发

万能引用与完美转发

概念： `T&&` 在模板推导上下文中是"万能引用"（转发引用），能同时匹配左值与右值。配合引用折叠与 `std::forward`，可实现完美转发：把参数的值类别原样转发给下一层函数。

要点：

- 万能引用：形如 `template<class T> void f(T&& x)`，`T` 被推导时 `T&&` 既可绑定左值也可绑定右值。
- 引用折叠：`T& &/ T& &&/ T&& &` 折叠为 `T&`；`T&& &&` 折叠为 `T&&`。
- `std::forward<T>(x)`：根据 `T` 推导结果，把参数按原值类别转发（左值保持左值、右值保持右值）。
- `forward` 与 `move` 区别：`move` 无条件转成右值，`forward` 有条件地保留原值类别。
- 只有 `T&&`（且 `T` 是模板参数、非 `const`）才是万能引用；`const T&&` 或具体类型 `int&&` 不是。

示例：

```

#include <iostream>
#include <utility>

void process(int& x) { std::cout << "左值版本\n"; }
void process(int&& x) { std::cout << "右值版本\n"; }

template<class T>
void relay(T&& arg) {
    process(std::forward<T>(arg)); // 完美转发: 保留 arg 的值类别
}

int main() {
    int a = 1;
    relay(a);           // 左值 -> 左值版本
    relay(2);           // 右值 -> 右值版本
    relay(std::move(a)); // 右值 -> 右值版本
    return 0;
}

```

易错点/注意:

- 具名变量本身是左值: 即使 `arg` 被推导为右值, `arg` 这个"名字"在函数体内仍是左值, 必须用 `std::forward` 才能恢复右值性。
- 万能引用的前提是"形参直接是 `T&&` 且 `T` 由本函数推导"; `std::vector<T>&&` 不是万能引用。

本部分小结

本部分小结

概念: 本部分覆盖了指针、引用、动态内存与智能指针的核心知识点。下表汇总最易遗忘的要点, 供考前快速回顾。

要点:

主题	易忘点
指针	指针存地址；* 解引用；nullptr 类型安全
数组	数组名退化为首元素指针；sizeof(arr) 是整数组
const 指针	const 在 * 左修饰对象、在右修饰指针
引用	必须初始化、不可重绑定；勿返回局部变量引用
new/delete	new[] 配 delete[]；placement new 只构造不分配
内存分区	栈自动、堆手动、静态区贯穿全程、常量区只读
内存错误	泄漏/悬垂/野指针/重复释放/溢出；用 ASan 检测
RAII	资源绑定对象生命周期，析构自动释放
unique_ptr	独占、可移动、make_unique、自定义删除器
shared_ptr	引用计数、make_shared 一次分配、计数线程安全但对象不保证
weak_ptr	lock() 安全访问、破解循环引用
enable_shared_from_this	内部安全获取共享自己的 shared_ptr
移动语义	std::move 只是 cast；移动构造应 noexcept
完美转发	T&& + std::forward；引用折叠规则

第四部分：字符串、数组与 STL 容器

C 风格字符串

C 风格字符串

概念：C 风格字符串是以空字符 '\0' 结尾的字符数组，是 C 语言遗留的字符串表示方式。它没有长度信息，靠终止符界定结尾，配合 <cstring> 中的函数操作，但存在大量缓冲区安全隐患。

要点：

- 表示：char s[] = "abc"; 实际长度为 4（含结尾 '\0'）；char* p = "abc"; 指向只读常量区。
- 常用函数：strlen（求长，不含 \0）、strcpy（拷贝）、strcat（拼接）、strcmp（比较）。
- 安全问题：strcpy/strcat/gets 不检查目标缓冲区大小，易溢出；strncpy 不保证结尾 \0。
- strcmp 返回值：相等为 0，前者小于后者为负，大于为正（按字典序逐字节比较）。

示例：

```
#include <iostream>
#include <cstring>

int main() {
    char s1[] = "hello";           // 6 字节，含 '\0'
    char s2[20];
```

```

std::cout << "strlen(s1) = " << std::strlen(s1) << "\n"; // 5

std::strcpy(s2, s1);           // 拷贝
std::strcat(s2, " world");     // 拼接
std::cout << "s2 = " << s2 << "\n"; // hello world

int cmp = std::strcmp(s1, "world");
std::cout << "strcmp = " << cmp << "\n"; // 负数 ('h' < 'w')

// 安全做法：用 strncpy 并手动补 '\0'
char s3[10];
std::strncpy(s3, s1, sizeof(s3) - 1);
s3[sizeof(s3) - 1] = '\0';
std::cout << "s3 = " << s3 << "\n";
return 0;
}

```

易错点/注意：`strlen` 是 $O(n)$ 遍历到 `\0`，不要放在循环条件里反复调用；忘记给 `\0` 留空间会导致越界读写。现代 C++ 应优先用 `std::string`。

std::string 基础

std::string 基础

概念：`std::string` 是 C++ 标准库的字符串类，自动管理内存、记录长度，支持构造、拼接、比较、遍历等丰富操作，是首选字符串类型，避免 C 风格字符串的手动管理。

要点：

- 构造：默认、`const char*`、重复字符、子串、拷贝/移动等多种构造方式。
- 拼接：`+`、`+=`、`append`；与 `const char*`、字符均可拼接。
- 比较：`==`、`!=`、`<` 等按字典序，也支持 `compare`。
- 遍历：下标 `[]`（不检查越界）、`at`（抛 `std::out_of_range`）、范围 `for`、迭代器。
- 自动管理：内部维护堆缓冲区，扩容时自动重新分配，无需手动释放。

示例：

```

#include <iostream>
#include <string>

int main() {
    std::string s0;           // 空串
    std::string s1 = "hello"; // 从 C 串构造
    std::string s2(3, 'a');   // "aaa"
    std::string s3 = s1 + ", " + "world"; // 拼接
    s3 += "!";               // 追加

    std::cout << "s2 = " << s2 << "\n";
    std::cout << "s3 = " << s3 << "\n";

    std::cout << "s1 < s2 ? " << (s1 < s2) << "\n"; // 按字典序

    for (char c : s1) { std::cout << c << ' '; } // 范围 for 遍历
    std::cout << "\n";
    for (size_t i = 0; i < s1.size(); ++i) {

```

```

        std::cout << s1[i];
    }
    std::cout << "\n";
    return 0;
}

```

易错点/注意：`s1 < s2` 是字典序而非长度比较；`[]` 越界是未定义行为，需要安全检查用 `at`。频繁拼接大字符串可先 `reserve` 预留容量。

std::string 常用操作

std::string 常用操作

概念：`std::string` 提供查找、子串、替换、插入、删除、转换等常用成员函数，以及与 C 串和数值类型的互转能力，是日常字符串处理的核心工具。

要点：

- 查找：`find` (正向)、`rfind` (反向)，找不到返回 `std::string::npos`。
- 子串/修改：`substr`、`replace`、`insert`、`erase`。
- C 串接口：`c_str()` 返回 `const char*` (保证 `\0` 结尾)；`data()` 自 C++17 起也返回 `const char*`，可空字符结尾。
- 数值转换：`std::to_string` (数值→串)、`std::stoi/stod` (串→数值，可抛异常)。
- `starts_with/ends_with` (C++20)：判断前缀/后缀。

示例：

```

#include <iostream>
#include <string>

int main() {
    std::string s = "hello world";

    // 查找
    auto pos = s.find("world");
    std::cout << "find pos = " << pos << "\n"; // 6
    std::cout << "rfind 'o' = " << s.rfind('o') << "\n"; // 7

    // 子串与修改
    std::string sub = s.substr(0, 5); // "hello"
    s.replace(6, 5, "C++"); // "hello C++"
    s.insert(5, ","); // "hello, C++"
    s.erase(0, 6); // "C++"
    std::cout << "s = " << s << "\n";

    // 数值转换
    std::string num = std::to_string(42);
    int v = std::stoi("123");
    double d = std::stod("3.14");
    std::cout << num << ' ' << v << ' ' << d << "\n";

    // C++20 前后缀判断
    std::string t = "example.txt";
    std::cout << "ends_with .txt: " << t.ends_with(".txt") << "\n"; // 1 (C++20)
    std::cout << "starts_with ex: " << t.starts_with("ex") << "\n"; // 1 (C++20)
    return 0;
}

```

```
}
```

易错点/注意:

- 未找到时返回 `npos`，务必与 `std::string::npos` 比较，不能当作 `-1` 的普通 `int` 直接判断。
- `c_str()` 返回的指针在字符串被修改后可能失效，勿长期保存。
- `stoi` 遇非法输入抛 `std::invalid_argument`，越界抛 `std::out_of_range`。

std::string_view

std::string_view (C++17)

概念: `std::string_view` (C++17) 是字符串的"只读、非拥有"视图，保存指针与长度，不拷贝数据。它适合作为函数参数接受任意字符串类型 (`string`、`const char*`、字符数组)，避免不必要的拷贝。

要点:

- 零拷贝：只存指针和长度，构造/拷贝极廉价。
- 非拥有：不负责内存管理，必须保证所指向的字符串在视图存活期间有效。
- 与 `string` 对比：`string` 拥有内存、可修改；`string_view` 只读、不拥有。
- 常见函数：`substr` 是 $O(1)$ (只调整指针/长度)、`remove_prefix/remove_suffix`、`find`。
- 生命周期坑：临时 `string` 或 `const char*` 被销毁后，视图悬垂。

示例:

```
#include <iostream>
#include <string>
#include <string_view>

void print(std::string_view sv) {           // 可接受任意字符串类型
    std::cout << "长度 " << sv.size() << ": " << sv << "\n";
}

int main() {
    std::string s = "hello world";
    const char* c = "hi";
    print(s);                               // 接受 string
    print(c);                               // 接受 const char*
    print("literal");                       // 接受字符串字面量

    std::string_view sv = s;
    auto part = sv.substr(0, 5);            // O(1), 不拷贝
    std::cout << "part = " << part << "\n"; // hello

    sv.remove_prefix(6);                    // 视图变为 "world"
    std::cout << "sv = " << sv << "\n";
    return 0;
}
```

易错点/注意:

- 不要让 `string_view` 持有临时 `std::string` 的视图：`std::string_view bad(std::string("x"))`; 中临时串即死，视图悬垂。
- `string_view` 不保证以 `\0` 结尾，传给需要 `c_str()` 的 C API 时要小心。

std::array

std::array (C++11)

概念： `std::array` (C++11) 是固定大小的数组封装，大小在编译期确定，元素连续存储，兼具 C 数组的性能与容器的接口（`size`、`begin`、迭代器）。它不退化、可整体赋值，比裸数组更安全。

要点：

- 定义：`std::array<int, 3> a = {1, 2, 3};` 大小是类型的一部分。
- 不退化：作为参数传递时不会退化为指针，保留大小信息。
- 支持 `size()`、`at()`（越界抛异常）、`[]`、`fill`、迭代器、范围 `for`。
- 与 C 数组对比：接口更丰富、可整体赋值/比较（C++20 起支持比较）、但大小仍是编译期常量。

示例：

```
#include <iostream>
#include <array>
#include <algorithm>

int main() {
    std::array<int, 3> a = {1, 2, 3};
    std::array<int, 3> b = a;           // 整体赋值（C 数组做不到）

    std::cout << "a.size() = " << a.size() << "\n"; // 3
    std::cout << "a[1] = " << a[1] << "\n";
    try {
        std::cout << a.at(5) << "\n"; // 越界：抛 std::out_of_range
    } catch (const std::out_of_range& e) {
        std::cout << "越界异常： " << e.what() << "\n";
    }

    a.fill(7);                         // 全部填 7
    std::sort(a.begin(), a.end());      // 支持 STL 算法
    for (int x : a) { std::cout << x << ' '; }
    std::cout << "\n";
    return 0;
}
```

易错点/注意： `std::array` 大小必须是编译期常量，不能是运行时变量；虽然它不退化，但仍可能有额外包装的细微差别（如空 `array` 的 `data()` 行为）。

std::vector

std::vector

概念： `std::vector` 是动态数组，元素连续存储、支持随机访问、末尾高效增删。它是 STL 中最常用的容器，通过“容量翻倍”策略动态扩容，兼顾随机访问与尾部扩展。

要点：

- 动态扩容原理：`size` 达到 `capacity` 时，分配更大的新内存（常见 1.5 倍或 2 倍）、搬移旧元素、释放旧内存。
- `reserve`：预留容量，避免反复扩容；`resize`：改变元素个数（新增元素值初始化）。
- `shrink_to_fit`：请求把容量缩小到与 `size` 接近（不保证一定释放）。
- `emplace_back` vs `push_back`：前者原地构造、免临时对象，通常更高效。

- `vector<bool>` 特化：按位存储，`operator[]` 返回代理对象而非 `bool&`，存在性能与语义坑。
- 二维 vector：`vector<vector<int>>` 表示矩阵，注意 `>>` 之间 C++11 起无需空格。

示例：

```
#include <iostream>
#include <vector>
#include <string>

int main() {
    std::vector<int> v;
    v.reserve(8); // 预留容量
    for (int i = 0; i < 5; ++i) v.push_back(i);

    std::cout << "size = " << v.size()
              << ", capacity = " << v.capacity() << "\n"; // 5, 8

    v.resize(3); // 缩小到 3 个元素
    v.shrink_to_fit(); // 请求释放多余容量

    struct P { int x; std::string s; };
    std::vector<P> ps;
    ps.emplace_back(1, "a"); // 原地构造，免临时对象
    ps.push_back(P{2, "b"}); // 先构造临时再移动

    // 二维 vector
    std::vector<std::vector<int>> mat(2, std::vector<int>(3, 0));
    mat[0][1] = 9;
    std::cout << "mat[0][1] = " << mat[0][1] << "\n";
    return 0;
}
```

示例：`vector<bool>` 特化的坑：

```
#include <iostream>
#include <vector>

int main() {
    std::vector<bool> vb = {true, false, true};

    auto b = vb[0]; // 得到"代理对象"，不是 bool&
    b = false; // 只改代理，不影响 vb[0]
    std::cout << "vb[0] = " << vb[0] << "\n"; // 仍是 1 (true)

    auto&& r = vb[0]; // 用 auto&& 保留代理，可修改
    r = false;
    std::cout << "vb[0] = " << vb[0] << "\n"; // 0 (false)

    // 需要真正的 bool 引用时，改用 vector<char> 或 deque<bool>
    return 0;
}
```

易错点/注意：

- `resize` 改变 size，`reserve` 只改 capacity 不改 size，两者勿混淆。

- `vector<bool>` 的 `[]` 返回代理, `auto b = v[0]`; 得到的是代理而非 `bool&`, 修改不生效; 需用 `auto&&` 或改用 `deque<bool>/vector<char>`。
- 扩容会使所有指向元素的指针/引用/迭代器失效。

std::list 与 std::forward_list

std::list 与 std::forward_list

概念: `std::list` 是双向链表, `std::forward_list` (C++11) 是单向链表。它们在任何位置插入/删除都是 $O(1)$ (已定位到该位置), 但不支持随机访问, 遍历和寻址是 $O(n)$ 。

要点:

- `list`: 双向, 支持 `push_back` / `push_front` / `pop_back` / `pop_front`。
- `forward_list`: 单向, 只有 `push_front` / `pop_front`, 更省内存, 但无 `size()` ($O(n)$) 和 `back`。
- `splice`: 把另一链表的元素"拼接"到本链表指定位置, $O(1)$, 不拷贝元素。
- 适用场景: 频繁在中间插入/删除、且不需要随机访问时优于 `vector`。
- 迭代器稳定性: 插入/删除不使其他迭代器失效 (`vector` 会失效)。

示例:

```
#include <iostream>
#include <list>
#include <forward_list>

int main() {
    std::list<int> l = {1, 2, 3};
    l.push_front(0);           // 头插
    l.push_back(4);            // 尾插
    auto it = l.begin();
    ++it;                      // 移动到第二个元素
    l.insert(it, 99);           // 在 it 前插入, O(1)

    std::list<int> l2 = {7, 8};
    l.splice(l.end(), l2);      // 把 l2 全部拼到 l 末尾
    std::cout << "l2 为空? " << l2.empty() << "\n"; // 1

    for (int x : l) std::cout << x << ' ';
    std::cout << "\n";

    std::forward_list<int> fl = {1, 2, 3};
    fl.push_front(0);           // 单向链表只能头插
    for (int x : fl) std::cout << x << ' ';
    std::cout << "\n";
    return 0;
}
```

易错点/注意: 链表不能 `l[i]` 随机访问; `forward_list` 没有 `size()` 和尾部访问, 插入需用 `insert_after` 而非 `insert` (在"给定位置之后"插入)。

std::deque

std::deque

概念： `std::deque` (double-ended queue, 双端队列) 支持在头尾 $O(1)$ 插入/删除, 同时支持随机访问 (接近 `vector`)。内部由多块连续内存 (分段数组 + 中控指针数组) 组成, 元素并非整体连续。

要点：

- 内部结构：中央控制块 (指针数组) 指向多个固定大小缓冲区, 逻辑上连续、物理上分段。
- 头尾插入 $O(1)$, 中间插入 $O(n)$; 随机访问 $O(1)$ 但常数略高于 `vector`。
- 与 `vector` 对比: `vector` 只在尾端高效; `deque` 头尾都高效; `deque` 扩容不需搬移整个旧数据。
- 迭代器失效: 头尾插入/删除通常不使现有迭代器失效, 但可能使引用失效; 中间操作会失效。
- 典型用途: 需要头尾增删 + 随机访问的场景, 如任务队列底层。

示例：

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> d;
    d.push_back(2);           // 尾插
    d.push_back(3);
    d.push_front(1);         // 头插
    d.push_front(0);

    std::cout << "front = " << d.front() << ", back = " << d.back() << "\n";
    std::cout << "d[2] = " << d[2] << "\n"; // 随机访问 2

    d.pop_front();           // 头删
    d.pop_back();            // 尾删
    for (int x : d) std::cout << x << ' '; // 1 2
    std::cout << "\n";
    return 0;
}
```

易错点/注意： `deque` 元素不保证整体连续, 不能像 `vector` 那样依赖 `&d[0]` 到 `&d[n-1]` 的连续性传给需要连续内存的 C API; 如需连续数组优先 `vector`。

容器适配器: stack、queue、priority_queue

stack、queue

概念： `stack`、`queue`、`priority_queue` 是容器适配器, 它们基于底层容器 (默认 `deque`, `priority_queue` 默认 `vector`) 提供受限接口: 栈后进先出 (LIFO), 队列先进先出 (FIFO)。适配器不直接暴露底层容器的全部操作。

要点：

- `stack`: `push/pop/top`, 默认底层 `deque`。
- `queue`: `push/pop/front/back`, 默认底层 `deque`。
- `priority_queue`: `push/pop/top`, 默认大顶堆 (最大元素在队首)。
- 可用第二模板参数指定底层容器, 如 `stack<int, vector<int>>`。
- `pop()` 不返回值, 需先 `top()/front()` 取值再 `pop()`。

示例：

```

#include <iostream>
#include <stack>
#include <queue>

int main() {
    std::stack<int> st;
    st.push(1); st.push(2); st.push(3);
    std::cout << "栈顶 = " << st.top() << "\n"; // 3
    st.pop();
    std::cout << "弹出后栈顶 = " << st.top() << "\n"; // 2

    std::queue<int> q;
    q.push(1); q.push(2); q.push(3);
    std::cout << "队首 = " << q.front() << ", 队尾 = " << q.back() << "\n";
    q.pop(); // 弹出队首
    std::cout << "弹出后队首 = " << q.front() << "\n";
    return 0;
}

```

易错点/注意： `pop()` 返回 `void`，不能 `int x = st.pop();`；必须先用 `top()` / `front` 取值。适配器不提供迭代器。

priority_queue 与自定义比较器

概念： `priority_queue` 默认是大顶堆（`top` 返回最大元素）。通过自定义比较器可构造小顶堆或按自定义键排序。比较器语义与 `std::sort` 相反，容易写反。

要点：

- 默认： `priority_queue<int>` 是大顶堆， `top()` 是最大值。
- 小顶堆： `priority_queue<int, vector<int>, greater<int>>`（需 `<functional>`）。
- 自定义比较：传入比较器类型或 lambda（C++20 起可用 lambda 作为模板实参；更通用用 `decltype`）。
- 注意：优先队列的“比较”是“优先级更低者排在后面”， `greater` 得到小顶堆，与直觉相反。
- 复杂度： `push/pop` 为 $O(\log n)$ ， `top` 为 $O(1)$ 。

示例：

```

#include <iostream>
#include <queue>
#include <vector>
#include <functional>

int main() {
    // 大顶堆（默认）
    std::priority_queue<int> maxHeap;
    for (int x : {3, 1, 4, 1, 5}) maxHeap.push(x);
    std::cout << "大顶堆 top = " << maxHeap.top() << "\n"; // 5

    // 小顶堆
    std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
    for (int x : {3, 1, 4, 1, 5}) minHeap.push(x);
    std::cout << "小顶堆 top = " << minHeap.top() << "\n"; // 1

    // 自定义比较：按 pair 的 first 建小顶堆
    using P = std::pair<int, int>;
    auto cmp = [](const P& a, const P& b) { return a.first > b.first; };
}

```

```

std::priority_queue<P, std::vector<P>, decltype(cmp)> pq(cmp);
pq.push({2, 0}); pq.push({1, 1}); pq.push({3, 2});
std::cout << "自定义 top = " << pq.top().second << "\n"; // 1
return 0;
}

```

易错点/注意： `std::greater<int>` 表示"小顶堆"；比较器写反会导致堆序颠倒。旧标准中 lambda 不能直接作模板实参，需用 `decltype(cmp)` 并传实例。

有序关联容器：set/multiset/map/multimap

set/multiset/map/multimap 基础

概念：有序关联容器基于红黑树（自平衡二叉搜索树）实现，元素按比较器有序排列，查找/插入/删除均为 $O(\log n)$ 。 `set/multiset` 存单值， `map/multimap` 存键值对；带 `multi` 的允许重复键。

要点：

- 红黑树：保证树近似平衡，操作 $O(\log n)$ 。
- 有序遍历：迭代器按键升序遍历。
- 自定义比较：`set<int, greater<int>>` 或传入比较器类型。
- `lower_bound`（第一个 $\geq$ key）、`upper_bound`（第一个 $>$ key）、`equal_range`（等于 key 的区间）。
- `map` 的 `operator[]`：不存在则插入默认值并返回引用；`at`：不存在则抛 `out_of_range`。

示例：

```

#include <iostream>
#include <set>
#include <map>

int main() {
    std::set<int> s = {3, 1, 4, 1, 5}; // 自动去重排序
    for (int x : s) std::cout << x << ' '; // 1 3 4 5
    std::cout << "\n";

    auto it = s.lower_bound(3); // 第一个 >= 3
    std::cout << "lower_bound(3) = " << *it << "\n"; // 3
    auto it2 = s.upper_bound(3); // 第一个 > 3
    std::cout << "upper_bound(3) = " << *it2 << "\n"; // 4
    auto r = s.equal_range(3); // [3,4)
    std::cout << "equal_range(3) 有 " << std::distance(r.first, r.second) << " 个\n";

    std::map<std::string, int> m;
    m["a"] = 1; // operator[]: 插入并赋值
    std::cout << "m[\"b\"] = " << m["b"] << "\n"; // 不存在，插入默认 0
    std::cout << "m.at(\"a\") = " << m.at("a") << "\n"; // 1
    // m.at("c"); // 抛 std::out_of_range
    return 0;
}

```

易错点/注意：

- `operator[]` 会静默插入默认值，只读场景用 `find` 或 `at/count`，避免意外修改。
- `multiset/multimap` 的 `equal_range` 用于遍历重复键；它们没有 `operator[]`。

有序关联容器自定义比较

概念：有序容器的"顺序"由比较器决定。默认 `std::less<Key>` 得到升序，传入 `std::greater` 或自定义仿函数/lambda 可改变排序规则。比较器必须定义严格弱序。

要点：

- 升序：默认；降序：`std::set<int, std::greater<int>>`。
- 自定义比较：传入仿函数类型，构造时可传实例。
- 严格弱序要求：不可逆反（`a<b` 与 `b<a` 不同时为真）、传递、等价传递。
- 用于 `map` 的自定义比较同样作用于键。

示例：

```
#include <iostream>
#include <set>
#include <string>

struct Person {
    std::string name;
    int age;
};

struct ByAge { // 自定义比较器
    bool operator()(const Person& a, const Person& b) const {
        return a.age < b.age; // 按年龄升序
    }
};

int main() {
    std::set<int, std::greater<int>> ds = {1, 3, 2};
    for (int x : ds) std::cout << x << ' '; // 3 2 1 (降序)
    std::cout << "\n";

    std::set<Person, ByAge> people = {
        {"Alice", 30}, {"Bob", 20}, {"Cindy", 25}
    };
    for (const auto& p : people) {
        std::cout << p.name << '(' << p.age << ") ";
    }
    std::cout << "\n"; // Bob(20) Cindy(25) Alice(30)
    return 0;
}
```

易错点/注意：比较器若写成 `return a.age <= b.age;` 会破坏严格弱序（等价元素同时满足两边），导致容器行为未定义。等价元素用 `multi` 容器或复合比较。

无序关联容器：unordered_map/unordered_set

unordered_map/unordered_set 基础

概念：无序关联容器（C++11）基于哈希表（哈希桶）实现，平均查找/插入/删除为 $O(1)$ ，但元素无序。它需要哈希函数与相等比较，通过 `rehash/load_factor` 控制装载因子与桶数。

要点：

- 哈希桶原理：对键算哈希映射到桶，桶内用链表（或开链法）处理冲突，负载过高时扩容重哈希。

- 平均 $O(1)$ ，最坏 $O(n)$ （哈希碰撞严重时退化）。
- 自定义哈希与相等比较：提供 `std::hash` 特化或自定义仿函数。
- `rehash(n)` 设桶数、`reserve(n)` 预留元素数、`load_factor()` 返回装载因子（元素数/桶数）。
- 迭代器稳定性：`rehash` 使所有迭代器失效；插入可能触发 `rehash` 而失效；删除只使被删元素迭代器失效。

示例：

```
#include <iostream>
#include <unordered_map>
#include <string>

int main() {
    std::unordered_map<std::string, int> um;
    um["apple"] = 3;
    um["banana"] = 5;
    um["cherry"] = 7;

    for (const auto& [k, v] : um) {    // 结构化绑定 (C++17)
        std::cout << k << ":" << v << ' '; // 顺序不定
    }
    std::cout << "\n";

    std::cout << "bucket_count = " << um.bucket_count() << "\n";
    std::cout << "load_factor = " << um.load_factor() << "\n";
    um.rehash(100);                // 调整桶数，迭代器失效
    std::cout << "rehash 后 bucket_count = " << um.bucket_count() << "\n";
    return 0;
}
```

易错点/注意：无序容器遍历顺序与插入顺序无关，不可依赖；`rehash` 后勿再使用旧迭代器。哈希函数质量差会导致大量碰撞、性能退化。

unordered 容器自定义哈希与相等

概念：使用自定义类型作为无序容器键时，必须提供哈希函数和相等比较（默认为 `std::hash<Key>` 和 `operator==`）。可通过特化 `std::hash` 或传入自定义仿函数实现。

要点：

- 自定义哈希：实现 `size_t operator()(const Key&) const`，可用移位/异或混合字段。
- 自定义相等：实现 `bool operator()(const Key&, const Key&) const` 或为类型重载 `operator==`。
- 相等对象必须有相同哈希值（哈希一致性），否则容器行为未定义。
- 好的哈希应尽量分散，减少碰撞。

示例：

```
#include <iostream>
#include <unordered_set>
#include <string>

struct Point {
    int x, y;
    bool operator==(const Point& o) const {    // 相等比较

```

```
        return x == o.x && y == o.y;
    }
};

struct PointHash {                                // 自定义哈希
    size_t operator()(const Point& p) const {
        return std::hash<int>()(p.x) ^ (std::hash<int>()(p.y) << 1);
    }
};

int main() {
    std::unordered_set<Point, PointHash> s;
    s.insert({1, 2});
    s.insert({3, 4});
    s.insert({1, 2});                                // 重复，去重

    std::cout << "size = " << s.size() << "\n"; // 2
    std::cout << "含 (3,4)? " << s.count({3, 4}) << "\n"; // 1
    return 0;
}
```

易错点/注意：相等对象哈希必须一致；若修改了键对象（破坏哈希），容器会失去一致性，故无序容器键通常为 const 或不可变。

迭代器体系

迭代器体系

概念：迭代器是连接算法与容器的“通用指针”，提供统一的遍历/访问接口。C++ 将迭代器按能力分为五类：输入、输出、前向、双向、随机访问。理解其能力差异有助于判断算法适用性。

要点：

- 输入迭代器：只读、单遍（`istream_iterator`）。
- 输出迭代器：只写、单遍（`ostream_iterator`）。
- 前向迭代器：可读可写、多遍、只能前进（`forward_list`）。
- 双向迭代器：可前进后退（`list`、`set`、`map`）。
- 随机访问迭代器：支持 `+n`、`[]`、`<`（`vector`、`deque`、`array`、原生指针）。
- `begin/end` 普通、`cbegin/cend` 只读 const、`rbegin/rend` 反向。

要点：五类迭代器能力表（能力逐级增强）：

能力	输入	输出	前向	双向	随机访问
读取 <code>*it</code>	☑	✗	☑	☑	☑
写入 <code>*it</code>	✗	☑	☑	☑	☑
前进 <code>++it</code>	☑	☑	☑	☑	☑
后退 <code>--it</code>	✗	✗	✗	☑	☑
跳转 <code>it+n</code> / <code>[]</code>	✗	✗	✗	✗	☑
多遍遍历	✗	✗	☑	☑	☑
典型载体	<code>istream_iterator</code>	<code>ostream_iterator</code>	<code>forward_list</code>	<code>list/set/map</code>	<code>vector/deque/array/指针</code>

示例：

```

#include <iostream>
#include <vector>
#include <iterator>
#include <list>

int main() {
    std::vector<int> v = {1, 2, 3, 4};
    auto it = v.begin();           // 随机访问迭代器
    it += 2;                       // 支持 +n
    std::cout << "v[2] = " << *it << "\n";

    std::list<int> l = {1, 2, 3};
    auto lit = l.begin();          // 双向迭代器
    ++lit;
    // lit += 2;                   // 错误：双向不支持 +n

    for (auto rit = v.rbegin(); rit != v.rend(); ++rit) { // 反向
        std::cout << *rit << ' '; // 4 3 2 1
    }
    std::cout << "\n";
    return 0;
}

```

易错点/注意：不同容器迭代器能力不同，`std::sort` 需要随机访问迭代器，不能用于 `list` (`list` 有成员 `sort`)。用 `cbegin/cend` 保证只读，避免误改。

流迭代器简介

概念：流迭代器把 I/O 流包装成迭代器：`istream_iterator` 用于从流读入，`ostream_iterator` 用于向流写出，使标准算法能直接作用于流。

要点：

- `istream_iterator<T>`：默认构造的是"结束哨兵"；`++/*` 读取下一个元素。
- `ostream_iterator<T>(os, delim)`：每次赋值写入一个元素并追加分隔符。
- 可与 `copy`、`accumulate` 等算法配合。
- 属于输入/输出迭代器，单遍，不能反复遍历。

示例：

```

#include <iostream>
#include <iterator>
#include <vector>
#include <algorithm>

int main() {
    std::istream_iterator<int> in(std::cin), eof; // 从标准输入读整数
    std::vector<int> v(in, eof);                  // 读到文件结束 (Ctrl+D/Z)

    std::ostream_iterator<int> out(std::cout, " "); // 输出，空格分隔
    std::copy(v.begin(), v.end(), out);             // 打印所有元素
    std::cout << "\n";
    return 0;
}

```

易错点/注意：`istream_iterator` 读到流结束或类型不匹配时停止，需注意输入类型必须与模板参数一致；它是单遍迭代器，不能二次遍历。

迭代器失效问题汇总

迭代器失效问题汇总

概念：容器在插入/删除/扩容等操作后，部分迭代器、指针、引用可能失效（指向的内存被释放或重分配），继续使用会导致未定义行为。不同容器失效规则不同，是 STL 使用中最易踩的坑。

要点：

- `vector`：尾插可能扩容 → 全部失效；中间插入/删除 → 位置及之后失效；
`reserve/shrink_to_fit` 可能全部失效。
- `deque`：头尾增删不影响迭代器（可能使引用失效）；中间插入/删除 → 全部失效。
- `list/forward_list`：插入/删除不使其他迭代器失效，只有被删除元素的迭代器失效。
- `set/map/multiset/multimap`：删除只使被删元素迭代器失效，插入不失效。
- `unordered_*`：`rehash` 全部失效；插入可能触发 `rehash`；删除只使被删元素失效。

示例：

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};
    auto it = v.begin();           // 指向 1
    v.push_back(4);                // 可能扩容 → it 失效
    // std::cout << *it;          // 危险：未定义行为

    // 正确做法：删除后重新获取迭代器（erase 返回下一个有效迭代器）
    std::vector<int> w = {1, 2, 3, 4, 5};
    for (auto i = w.begin(); i != w.end(); ) {
        if (*i % 2 == 0) {
            i = w.erase(i);        // 用返回值更新，避免失效
        } else {
            ++i;
        }
    }
    for (int x : w) std::cout << x << ' '; // 1 3 5
    std::cout << "\n";
    return 0;
}
```

易错点/注意：最稳妥的做法是"操作后重新获取迭代器"，或利用 `erase/insert` 的返回值；对可能扩容的 `vector`，先 `reserve` 足够容量可避免失效。

容器选择与复杂度速查表

容器选择与复杂度速查表

概念：不同容器在插入、删除、查找、随机访问上的复杂度差异巨大，选对容器能显著影响性能。下表汇总常用容器的平均复杂度，作为快速决策依据。

要点：

容器	随机访问	头部插入/删除	尾部插入/删除	中间插入/删除	查找
<code>vector</code>	$O(1)$	—	$O(1)$ 均摊	$O(n)$	$O(n)$
<code>deque</code>	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$
<code>list</code>	—	$O(1)$	$O(1)$	$O(1)$	$O(n)$
<code>forward_list</code>	—	$O(1)$	—	$O(1)$	$O(n)$
<code>set/map</code>	—	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
<code>unordered_set/map</code>	—	$O(1)$ 均摊	$O(1)$ 均摊	$O(1)$ 均摊	$O(1)$ 均摊

示例：

```
#include <iostream>
#include <vector>
#include <set>
#include <unordered_set>

int main() {
    // 查找频繁且需有序 → set/map; 无需有序 → unordered
    std::set<int> s = {3, 1, 2};
    std::cout << "set 查找 2: " << (s.find(2) != s.end()) << "\n"; // O(log n)

    std::unordered_set<int> us = {3, 1, 2};
    std::cout << "unordered 查找 2: " << (us.find(2) != us.end()) << "\n"; // O(1)

    // 尾部连续增长 + 随机访问 → vector; 头尾都操作 → deque
    std::vector<int> v;
    for (int i = 0; i < 3; ++i) v.push_back(i); // 均摊 O(1)
    std::cout << "v 大小 = " << v.size() << "\n";
    return 0;
}
```

易错点/注意：复杂度是"平均/均摊"，最坏情况不同（如无序容器哈希碰撞退化为 $O(n)$ 、vector 单次扩容可能 $O(n)$ ）。选择容器需结合"访问模式 + 是否需有序 + 是否需稳定迭代器"综合判断。

本部分小结

本部分小结

概念：本部分覆盖了字符串、数组与 STL 主要容器。下表汇总最易遗忘的要点，供考前快速回顾。

要点：


```

    std::string name_;           // 成员变量
protected:                     // 派生类可见
    int age_ = 0;
};

struct Point {                  // struct 默认 public
    int x;                      // 默认 public
    int y;
};

int main() {
    Person p;
    p.setName("Alice");         // public 成员可访问
    std::cout << p.getName() << std::endl; // Alice
    // p.name_ = "x";           // 错误: private 不可访问
    Point pt{1, 2};             // struct 聚合初始化
    std::cout << pt.x << pt.y << std::endl; // 12
}

```

易错点/注意：struct 也能有 private 成员和成员函数，class 也能聚合初始化；不要误以为 struct 只能放数据。

封装的意义

概念：封装把数据细节隐藏起来，只暴露必要的接口，使外部无法破坏对象内部状态（不变量），便于后续修改实现而不影响调用方。

要点：

- 数据成员通常设为 private，通过 getter/setter 暴露。
- 接口稳定是封装的核心价值。
- 封装与性能无关，只是编译期访问控制。

示例：

```

class BankAccount {
public:
    bool withdraw(double amount) {           // 通过接口校验
        if (amount < 0 || amount > balance_) return false;
        balance_ -= amount;
        return true;
    }
    double balance() const { return balance_; }
private:
    double balance_ = 0.0;                   // 外部无法直接篡改
};

int main() {
    BankAccount a;
    a.withdraw(100);                         // 直接改 balance_ 会绕过校验
}

```

易错点/注意：封装不仅是不让外部改，更是把不变量（invariant）的维护集中到类内部，避免在每处调用点重复校验。

this 指针

this 指针的本质与链式调用

概念： this 是成员函数内隐含的指针，指向调用该函数的对象；在成员函数内部访问成员变量/函数，本质上都是通过 this 访问。

要点：

- 类型：在非 const 成员函数中为 `T* const`（指向不可改），在 const 成员函数中为 `const T* const`。
- 静态成员函数没有 this。
- 返回 `*this` 可实现链式调用（如流插入运算符、Builder 模式）。

示例：

```
#include <iostream>

class Counter {
public:
    Counter(int v = 0) : value_(v) {}
    Counter& add(int n) {                // 返回 *this 实现链式
        value_ += n;
        return *this;
    }
    Counter& reset() {
        value_ = 0;
        return *this;
    }
    int value() const { return value_; }
    void whereAmI() {
        std::cout << this << std::endl;    // 打印对象地址
    }
private:
    int value_;
};

int main() {
    Counter c;
    c.add(5).add(10).reset().add(3);        // 链式调用
    std::cout << c.value() << std::endl;    // 3
    c.whereAmI();                          // this 即 &c
}
```

易错点/注意： 返回 `*this` 时若函数返回类型是 `T`（值）而非 `T&`，会拷贝一份导致链式作用在临时对象上，结果错误；务必返回引用。

构造函数

构造函数与初始化列表

概念： 构造函数是创建对象时自动调用的特殊成员函数，负责初始化对象。初始化列表在进入构造函数体之前完成初始化。

要点：

- 构造函数名与类名相同、无返回类型。
- 初始化列表：`member(value)` 比函数体内赋值更高效，且是初始化 const 成员、引用成员的唯一途径。

- 成员初始化顺序只取决于声明顺序，与初始化列表书写顺序无关。

示例：

```
#include <iostream>
#include <string>

class Widget {
public:
    Widget(int id, const std::string& name) : id_(id), name_(name) {} // 初始化列表
    Widget() : Widget(0, "default") {} // 委托构造 (C++11)
    void show() const {
        std::cout << id_ << ": " << name_ << std::endl;
    }
private:
    int id_;
    std::string name_;
};

int main() {
    Widget w1(1, "a");
    Widget w2; // 调用默认构造
    w1.show(); // 1: a
    w2.show(); // 0: default
}
```

易错点/注意：初始化顺序与声明顺序不一致时，若一个成员依赖另一个成员已初始化，会产生未定义行为；应保持初始化列表顺序与声明顺序一致。

委托构造、explicit 与 =default/=delete

概念：委托构造允许一个构造函数调用本类的另一个构造函数；explicit 禁止隐式类型转换；=default 要求编译器生成默认版本，=delete 禁止使用某函数。

要点：

- 委托构造 (C++11)：C() : C(args) {}。
- explicit 应加在单参数（或可单参调用）构造函数与转换运算符上。
- =default/=delete 是 (C++11) 特性。

示例：

```
class Matrix {
public:
    explicit Matrix(int size) : size_(size) { data_ = new double[size]; } //
    // explicit 禁止隐式转换
    Matrix() : Matrix(10) {} // 委托构造 (C++11)
    Matrix(const Matrix&) = delete; // 禁止拷贝 (C++11)
    Matrix& operator=(const Matrix&) = delete;
    ~Matrix() { delete[] data_; }
private:
    int size_;
    double* data_;
};

void foo(Matrix m) {}
```

```
int main() {
    Matrix m(5);
    // Matrix m2 = 5;      // 错误: explicit 禁止隐式转换
    // foo(5);             // 同样错误
    Matrix m3;             // 委托到 Matrix(10)
    // Matrix m4 = m;      // 错误: 拷贝被 delete
}
```

易错点/注意: 忘记 explicit 会导致意外的隐式转换 (如 `void foo(Matrix)` 被 `foo(5)` 误调用) ; 对可隐式转换的单参构造函数加 explicit 是防御性编程习惯。

析构函数

析构函数的调用时机与顺序

概念: 析构函数在对象生命周期结束时自动调用, 负责释放资源。析构顺序与构造顺序相反。

要点:

- 析构函数名 `~类名`, 无参数无返回类型。
- 栈对象离开作用域、堆对象 `delete`、容器销毁元素时都会调用。
- 成员析构顺序与构造顺序相反; 派生类先析构派生部分, 再析构基类部分。

示例:

```
#include <iostream>

struct Tracer {
    int id;
    Tracer(int i) : id(i) { std::cout << "构造 " << id << std::endl; }
    ~Tracer() { std::cout << "析构 " << id << std::endl; } // 析构函数
};

int main() {
    Tracer a(1);
    {
        Tracer b(2);      // 内层作用域
    }                    // 离开内层作用域, b 析构
    Tracer* p = new Tracer(3);
    delete p;             // 显式 delete 触发析构
    }                    // a 最后析构
// 输出: 构造 1 / 构造 2 / 析构 2 / 构造 3 / 析构 3 / 析构 1
```

易错点/注意: new 出来的对象不 delete 就不会析构, 造成资源泄漏; 栈对象则随作用域自动析构。

虚析构、=default 与 noexcept

概念: 通过基类指针删除派生类对象时, 基类析构函数必须是虚函数, 否则只调用基类析构导致资源泄漏。析构函数默认是 `noexcept(true)` (C++11)。

要点:

- 作为多态基类的类, 析构函数应声明为 virtual。
- `~C() = default;` 要求编译器生成默认析构。
- 除非显式声明 `noexcept(false)`, 析构函数默认不抛异常。

示例：

```
#include <iostream>

class Base {
public:
    virtual ~Base() = default;    // 虚析构（默认 noexcept）
    virtual void f() {}
};

class Derived : public Base {
public:
    ~Derived() override { std::cout << "Derived 析构" << std::endl; }
};

int main() {
    Base* p = new Derived;
    delete p;                    // 正确调用 Derived::~~Derived
}
```

易错点/注意：若 Base 析构非 virtual，`delete` 基类指针时派生类析构不会被调用，典型内存泄漏；为多态基类提供虚析构是必须的。

拷贝构造与拷贝赋值

深拷贝与浅拷贝、自我赋值

概念：拷贝构造用同类型对象初始化新对象；拷贝赋值把已有对象的值赋给另一个已有对象。默认生成的是“逐成员拷贝”（浅拷贝），对持有资源的类通常需要深拷贝。

要点：

- 浅拷贝只复制指针本身，导致两个对象共享同一块资源、可能双重释放。
- 深拷贝复制指针指向的内容。
- 拷贝赋值需处理自我赋值（`if (this != &rhs)`）与异常安全。

示例：

```
#include <cstring>
#include <iostream>

class String {
public:
    String(const char* s = "") {                // 构造函数
        size_ = std::strlen(s);
        data_ = new char[size_ + 1];
        std::strcpy(data_, s);
    }
    String(const String& rhs) {                  // 拷贝构造：深拷贝
        size_ = rhs.size_;
        data_ = new char[size_ + 1];
        std::strcpy(data_, rhs.data_);
    }
    String& operator=(const String& rhs) {       // 拷贝赋值：深拷贝 + 自我赋值保护
        if (this != &rhs) {
            char* tmp = new char[rhs.size_ + 1]; // 先分配，保证异常安全
            std::strcpy(tmp, rhs.data_);
        }
    }
};
```

```

        delete[] data_;
        data_ = tmp;
        size_ = rhs.size_;
    }
    return *this;
}
~String() { delete[] data_; }
void print() const { std::cout << data_ << std::endl; }
private:
    char* data_;
    size_t size_;
};

int main() {
    String a("hello");
    String b(a);           // 拷贝构造
    String c;
    c = a;                 // 拷贝赋值
    a = a;                 // 自我赋值，安全
    b.print(); c.print();
}

```

易错点/注意：拷贝赋值若先 delete 自己的资源再拷贝 rhs，当 `this == &rhs` 时会删除正在读取的数据；自我赋值检查必不可少。

拷贝省略 (Copy Elision)

概念：编译器可以省略不必要的拷贝/移动构造，例如从临时对象初始化对象时直接就地构造。C++17 起某些场景强制省略。

要点：

- 返回值优化 (RVO/NRVO)：函数返回局部对象时直接构造到目标位置。
- C++17 起，纯右值初始化对象的拷贝/移动被强制省略 (guaranteed copy elision)。
- 依赖拷贝构造副作用（如计数）的代码在省略下行为不同，应避免。

示例：

```

#include <iostream>

struct S {
    S() { std::cout << "构造" << std::endl; }
    S(const S&) { std::cout << "拷贝" << std::endl; }
};

S make() {
    return S{};           // 可能直接就地构造，无拷贝
}

int main() {
    S s = make();         // C++17 起强制省略拷贝/移动
}
// 输出（现代编译器）：只打印一次"构造"

```

易错点/注意：即使拷贝构造被 delete，`S s = S{};` 在 C++17 也能编译（因为无实际拷贝发生）；但不要依赖拷贝/移动构造被调用的副作用。

移动构造与移动赋值

移动语义的实现 (C++11)

概念：移动构造/赋值把资源从源对象"转移"到目标对象，源对象此后处于"有效但未指定"状态，通常开销远小于深拷贝。

要点：

- 移动构造 `T(T&&)`、移动赋值 `T& operator=(T&&)`。
- 实现上窃取源对象指针后把源对象指针置空，避免双重释放。
- 移动操作应标记 `noexcept`，以支持容器扩容时的强异常保证。

示例：

```
#include <utility>
#include <cstring>

class String {
public:
    String(const char* s = "") {
        size_ = std::strlen(s);
        data_ = new char[size_ + 1];
        std::strcpy(data_, s);
    }
    String(String&& rhs) noexcept {           // 移动构造 (C++11)
        data_ = rhs.data_;                   // 窃取资源
        size_ = rhs.size_;
        rhs.data_ = nullptr;                 // 源对象置空
        rhs.size_ = 0;
    }
    String& operator=(String&& rhs) noexcept { // 移动赋值
        if (this != &rhs) {
            delete[] data_;
            data_ = rhs.data_;
            size_ = rhs.size_;
            rhs.data_ = nullptr;
            rhs.size_ = 0;
        }
        return *this;
    }
    ~String() { delete[] data_; }
private:
    char* data_;
    size_t size_;
};

int main() {
    String a("hello");
    String b(std::move(a)); // 移动构造，a 被掏空
    String c;
    c = std::move(b);      // 移动赋值
}
```

易错点/注意：移动后源对象处于"有效但未指定"状态，除析构和重新赋值外不要依赖其值；再次使用前应重新赋值或重置。

Rule of Three / Five / Zero

何时需要自定义拷贝/移动/析构

概念：若类需要自定义析构、拷贝构造或拷贝赋值中的一个，通常三者都需要（Rule of Three）；加入移动语义后扩展为五者（Rule of Five）；若类只由自动管理资源的成员组成，则应全部交给编译器（Rule of Zero）。

要点：

规则	内容
Rule of Three	自定义析构/拷贝构造/拷贝赋值其一 → 三者都应自定义
Rule of Five (C++11)	在上述基础上加上移动构造与移动赋值
Rule of Zero	不直接管理资源的类不写任何拷贝/移动/析构，用智能指针/容器管理

示例：

```
#include <memory>

// Rule of Zero: 用 unique_ptr 管理资源，无需手写拷贝/移动/析构
class Buffer {
public:
    Buffer(size_t n) : data_(std::make_unique<int[]>(n)), size_(n) {}
private:
    std::unique_ptr<int[]> data_;
    size_t size_;
};
// 该类的拷贝被自动删除，移动自动生成，行为正确
```

易错点/注意：手写资源管理且只写析构而漏写拷贝构造/赋值时，默认的浅拷贝会导致双重释放；反之写了拷贝却漏析构则泄漏。优先采用 Rule of Zero。

static 成员

静态数据成员与静态成员函数

概念：static 成员属于类本身而非某个对象，所有对象共享一份。静态数据成员需在类外定义（(C++17) 起 inline 可类内定义），静态成员函数无 this 指针。

要点：

- 静态数据成员：类内仅声明，类外定义一次（`int C::n = 0;`）。
- `static constexpr` 整型/enum 类型可在类内初始化；(C++17) 起 `static constexpr` 类内即内联定义，不再需要类外定义。
- 静态成员函数只能访问静态成员，不能访问非静态成员，也没有 this。

示例：

```
#include <iostream>

class Counter {
public:
    Counter() { ++count_; }
```

```

~Counter() { --count_; }
static int getCount() { return count_; } // 静态成员函数
private:
    static int count_; // 类内声明
    static constexpr int MAX = 100; // 类内初始化 (C++17)
};

int Counter::count_ = 0; // 类外定义

int main() {
    Counter a, b;
    std::cout << Counter::getCount() << std::endl; // 2 (通过类名访问)
    std::cout << Counter::MAX << std::endl; // 100
}

```

易错点/注意：非 const 的静态数据成员若不在类外定义，链接时报"undefined reference"；静态成员函数不能调用非静态成员（没有 this）。

const 成员函数与 mutable

const 重载与 mutable

概念：const 成员函数承诺不修改对象，this 类型为 `const T*`。同一函数可按 const 与否重载，const 对象调用 const 版本。mutable 成员即使在 const 成员函数中也可被修改。

要点：

- const 成员函数只能调用 const 成员函数、读非 mutable 成员。
- 按 const 重载常用于下标运算符（返回引用 vs const 引用）。
- mutable 用于缓存、锁、计数器等"逻辑上不改变对象"的成员。

示例：

```

#include <string>

class Text {
public:
    Text() = default;
    const std::string& get() const { // const 版本
        ++access_count_; // mutable 成员可修改
        return s_;
    }
    std::string& get() { // 非 const 版本（重载）
        return s_;
    }
private:
    std::string s_ = "hello";
    mutable int access_count_ = 0; // mutable 打破 const
};

int main() {
    const Text ct;
    ct.get(); // 调用 const 版本
    Text t;
    t.get() = "world"; // 调用非 const 版本，可修改
}

```

易错点/注意：const 成员函数内不能修改普通成员（编译错误），但可通过 const_cast 或指针间接突破——这属于未定义行为，不要这么做；mutable 才是合法手段。

friend

友元函数、友元类与友元成员函数

概念：friend 声明授权某个函数或类访问本类的私有/保护成员。友元关系是单向的、不被继承的、不传递的。

要点：

- 友元函数：可以是普通函数或运算符重载（如流插入 `operator<<`）。
- 友元类：该类的所有成员函数都可访问本类私有成员。
- 友元成员函数：仅某个类的某个成员函数被授权。
- 友元破坏封装，应谨慎、有明确理由时使用。

示例：

```
#include <iostream>

class Accessor; // 前置声明

class Vector2 {
public:
    Vector2(double x, double y) : x_(x), y_(y) {}
    friend std::ostream& operator<<(std::ostream& os, const Vector2& v); // 友元函数
    friend class Accessor; // 友元类
    friend void Accessor::peek(const Vector2&); // 友元成员函数
private:
    double x_, y_;
};

std::ostream& operator<<(std::ostream& os, const Vector2& v) {
    return os << "(" << v.x_ << ", " << v.y_ << ")"; // 访问私有成员
}

class Accessor {
public:
    void peek(const Vector2& v) { std::cout << v.x_ << v.y_ << std::endl; } // 友元成员函数
};

int main() {
    Vector2 v(1, 2);
    std::cout << v << std::endl; // (1, 2)
    Accessor a; a.peek(v); // 12
}
```

易错点/注意：友元成员函数的声明需要类的前置声明且顺序正确，较繁琐；一般优先用成员函数或非友元接口实现，流插入运算符是典型例外。

继承基础

继承语法与访问权限影响

概念：继承让派生类复用并扩展基类。继承方式（public/protected/private）决定基类成员在派生类中的最高可见级别。

要点：

继承方式	基类 public	基类 protected	基类 private
public 继承	public	protected	不可访问
protected 继承	protected	protected	不可访问
private 继承	private	private	不可访问

- 最常用的是 public 继承，表示"is-a"关系。
- 派生类构造函数先构造基类部分，析构先析构派生类部分。

示例：

```
#include <iostream>

class Animal {
public:
    Animal() { std::cout << "Animal 构造" << std::endl; }
    ~Animal() { std::cout << "Animal 析构" << std::endl; }
    void breathe() { std::cout << "呼吸" << std::endl; }
protected:
    int legs_ = 0;
};

class Dog : public Animal {           // public 继承
public:
    Dog() { std::cout << "Dog 构造" << std::endl; }
    ~Dog() { std::cout << "Dog 析构" << std::endl; }
};

int main() {
    Dog d;                            // Animal 构造 → Dog 构造
    d.breathe();                      // 继承的 public 成员
}
```

易错点/注意：private 继承与 protected 继承会缩小基类成员的可见性，通常不推荐；构造顺序为基类→派生类，析构顺序为派生类→基类。

名字隐藏与 using 引入

概念：派生类中与基类同名的成员会"隐藏"基类同名成员（即使参数不同、即使是重载集）。可用 `using 基类::名字` 将基类重载引入派生类作用域。

要点：

- 名字隐藏不构成重载（重载只发生在同一作用域）。
- `using Base::f;` 把基类所有名为 f 的重载引入派生类。
- 想覆盖单个基类函数时需注意其余重载会被隐藏。

示例：

```

#include <iostream>

class Base {
public:
    void f(int) { std::cout << "f(int)" << std::endl; }
    void f(double) { std::cout << "f(double)" << std::endl; }
};

class Derived : public Base {
public:
    using Base::f; // 引入基类所有 f 重载
    void f(const char*) { std::cout << "f(const char*)" << std::endl; }
};

int main() {
    Derived d;
    d.f(1); // f(int) (若不加 using 会编译错误)
    d.f(3.14); // f(double)
    d.f("hi"); // f(const char*)
}

```

易错点/注意：去掉 `using Base::f;` 后，`d.f(1)` 因名字隐藏而报错——派生类的 `f` 隐藏了基类所有 `f`。这是常见坑。

虚函数与多态

虚函数与动态绑定

概念：虚函数通过基类指针/引用调用时，实际调用的是对象的动态类型对应版本，即动态绑定（多态）。这是运行时多态的基础。

要点：

- 用 `virtual` 声明；通过指针/引用调用才有多态，通过值调用仍是静态绑定。
- 派生类重写时应写 `override` (C++11) 防误写。
- 实现原理：含虚函数的类对象有虚指针（`vp`tr），指向虚表（`v`table），虚表中存虚函数地址。

示例：

```

#include <iostream>

class Shape {
public:
    virtual double area() const { return 0.0; } // 虚函数
    virtual ~Shape() = default; // 虚析构
};

class Circle : public Shape {
public:
    Circle(double r) : r_(r) {}
    double area() const override { return 3.14159 * r_ * r_; }
private:
    double r_;
};

int main() {

```

```

circle c(2);
Shape* p = &c; // 基类指针指向派生类
std::cout << p->area() << std::endl; // 动态绑定, 调用 Circle::area
Shape s = c; // 值拷贝: 对象切片, 静态类型 Shape
std::cout << s.area() << std::endl; // 0.0 (调用 Shape::area)
}

```

易错点/注意: 多态必须通过指针或引用; 按值传递/赋值发生"切片" (slicing), 只保留基类部分, 丢失派生类行为。

override 与 final

防止误写与禁止继承/重写 (C++11)

概念: override 显式声明"本函数重写基类虚函数", 签名不符时编译器报错; final 可用于类 (禁止被继承) 或虚函数 (禁止被进一步重写)。

要点:

- override 必须加在派生类重写函数上, 可避免拼错、参数类型不符、const 缺失等静默错误。
- final 加在类名后禁止继承; 加在虚函数声明后禁止重写。
- override/final 是上下文关键字, 可作普通标识符用 (但不推荐)。

示例:

```

class Base {
public:
    virtual void f(int) const {}
    virtual void g() {}
};

class Derived : public Base {
public:
    void f(int) const override {} // 正确重写
    // void f(double) override {} // 错误: 签名不符, 编译报错
    void g() final {} // 禁止进一步重写
};

class Leaf final {}; // 禁止被继承
// class Sub : public Leaf {}; // 错误: Leaf 是 final

```

易错点/注意: 漏写 const 或参数类型写错时, 原函数是"新增同名函数"而非重写, 运行结果不符合预期; 用 override 让编译器帮你检查。

纯虚函数与抽象类

接口设计

概念: 纯虚函数声明为 `= 0`, 含纯虚函数的类是抽象类, 不能实例化。抽象类用于定义接口, 派生类必须实现所有纯虚函数才能实例化。

要点:

- 纯虚函数可以有实现, 但派生类仍需重写才能实例化。
- 抽象类可包含非纯虚函数与数据成员。
- 抽象类指针/引用可用于多态。

示例：

```
#include <iostream>

class IShape {                                // 接口：全部纯虚
public:
    virtual double area() const = 0; // 纯虚函数
    virtual ~IShape() = default;
};

class Square : public IShape {
public:
    Square(double s) : s_(s) {}
    double area() const override { return s_ * s_; }
private:
    double s_;
};

int main() {
    // IShape s;                                // 错误：抽象类不能实例化
    Square sq(3);
    IShape* p = &sq;                            // 抽象类指针可以
    std::cout << p->area() << std::endl; // 9
}
```

易错点/注意：派生类若未实现全部纯虚函数，仍为抽象类，无法实例化；抽象类必须提供虚析构，否则通过基类指针删除派生对象有泄漏风险。

多重继承与虚继承

菱形继承与虚基类

概念：多重继承让一个类继承多个基类。菱形继承会导致共同基类子对象重复。虚继承（virtual 基类）使共同基类在最终派生类中只保留一份。

要点：

- 普通多重继承中，共同基类会存在多份，访问其成员会歧义。
- 虚继承：class B : virtual public A，使 A 在菱形最底层只有一份。
- 虚基类由"最派生类"负责初始化，而非直接派生类。

示例：

```
#include <iostream>

struct Base {
    int x = 0;
    Base(int v) : x(v) { std::cout << "Base(" << v << ")" << std::endl; }
};

struct A : virtual public Base { // 虚继承
    A(int v) : Base(v) {}
};

struct B : virtual public Base {
    B(int v) : Base(v) {}
};
```

```

struct C : public A, public B {    // 最派生类负责初始化虚基类
    C() : Base(42), A(1), B(2) {} // Base 只被初始化一次
};

int main() {
    C c;
    std::cout << c.x << std::endl; // 42 (只有一份 Base)
}
// 输出: Base(42)

```

易错点/注意：虚基类的初始化参数由最派生类决定，A/B 构造函数中对 Base 的初始化在 C 场景下被忽略；直接派生类对虚基类的初始化仅在其自身作为最派生类时生效。

运算符重载

规则与常用运算符

概念：运算符重载为用户自定义类型提供与内置类型一致的运算符语法。本质是名为 operator@ 的函数（成员或非成员）。

要点：

- 至少有一个操作数是类类型；不能创造新运算符；不能改变优先级/结合性/操作数个数。
- 赋值、下标、调用、成员访问等必须是成员函数；算术、比较、流插入常用非成员（配合友元）。
- 不可重载：`::`、`.*`、`.`、`?:`、`sizeof`、`typeid` 等。

示例：

```

#include <iostream>

class Vec {
public:
    Vec(int x = 0, int y = 0) : x_(x), y_(y) {}
    Vec operator+(const Vec& rhs) const { return Vec(x_ + rhs.x_, y_ + rhs.y_); }
} // 算术
    bool operator==(const Vec& rhs) const { return x_ == rhs.x_ && y_ == rhs.y_; }
} // 比较
    Vec& operator+=(const Vec& rhs) { x_ += rhs.x_; y_ += rhs.y_; return *this; }
} // 赋值类
    int& operator[](int i) { return i == 0 ? x_ : y_; } // 下标（非 const）
    int operator()(int i) const { return i == 0 ? x_ : y_; } // 调用运算符（仿函数）
    Vec& operator++() { ++x_; ++y_; return *this; } // 前置自增
    Vec operator++(int) { Vec t = *this; ++*this; return t; } // 后置自增（哑元
int)
    friend std::ostream& operator<<(std::ostream& os, const Vec& v) { // 流插入
        (友元)
        return os << "(" << v.x_ << ", " << v.y_ << ")";
    }
private:
    int x_, y_;
};

int main() {
    Vec a(1, 2), b(3, 4);
    Vec c = a + b; // (4, 6)
    a += b; // a 变为 (4, 6)
    std::cout << c << std::endl;
}

```

```
std::cout << (a == c) << std::endl; // 1
}
```

易错点/注意：后置自增/自减要带一个 int 哑元参数以区别于前置；下标运算符常提供 const 与非 const 两个重载。

类型转换运算符与不可重载运算符

概念：类型转换运算符 `operator 目标类型()` 定义对象到其他类型的隐式转换。某些运算符（如成员访问 `.`、作用域 `::`、三元 `?:`）不可重载。

要点：

- 转换运算符无返回类型、无参数，通常是 const 成员函数。
- 常配合 explicit 使用以避免意外隐式转换。
- 不可重载：`.`、`.*`、`::`、`?:`、`sizeof`、`typeid`、`#`、`##`。

示例：

```
#include <iostream>

class Number {
public:
    Number(int v) : v_(v) {}
    explicit operator int() const { return v_; }    // 显式转换运算符 (C++11 起支持 explicit)
    explicit operator bool() const { return v_ != 0; }
private:
    int v_;
};

int main() {
    Number n(10);
    int x = static_cast<int>(n);    // 需显式转换
    if (n) { std::cout << "非零" << std::endl; } // bool 语境允许显式转换
    std::cout << x << std::endl;    // 10
}
```

易错点/注意：转换运算符易引发歧义重载（尤其搭配转换构造函数）；非必要时尽量加 explicit，或只定义转换到 bool。

嵌套类与局部类

嵌套类与局部类

概念：嵌套类是定义在另一个类内部的类，用于把实现相关的辅助类型限制在宿主类作用域内。局部类是定义在函数体内部的类。

要点：

- 嵌套类默认不是宿主类的友元，不能自动访问宿主类私有成员（除非显式声明）。
- 局部类只能使用外层作用域的静态/枚举类型，不能有静态数据成员，不能使用局部非静态变量。
- 二者均较少使用，多用于实现细节隔离。

示例：

```
#include <iostream>
```

```

class Outer {
public:
    class Inner {                // 嵌套类
    public:
        int val = 7;
        void show() { std::cout << "Inner" << std::endl; }
    };
    Inner make() { return Inner{}; }
};

int main() {
    Outer::Inner i = Outer{}.make(); // 用 Outer::Inner 引用嵌套类
    i.show();

    class Local {                // 局部类
    public:
        int square(int x) { return x * x; }
    };
    Local l;
    std::cout << l.square(4) << std::endl; // 16
}

```

易错点/注意：嵌套类默认不是宿主类的友元，不能自动访问宿主类私有成员；局部类成员函数定义在函数内，实践中心用。

面向对象设计

三特性与 SOLID 原则

概念：面向对象三大特性：封装（隐藏细节）、继承（复用与扩展）、多态（同一接口不同实现）。SOLID 是五个设计原则，帮助写出易维护、可扩展的代码。

要点：

- 单一职责（SRP）：一个类只做一件事。
- 开闭原则（OCP）：对扩展开放、对修改关闭。
- 里氏替换（LSP）：派生类必须能替换基类。
- 接口隔离（ISP）：接口小而专。
- 依赖倒置（DIP）：依赖抽象而非具体实现。
- 组合优于继承：优先用"持有对象"复用而非继承，降低耦合。

示例：

```

#include <iostream>
#include <vector>
#include <memory>

// 依赖抽象（DIP）：通过接口编程
class Shape {                    // 抽象接口
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {    // 对扩展开放（OCP）
public:

```

```
Circle(double r) : r_(r) {}
double area() const override { return 3.14159 * r_ * r_; }
private:
    double r_;
};

double totalArea(const std::vector<std::unique_ptr<Shape>>& shapes) { // 依赖抽象
    double sum = 0;
    for (auto& s : shapes) sum += s->area();
    return sum;
}

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1));
    std::cout << totalArea(shapes) << std::endl; // 无需修改 totalArea 即可扩展新形状
}
```

易错点/注意：组合优于继承不否定继承——"is-a"用继承，"has-a"用组合；滥用深继承层次会导致脆弱基类问题。

本部分小结

易忘点	说明
struct vs class	仅默认访问级别不同（public vs private）
初始化顺序	成员按声明顺序初始化，与初始化列表书写顺序无关
多态前提	必须通过指针/引用 + virtual；值传递会切片
虚析构	多态基类必须有虚析构，否则 delete 基类指针会泄漏
自我赋值	拷贝赋值必须处理 <code>this == &rhs</code>
移动后	源对象"有效但未指定"，仅可析构/重新赋值
名字隐藏	派生类同名成员隐藏基类全部重载，需 using 引入
虚继承	最派生类负责初始化虚基类
不可重载	<code>.</code> 、 <code>.*</code> 、 <code>::</code> 、 <code>?::</code> 、 <code>sizeof</code> 、 <code>typeid</code>

第六部分：模板与泛型编程

函数模板

函数模板定义与实参推导

概念：函数模板是参数化类型的函数蓝图，编译器根据实参推导模板参数并实例化出具体函数。泛型编程即通过模板让同一份代码适用于多种类型。

要点：

- `template<typename T>` 声明模板参数，T 是类型占位符。
- 实参推导：从函数实参推断 T，可隐式推导或显式指定 `func<int>(...)`。

- 多个模板参数用逗号分隔。

示例：

```
#include <iostream>

template<typename T>
T max(T a, T b) { // 函数模板
    return a > b ? a : b;
}

template<typename T>
void print(T v) {
    std::cout << v << std::endl;
}

int main() {
    std::cout << max(3, 5) << std::endl; // 推导 T=int
    std::cout << max<double>(3, 5.5) << std::endl; // 显式指定 T=double
    std::cout << max(1.5, 2.5) << std::endl; // T=double
    print("hello"); // T=const char*
}
```

易错点/注意： `max(3, 5.5)` 会因参数类型不一致推导失败（一个 int 一个 double），需显式指定或转换实参；模板参数 T 与实参类型不一致时同样报错。

函数模板重载

概念：函数模板可与普通函数、其他模板重载；重载决议优先选择最匹配的版本，同样匹配时普通函数通常优先于模板实例化。

要点：

- 模板与普通函数可同名共存。
- 更特化的版本优先被选中。
- 指针类型常需要专门重载（避免按地址比较）。

示例：

```
#include <iostream>
#include <cstring>

template<typename T>
T max(T a, T b) { return a > b ? a : b; }

const char* max(const char* a, const char* b) { // 普通函数重载（更特化）
    return std::strcmp(a, b) > 0 ? a : b;
}

int main() {
    std::cout << max(1, 2) << std::endl; // 模板实例化
    std::cout << max("ab", "bc") << std::endl; // 调用普通函数（比较内容）
}
```

易错点/注意：若不提供 `const char*` 重载，模板会按指针比较（比较地址而非内容），结果错误；为指针类型提供专门重载是常见做法。

多个模板参数与返回类型推导

概念：函数模板可多个类型参数；返回类型可显式写出，也可用 `decltype` 或 `auto` 从实参推导（C++14 起 `auto` 返回类型推导）。

要点：

- 多参数：`template<typename T, typename U>`，每个参数独立推导。
- 返回类型推导：后置 `-> decltype(a + b)` 或 `auto` (C++14)。
- 返回类型不参与实参推导，需显式指定或借助 `auto/decltype`。

示例：

```
#include <iostream>

template<typename T, typename U>           // 多个模板参数
auto add(T a, U b) -> decltype(a + b) {   // 后置返回类型推导
    return a + b;
}

template<typename T, typename U>           // C++14 auto 返回类型推导
auto mul(T a, U b) {
    return a * b;
}

int main() {
    std::cout << add(1, 2.5) << std::endl; // 3.5 (返回 double)
    std::cout << mul(3, 4) << std::endl;   // 12
}
```

易错点/注意：返回类型若直接写成 `T`，当 `T`、`U` 类型不同时（如 `add(1, 2.5)`）可能发生不期望的窄化；用 `decltype/auto` 让编译器推导更稳妥。

类模板

类模板与成员函数类外定义

概念：类模板是参数化类型的类，成员函数在类外定义时需带 `template<...>` 前缀并用 `类名<T>::` 限定。

要点：

- 类模板实例化需显式提供实参；C++17 起可用类模板实参推导（CTAD）自动推导。
- 成员函数类外定义语法：`template<typename T> 返回类型 类名<T>::函数名(...) { ... }`。
- CTAD：`std::pair p{1, 2.0};` 无需写模板实参。

示例：

```
#include <iostream>

template<typename T>
class Box {
public:
    Box(T v) : value_(v) {}
    T get() const;           // 类内声明
    void set(T v) { value_ = v; }
private:
    T value_;
```

```

    T value_;
};

template<typename T>                // 类外定义
T Box<T>::get() const {
    return value_;
}

int main() {
    Box<int> b(42);                  // 显式指定 T=int
    std::cout << b.get() << std::endl;

    Box b2(3.14);                   // C++17 CTAD 推导 T=double
    std::cout << b2.get() << std::endl;
}

```

易错点/注意：类模板成员函数类外定义时漏写 `template<typename T>` 或漏写 `<T>` 都会编译错误；类模板定义必须放在头文件（见"分离编译"）。

类模板的默认实参 (C++11)

概念：类模板参数可有默认值，实例化时省略对应实参则使用默认类型。默认实参需从右往左给出。

要点：

- 语法：`template<typename T, typename Alloc = std::allocator<T>>` // 默认实参
- 常用在容器类（如分配器类型）。
- 与函数默认参数类似，靠右的参数先给默认值。

示例：

```

#include <vector>
#include <iostream>

template<typename T, typename Alloc = std::allocator<T>> // 默认实参
class MyVec {
public:
    void push(const T& v) { data_.push_back(v); }
    size_t size() const { return data_.size(); }
private:
    std::vector<T, Alloc> data_;
};

int main() {
    MyVec<int> v;                // Alloc 使用默认
    v.push(1);
    std::cout << v.size() << std::endl; // 1
}

```

易错点/注意：默认实参只能从右往左给，不能跳过前面的参数只给后面的默认值。

类模板中的静态成员

概念：类模板的静态成员是"每个实例化类型各一份"，即 `C<int>` 与 `C<double>` 拥有各自独立的静态成员。

要点：

- 静态成员在类外定义时需带模板声明：`template<typename T> int C<T>::count = 0;`。
- 不同模板实参对应不同静态成员实例。
- 静态成员函数定义同样带 `template` 前缀。

示例：

```
#include <iostream>

template<typename T>
class Counter {
public:
    Counter() { ++count_; }
    static int getCount() { return count_; }
private:
    static int count_;
};

template<typename T>                // 类外定义
int Counter<T>::count_ = 0;

int main() {
    Counter<int> a, b;
    Counter<double> c;
    std::cout << Counter<int>::getCount() << std::endl;    // 2
    std::cout << Counter<double>::getCount() << std::endl; // 1
}
```

易错点/注意：类模板静态成员类外定义时漏写 `template<typename T>` 会报错；不同实参类型的静态成员相互独立，不要误以为共享。

模板特化

全特化与偏特化

概念：模板特化针对特定模板实参提供专门实现。全特化指定所有模板参数；偏特化只指定部分/限定参数（仅类模板支持，函数模板不支持偏特化，用重载替代）。

要点：

- 全特化：`template<> class C<int> {...}`。
- 偏特化：`template<typename T> class C<T*> {...}`（指针特化）。
- 函数模板只支持全特化，偏特化需求用重载实现。

示例：

```
#include <iostream>

template<typename T>
class Printer {
public:
```

```

    void print(const T& v) { std::cout << "通用: " << v << std::endl; }
};

template<> // 全特化
class Printer<const char*> {
public:
    void print(const char* v) { std::cout << "特化: " << v << std::endl; }
};

template<typename T> // 偏特化: 指针
class Printer<T*> {
public:
    void print(const T* v) { std::cout << "指针: " << *v << std::endl; }
};

int main() {
    Printer<int> p1; p1.print(1); // 通用
    Printer<const char*> p2; p2.print("hi"); // 特化
    int x = 5;
    Printer<int*> p3; p3.print(&x); // 指针 (偏特化)
}

```

易错点/注意: 函数模板没有偏特化; 想对 `T*` 提供不同实现, 用函数重载 `template<typename T> void f(T* p)` 代替。

非类型模板参数

值模板参数与 C++20 类类型非类型参数

概念: 非类型模板参数是编译期常量 (整数、枚举、指针、引用等), 用于编译期确定大小等场景。

要点:

- 如 `template<int N>`, `N` 是编译期常量。
- 常见用途: `std::array<T, N>`。
- C++20 起允许类类型作为非类型模板参数 (需满足强结构化相等条件), 可用于字符串字面量等。

示例:

```

#include <array>
#include <iostream>

template<typename T, int N> // 非类型参数 N
class FixedArray {
public:
    T& operator[](int i) { return data_[i]; }
    int size() const { return N; }
private:
    T data_[N];
};

int main() {
    FixedArray<int, 5> a; // N=5 编译期确定
    a[0] = 10;
    std::cout << a.size() << std::endl; // 5
    std::array<int, 3> b{1, 2, 3}; // 标准库例子
    std::cout << b[1] << std::endl; // 2
}

```

```
}
```

易错点/注意：非类型参数必须是编译期常量表达式，运行期变量不能作为非类型模板参数（`int n; FixedArray<int, n>` 编译错误）。

默认模板参数与模板模板参数

默认模板参数与模板模板参数简介

概念：默认模板参数为模板参数提供默认类型/值；模板模板参数是"以模板为实参"的参数，用于把容器模板等作为参数传递。

要点：

- 默认模板参数从右往左给出。
- 模板模板参数：`template<template<typename> class C>`，C 本身是模板。
- C++17 起可用 `typename` 声明模板模板参数。

示例：

```
#include <vector>
#include <list>
#include <iostream>

template<typename T, typename Alloc = std::allocator<T>> // 默认模板参数
class Stack {
public:
    void push(const T& v) { data_.push_back(v); }
    size_t size() const { return data_.size(); }
private:
    std::vector<T, Alloc> data_;
};

template<typename T, template<typename, typename> class Container = std::vector>
// 模板模板参数
class Wrapper {
public:
    void add(const T& v) { c_.push_back(v); }
    size_t size() const { return c_.size(); }
private:
    Container<T, std::allocator<T>> c_;
};

int main() {
    Stack<int> s; s.push(1);
    Wrapper<int> w; w.add(2); // 用 vector
    Wrapper<int, std::list> w2; w2.add(3); // 用 list
    std::cout << w.size() << w2.size() << std::endl; // 11
}
```

易错点/注意：模板模板参数的形参列表要与实参模板匹配；`std::vector` 有两个模板参数，模板模板参数声明为 `template<typename, typename> class` 才能匹配。

可变参数模板

参数包展开与递归终止

概念：可变参数模板接受任意数量的模板参数/函数参数，通过递归展开参数包逐个处理，直至终止。

要点：

- `template<typename... Args>` 定义参数包；`Args...` 展开。
- 递归模式：处理第一个 + 递归剩余，需要一个终止（0 参数或 1 参数）重载。
- `sizeof...(Args)` 获取参数包中参数个数。

示例：

```
#include <iostream>

void print() { std::cout << std::endl; }    // 递归终止

template<typename T, typename... Args>
void print(T first, Args... rest) {        // 递归展开
    std::cout << first << " ";
    print(rest...);                        // 递归调用剩余参数
}

template<typename... Args>
size_t count(Args... args) {
    return sizeof...(args);               // 参数个数
}

int main() {
    print(1, 2.5, "hi", 'c');    // 1 2.5 hi c
    std::cout << count(1, 2, 3) << std::endl; // 3
}
```

易错点/注意：忘记递归终止重载会编译失败（递归无终点）；递归展开在编译期完成，参数多时编译时间较长。

C++17 折叠表达式

概念：折叠表达式（fold expression）把二元运算符应用到参数包的每个元素，替代冗长的递归展开。

要点：

- 一元左折叠（`... op pack`）、一元右折叠（`pack op ...`）。
- 二元折叠带初值：（`init op ... op pack`）。
- 支持 `,`、`+`、`&&`、`||` 等运算符。

示例：

```
#include <iostream>

template<typename... Args>
auto sum(Args... args) {
    return (args + ...);                // 一元右折叠 (C++17)
}

template<typename... Args>
auto sumFrom(int init, Args... args) {
    return (init + ... + args);        // 二元左折叠，带初值
}
```

```

}

template<typename... Args>
bool all(Args... args) {
    return (args && ...);           // 逻辑与折叠
}

int main() {
    std::cout << sum(1, 2, 3, 4) << std::endl;           // 10
    std::cout << sumFrom(100, 1, 2) << std::endl;         // 103
    std::cout << all(true, true, false) << std::endl;    // 0
}

```

易错点/注意：空参数包的一元折叠仅 `&&`（真）、`||`（假）、`,`（void）合法，`+` 等运算符在空包时编译错误（无单位元）；需要空包支持时用二元折叠带初值。

类型萃取 <type_traits>

常用 traits 与类型变换

概念：`<type_traits>` 提供编译期查询类型属性（`is_xxx`）与类型变换（`remove/add/decay/conditional` 等）的工具，是元编程与 SFINAE 的基础设施。

要点：

- 查询：`is_integral`、`is_floating_point`、`is_pointer`、`is_class`、`is_same` 等，结果 `::value` 或 `_v` (C++17)。
- 变换：`remove_reference`、`remove_const`、`decay`、`conditional`、`enable_if` 等，结果 `::type` 或 `_t` (C++14)。
- 常在模板元编程与 SFINAE 中组合使用。

示例：

```

#include <type_traits>
#include <iostream>
#include <cstdint>

int main() {
    std::cout << std::is_integral<int>::value << std::endl;           // 1
    std::cout << std::is_integral_v<double> << std::endl;             // 0 (C++17)
    std::cout << std::is_same_v<int, int32_t> << std::endl;           // 1

    using Raw = std::remove_reference<int&>::type;                   // int
    using Decayed = std::decay_t<const int&>;                        // int
    using Chosen = std::conditional_t<true, int, double>;           // int

    static_assert(std::is_same_v<Raw, int>);
    static_assert(std::is_same_v<Decayed, int>);
    static_assert(std::is_same_v<Chosen, int>);
    std::cout << "全部通过" << std::endl;
}

```

易错点/注意：`decay` 会同时移除引用与顶层 `const`、并对数组/函数退化为指针；`remove_reference` 只移除引用，两者不可混淆。

更多查询与变换 traits

概念：除整数判断外，`<type_traits>` 还提供大量属性查询（`is_class`、`is_enum`、`is_pointer`、`is_array` 等）与复合类型变换（`rank`、`extent`、`add_pointer` 等）。

要点：

- `is_class` / `is_enum` / `is_pointer` / `is_reference` 判断类别。
- `rank<T>` 返回数组维数、`extent<T, N>` 返回第 N 维长度。
- `add_pointer`、`add_const`、`add_lvalue_reference` 添加限定。
- 都可用 `_v` / `_t` 辅助变量模板简化。

示例：

```
#include <type_traits>
#include <string>
#include <iostream>

int main() {
    std::cout << std::is_class_v<std::string> << std::endl; // 1
    std::cout << std::is_pointer_v<int*> << std::endl; // 1
    std::cout << std::is_array_v<int[3]> << std::endl; // 1
    std::cout << std::rank_v<int[3][4]> << std::endl; // 2（二维）
    std::cout << std::extent_v<int[3][4], 0> << std::endl; // 3（第一维）

    using Ptr = std::add_pointer_t<int>; // int*
    using CRef = std::add_lvalue_reference_t<const int>; // const int&
    static_assert(std::is_same_v<Ptr, int*>);
    static_assert(std::is_same_v<CRef, const int&>);
    std::cout << "ok" << std::endl;
}
```

易错点/注意：`rank` 与 `extent` 仅对数组类型有意义，对非数组类型 `rank_v<int>` 为 0；变换类 traits 的 `_t` 别名在 C++14 才加入，早期需写 `::type`。

SFINAE

替换失败不是错误

概念：SFINAE（Substitution Failure Is Not An Error）：模板实参替换失败时，该候选模板被静默丢弃而非报错，使重载决议能在多个候选间选择。常用于用 `enable_if` 按类型属性启用/禁用函数模板。

要点：

- 替换失败发生在函数签名/返回类型推导阶段，不算错误。
- `std::enable_if<条件, 类型>` 条件为假时无 `type`，导致替换失败。
- 常用于“仅当 T 是整数/浮点/某特征时启用”的重载选择。

示例：

```
#include <type_traits>
#include <iostream>

template<typename T>
std::enable_if_t<std::is_integral_v<T>, T> // 仅整数启用
divide(T a, T b) {
```

```

    return a / b;                                // 整数除法
}

template<typename T>
std::enable_if_t<std::is_floating_point_v<T>, T> // 仅浮点启用
divide(T a, T b) {
    return a / b;                                // 浮点除法
}

int main() {
    std::cout << divide(7, 2) << std::endl;      // 3 (整数)
    std::cout << divide(7.0, 2.0) << std::endl;  // 3.5 (浮点)
}

```

易错点/注意：enable_if 放在返回类型、模板参数或函数参数都可以，但放在函数体内部不起作用（此时已实例化完成）；C++20 可用 requires 更清晰地表达。

void_t 检测惯用法

概念：std::void_t (C++17) 是 SFINAE 的辅助工具，常用于“检测类型是否具备某成员/操作”的惯用法，返回 void 以便配合 enable_if。

要点：

- void_t<...> 恒为 void，仅当模板实参全部合法时才成功替换。
- 用于检测成员函数、类型成员是否存在。
- 与 enable_if 组合可实现按能力分派。

示例：

```

#include <type_traits>
#include <iostream>
#include <vector>

// 检测 T 是否有 value_type 成员类型
template<typename T, typename = void>
struct has_value_type : std::false_type {};

template<typename T>
struct has_value_type<T, std::void_t<typename T::value_type>> : std::true_type
{};

int main() {
    std::cout << has_value_type<std::vector<int>>::value << std::endl; // 1
    std::cout << has_value_type<int>::value << std::endl;             // 0
}

```

易错点/注意：void_t 是 C++17 起才在标准库中提供，C++14 及更早需自行定义

template<typename...> using void_t = void;。

C++20 Concepts 与 requires

概念定义与约束

概念： Concepts (C++20) 用命名约束限制模板参数，requires 子句声明约束，编译期检查更早、错误信息更清晰，是 SFINAE 的更可读替代。

要点：

- `template<typename T> concept 名字 = 约束表达式;`
- `requires` 子句: `template<typename T> requires 约束`。
- `requires` 表达式可检查表达式合法性: `requires(T a) { a + a; }`。
- 对比 SFINAE: 概念可读性、错误信息、重载约束都更优。

示例：

```
#include <concepts>
#include <iostream>

template<typename T>
concept Addable = requires(T a, T b) { // 概念定义: 要求 a+b 合法
    { a + b } -> std::convertible_to<T>;
};

template<Addable T> // 用概念约束
T add(T a, T b) {
    return a + b;
}

template<typename T>
requires std::integral<T> // requires 子句
T twice(T v) { return v * 2; }

int main() {
    std::cout << add(1, 2) << std::endl; // 3
    std::cout << twice(21) << std::endl; // 42
    // add(std::string("a"), std::string("b")) 也合法 (string 可 +)
}
```

易错点/注意： 概念是 C++20 特性，需编译器支持（如 `-std=c++20`）；`requires` 表达式检查的是"表达式合法性"，不代表语义正确。

模板元编程简介

编译期计算与 `static_assert`

概念： 模板元编程（TMP）利用模板递归在编译期完成计算，结果作为常量嵌入程序，运行期零开销。

要点：

- 通过特化实现递归终止（如 `Factorial<0>`）。
- C++11 起 `constexpr` 可替代大部分 TMP，更易读。
- `static_assert(条件, "消息")` 编译期断言，失败则编译报错。

示例：

```
#include <iostream>

// 模板元编程: 编译期阶乘
```

```

template<int N>
struct Factorial {
    static constexpr int value = N * Factorial<N - 1>::value;
};
template<>
struct Factorial<0> { // 递归终止特化
    static constexpr int value = 1;
};

// 现代写法: constexpr 函数 (C++11)
constexpr int fib(int n) {
    return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}

int main() {
    std::cout << Factorial<5>::value << std::endl; // 120 (编译期算好)
    constexpr int f = fib(10); // 编译期计算
    std::cout << f << std::endl; // 55

    static_assert(Factorial<5>::value == 120, "阶乘错误"); // 编译期断言
    static_assert(fib(10) == 55, "斐波那契错误");
}

```

易错点/注意：模板递归层次过深会耗尽编译器资源（有默认深度限制，可用 `-ftemplate-depth` 调整）；现代 C++ 优先 `constexpr`/`constexpr` 而非复杂 TMP。

constexpr 与 constexpr

概念：`constexpr` 函数 (C++11) 在编译期求值时结果可作为常量；`constexpr` (C++20) 强制函数只能在编译期调用，运行期调用直接报错。

要点：

- `constexpr` 函数既可编译期也可运行期调用。
- `constexpr` (C++20) 仅编译期，参数与结果必须是常量。
- 编译期计算能消除运行期开销，结果直接内联为常量。

示例：

```

#include <iostream>

constexpr int square(int x) { // constexpr 函数 (C++11)
    return x * x;
}

constexpr int cube(int x) { // constexpr 函数 (C++20)
    return x * x * x;
}

int main() {
    constexpr int a = square(5); // 编译期求值
    int n = 10;
    int b = square(n); // 运行期也可调用
    constexpr int c = cube(3); // 编译期
    // int d = cube(n); // 错误: constexpr 不能运行期调用
    std::cout << a << " " << b << " " << c << std::endl; // 25 100 27
}

```

易错点/注意：constexpr 是 C++20 特性，需 -std=c++20；constexpr 函数若传入运行期变量则退化为运行期调用，不保证编译期求值。

模板与分离编译问题

定义放头文件与显式实例化

概念：模板只有被实例化时才生成代码，编译器需要在实例化点看到完整定义，因此模板定义通常放在头文件；显式实例化可在源文件中预先实例化特定类型以缩短编译时间。

要点：

- 若模板定义只在 .cpp 中，其他编译单元看不到定义，链接时报未定义引用。
- 解法一：定义放头文件（最常用）。
- 解法二：显式实例化 `template class C<int>;`，限定可用的模板实参集合。

示例：

```
// 头文件 add.h（模板定义放这里）
#ifndef ADD_H
#define ADD_H
template<typename T>
T add(T a, T b) { return a + b; } // 定义必须可见
#endif
```

```
// 显式实例化（某 .cpp 内）
#include "add.h"
template int add<int>(int, int); // 显式实例化 int 版本
template double add<double>(double, double);
```

易错点/注意：把模板定义放到 .cpp 且未显式实例化，是"模板链接错误"最常见的来源；除非确需限制类型并接受显式实例化维护成本，否则一律放头文件。

本部分小结

易忘点	说明
函数模板推导	实参类型不一致时推导失败，需显式指定或转换
函数模板偏特化	不支持，用重载替代
类模板成员外定义	需 template 前缀 + 类名::
非类型参数	必须是编译期常量
折叠表达式	空包时仅 &&、
enable_if	只能用于签名（返回类型/参数/模板参数）
Concepts	C++20，需 -std=c++20
分离编译	模板定义必须放头文件或显式实例化

第七部分：异常处理

异常处理基础

异常机制概述 (throw/try/catch)

概念： C++ 通过 `throw` 抛出异常对象、`try` 块包裹可能出错的代码、`catch` 子句捕获并处理异常。异常机制把"检测错误"和"处理错误"的代码分离，使错误处理逻辑不必散布在每一层调用中。

要点：

- `throw` 表达式：抛出一个异常对象，其类型可以是任意可拷贝类型（通常继承自 `std::exception`）。
- `try { ... } catch (类型 e) { ... }` 中，`try` 块内抛出的异常会与各 `catch` 子句依次匹配，命中后执行对应处理代码。
- 异常未被任何 `catch` 捕获时会沿调用链向上传播，最终若无人捕获则调用 `std::terminate` 终止程序。
- `catch` 的参数通常是引用（`const T&`），避免不必要的拷贝和对象切片。

示例：

```
#include <iostream>
#include <stdexcept>

// 一个可能抛出异常的函数
int safeDivide(int a, int b) {
    if (b == 0) {
        throw std::runtime_error("除数不能为零"); // throw: 抛出异常对象
    }
    return a / b;
}

int main() {
    try {
        int r = safeDivide(10, 0); // 可能抛出异常的调用
        std::cout << "结果: " << r << "\n";
    } catch (const std::runtime_error& e) { // catch: 捕获并处理
        std::cerr << "运行时错误: " << e.what() << "\n";
    }
    std::cout << "程序继续执行\n"; // 异常被处理后程序正常继续
    return 0;
}
```

易错点/注意：

- 不要抛出指向局部对象的指针或引用，栈展开后该对象已析构，导致悬垂引用。
- 未捕获的异常会调用 `std::terminate`，因此 `main` 外层通常应有一个总 `catch` 兜底。
- `catch` 应使用引用（`const T&`）接收；若按值 `catch(T e)`，派生类异常会被切片为 `T`，丢失派生信息。

补充：按值捕获导致切片：

```
#include <iostream>
#include <stdexcept>

struct BaseErr : std::exception {};
struct DerivedErr : BaseErr {};
```

```

int main() {
    // 按值捕获：派生类对象被切片为 BaseErr
    try {
        throw DerivedErr{};
    } catch (BaseErr) { // 按值，切片
        std::cout << "按值捕获 BaseErr (已切片) \n";
    }

    // 按引用捕获：保留动态类型，可多态访问
    try {
        throw DerivedErr{};
    } catch (const BaseErr&) { // 按引用，无切片
        std::cout << "按引用捕获 BaseErr (无切片) \n";
    }
    return 0;
}

```

栈展开 (stack unwinding)

概念：当 `throw` 抛出异常后，运行时会从抛出点沿调用栈逐层向上寻找匹配的 `catch`；在跳过每一层函数时，该层已经构造完成的局部对象会按构造的逆序自动析构，这个过程称为“栈展开”，是 C++ 异常安全的核心机制。

要点：

- 栈展开保证局部对象（含 RAII 对象）在异常传播中被正确释放，避免资源泄漏。
- 析构顺序与构造顺序相反：后构造的先析构。
- 只有“已完全构造”的对象才会被析构；构造中途失败的对象的已构造成员会被自动析构。
- 智能指针、锁（`std::lock_guard`）等 RAII 类型正是依赖栈展开来保证资源释放。

示例：

```

#include <iostream>
#include <stdexcept>
#include <string>

struct Resource {
    std::string name;
    Resource(std::string n) : name(std::move(n)) {
        std::cout << "构造 " << name << "\n";
    }
    ~Resource() { std::cout << "析构 " << name << "\n"; } // 栈展开时自动调用
};

void inner() {
    Resource r2("inner"); // 局部对象
    throw std::runtime_error("inner 抛出异常"); // 异常从这里抛出
}

void outer() {
    Resource r1("outer"); // 外层局部对象
    inner(); // 异常从这里传播出去
}

int main() {
    try {
        outer();
    }
}

```

```

    } catch (const std::exception& e) {
        std::cout << "捕获: " << e.what() << "\n";
    }
    // 输出顺序: 构造 outer -> 构造 inner -> 析构 inner -> 析构 outer -> 捕获...
    return 0;
}

```

易错点/注意:

- 若析构函数自身在栈展开过程中再次抛出异常，程序会直接调用 `std::terminate`（详见"析构函数中的异常"）。
- 裸指针成员不会因栈展开而被 `delete`，必须靠 RAII 包装（如 `std::unique_ptr`）才能自动释放。

异常匹配与重新抛出

catch 匹配规则

概念: 异常被抛出后，按 `try` 块后 `catch` 子句的书写顺序逐个匹配；匹配依据是异常对象的实际类型与 `catch` 声明类型之间的转换关系，遵循"派生类先于基类、兜底 `catch(...)` 放最后"的原则。

要点:

- 按类型匹配: `catch(const T&)` 捕获类型为 `T`（或其公有派生类）的异常。
- 基类捕获派生类: `catch` 声明为基类引用时，能捕获所有派生类异常对象（多态）。
- `catch(...)` 兜底: 可捕获任何类型的异常，但拿不到异常对象本身。
- `catch` 顺序: 派生类子句必须写在基类子句之前，否则派生类异常会被基类子句先"抢走"。

示例:

```

#include <iostream>
#include <stdexcept>

// 自定义异常层次
struct BaseError : std::exception {};
struct DerivedError : BaseError {};

int main() {
    // 1. 按声明顺序匹配: 派生类必须放在基类前面
    try {
        throw DerivedError{};
    } catch (const DerivedError&) {    // 先匹配派生类
        std::cout << "捕获 DerivedError\n";
    } catch (const BaseError&) {      // 再匹配基类
        std::cout << "捕获 BaseError\n";
    } catch (...) {                  // 兜底: 捕获一切
        std::cout << "捕获未知异常\n";
    }

    // 2. 基类引用可以捕获派生类对象
    try {
        throw DerivedError{};
    } catch (const BaseError&) {      // 基类捕获派生类
        std::cout << "基类捕获派生类\n";
    }

    return 0;
}

```

易错点/注意:

- 若把 `catch(const BaseError&)` 写在 `catch(const DerivedError&)` 前面, 派生类异常将永远走不到第二个子句 (编译期可能仅告警)。
- `catch(...)` 必须放在所有 `catch` 子句之后, 否则其后的子句永远不会执行。

异常的重新抛出

概念: 在 `catch` 块中再次抛出异常有两种写法: `throw;` (重新抛出"当前正在处理的异常对象", 保留其原始类型与内容) 和 `throw e;` (抛出 `e` 的一份拷贝, 静态类型被固定为 `e` 的声明类型, 可能发生对象切片)。

要点:

- `throw;` 只能出现在 `catch` 块 (或其调用链) 内, 用于"处理后继续传播原异常"。
- `throw;` 保留原始异常的实际类型, 外部仍可匹配到具体派生类。
- `throw e;` 抛出的是拷贝, 若 `e` 声明为基类引用, 实际抛出的类型被"切片"为基类。
- 常见用法: 在中间层记录日志或补充上下文, 再用 `throw;` 把原异常交给上层。

示例:

```
#include <iostream>
#include <stdexcept>

void rethrowCurrent() {
    try {
        throw std::runtime_error("原始异常");
    } catch (...) {
        std::cout << "记录日志后重新抛出 (throw;) \n";
        throw; // 重新抛出"当前异常", 保留原始类型与信息
    }
}

void rethrowCopy() {
    try {
        throw std::runtime_error("原始异常");
    } catch (const std::exception& e) {
        std::cout << "拷贝一份再抛出 (throw e;) \n";
        throw e; // 抛出 e 的拷贝, 静态类型被"切片"为 std::exception
    }
}

int main() {
    try { rethrowCurrent(); }
    catch (const std::runtime_error& e) { // throw; 保留类型, 能匹配到 runtime_error
        std::cout << "rethrowCurrent 捕获: " << e.what() << "\n";
    }

    try { rethrowCopy(); }
    catch (const std::exception& e) { // throw e; 类型已变为 std::exception
        std::cout << "rethrowCopy 捕获: " << e.what() << "\n";
    }
    return 0;
}
```

易错点/注意:

- 在 `catch(...)` 中只能用 `throw;` (拿不到具名对象, 无法 `throw e;`) 。
- 误用 `throw e;` 会丢失派生类信息, 导致上层只能按基类处理, 应优先使用 `throw;`。

构造与析构中的异常

构造函数中的异常

概念: 构造函数没有返回值, 当初始化失败时唯一的"报错"手段就是抛出异常; 构造函数一旦抛异常, 对象被视为"从未成功构造", 其析构函数不会被调用, 但已经构造完成的成员与基类子对象会被自动析构。

要点:

- 构造函数失败应抛出异常, 而不是返回一个"半初始化"对象。
- 构造抛异常时, 已构造的成员按逆序自动析构, 析构函数本身不执行。
- 用智能指针 (`std::unique_ptr`) 管理资源, 可避免"裸指针 + 手动 delete"在异常路径上泄漏。
- 初始化列表中的某个成员抛异常, 会触发已构造成员的自动析构, 无需手写清理代码。

示例:

```
#include <iostream>
#include <memory>
#include <stdexcept>

class A {
    int* p_;
public:
    A() : p_(new int(42)) { std::cout << "A 构造\n"; }
    ~A() { delete p_; std::cout << "A 析构 (自动释放内存) \n"; }
};

class B {
public:
    B() { std::cout << "B 构造\n"; throw std::runtime_error("B 初始化失败"); }
    ~B() { std::cout << "B 析构\n"; }
};

class Widget {
    std::unique_ptr<A> a_; // 已构造完成的成员
    B b_;                // 该成员构造时抛异常
public:
    // a_ 先构造成功; b_ 抛异常时 a_ 会自动析构, 不会泄漏内存
    Widget() : a_(std::make_unique<A>()), b_() {}
};

int main() {
    try {
        Widget w; // 构造失败: A 自动析构, b_ 的析构不会执行
    } catch (const std::exception& e) {
        std::cout << "捕获: " << e.what() << "\n";
    }
    return 0;
}
```

易错点/注意:

- 构造抛异常后不要依赖析构函数做清理 (它不会执行), 应把清理交给 RAII 成员的析构。

- 若在构造函数里 `new` 了裸指针，抛异常前又没 `delete`，就会泄漏；改用智能指针可根治。

析构函数中的异常

概念：自 C++11 起，析构函数默认是 `noexcept(true)`，即默认承诺不抛异常；若析构函数实际抛出异常，程序会调用 `std::terminate`。析构中抛异常是危险设计，应一律吞掉或改为不抛出。

要点：

- 析构默认 `noexcept(true)`：在析构中抛异常会导致 `std::terminate`。
- 若析构函数在“栈展开”期间抛出异常（即已有异常在传播），同样触发 `std::terminate`。
- 正确做法：析构中捕获所有异常并吞掉，或仅执行绝不抛出的清理操作。
- 需要显式让析构可抛时用 `noexcept(false)`，但绝大多数场景都不应这样做。

示例：

```
#include <iostream>
#include <stdexcept>
#include <type_traits>

struct SafeDtor {
    ~SafeDtor() = default; // 默认析构为 noexcept(true)
};

struct BadDtor {
    ~BadDtor() noexcept(false) { // 显式放开"可抛", 属不良设计
        throw std::runtime_error("析构中抛异常");
    }
};

int main() {
    std::cout << std::boolalpha;
    std::cout << "SafeDtor 析构 noexcept: "
                << std::is_nothrow_destructible<SafeDtor>::value << "\n"; // true
    std::cout << "BadDtor 析构 noexcept: "
                << std::is_nothrow_destructible<BadDtor>::value << "\n"; // false

    try {
        BadDtor b; // 离开作用域析构时抛异常 -> std::terminate
    } catch (const std::exception& e) { // 此 catch 不会命中
        std::cout << "捕获: " << e.what() << "\n";
    }
    return 0;
}
```

易错点/注意：

- 上面 `BadDtor` 示例运行时会调用 `std::terminate`，属于“反面教材”，实际代码切勿模仿。
- 若析构里必须调用可能抛异常的资源释放，用 `try { ... } catch (...) {}` 包住并吞掉。

异常安全保证

异常安全保证（基本保证/强保证/不抛出保证）

概念：异常安全指一个操作在抛出异常后，程序仍处于合法可用的状态。按强度从弱到强分为三个级别：基本保证、强保证、不抛出保证（`nothrow`）。

要点：

- 基本保证：异常发生后对象仍处于"有效但状态不确定"状态，不泄漏资源、不破坏不变量。
- 强保证：操作要么完全成功，要么完全失败且对象保持原样（事务式，`copy-and-swap` 常用）。
- 不抛出保证：操作承诺绝不抛异常，如 `swap`（通常 `noexcept`）、析构、移动构造。
- 强保证常用技巧"copy-and-swap"：先拷贝副本、在副本上修改、成功后再与自身交换。

示例：

```
#include <iostream>
#include <vector>
#include <stdexcept>

class Matrix {
    std::vector<int> data_;
public:
    Matrix(std::vector<int> d) : data_(std::move(d)) {}

    // 强保证：先复制再修改，失败时原对象不变
    void scale(int factor) {
        auto copy = data_;           // 先做副本
        for (auto& x : copy) {
            if (x * factor > 1000) throw std::runtime_error("数值溢出");
            x *= factor;
        }
        data_ = std::move(copy);      // 全部成功后才提交
    }

    // 基本保证：失败后对象仍有效，但状态不确定（日志可能与数据不一致）
    void appendWithLog(int v) {
        data_.push_back(v);           // 可能抛 bad_alloc
        std::cout << "已追加 " << v << "\n"; // 副作用可能已发生
    }

    void print() const {
        for (int x : data_) std::cout << x << ' ';
        std::cout << "\n";
    }
};

int main() {
    Matrix m({1, 2, 3});
    m.scale(10); // 成功 -> 10 20 30
    m.print();

    try { m.scale(100); } catch (const std::exception& e) { // 溢出，强保证：对象不变
        std::cout << "失败： " << e.what() << "\n";
    }
    m.print(); // 仍是 10 20 30
    return 0;
}
```

易错点/注意:

- `std::vector::push_back` 在扩容时若移动构造不 `noexcept` 会退回拷贝, 从而提供强保证; 因此移动构造应尽量标 `noexcept`。
- 强保证并非总能廉价实现; 日常写库代码至少要满足"基本保证", 绝不泄漏资源、绝不破坏不变量。

补充: copy-and-swap 实现强保证 (经典拷贝赋值) :

```
#include <iostream>
#include <vector>
#include <utility>

// 经典 copy-and-swap: 实现强保证的拷贝赋值运算符
class Array {
    std::vector<int> data_;
public:
    explicit Array(std::vector<int> d) : data_(std::move(d)) {}

    // 拷贝赋值: 按值传参 (此处会拷贝), 成功后交换, 异常时原对象不变
    Array& operator=(Array other) noexcept { // other 是临时副本
        swap(other);                        // swap 不抛, 成功后提交
        return *this;
    }

    void swap(Array& other) noexcept {      // 不抛出保证
        using std::swap;
        swap(data_, other.data_);
    }

    void print() const {
        for (int x : data_) std::cout << x << ' ';
        std::cout << "\n";
    }
};

int main() {
    Array a(std::vector<int>{1, 2, 3});
    Array b(std::vector<int>{9, 9, 9, 9});
    a = b; // 拷贝赋值: 强保证, 失败时 a 保持不变
    a.print(); // 9 9 9 9
    return 0;
}
```

noexcept

noexcept 说明符与运算符 (C++11/17)

概念: `noexcept` 有两个角色: 作**说明符**写在函数声明后, 承诺该函数不抛异常; 作**运算符**

`noexcept(expr)` 在编译期判断某表达式是否被声明为不抛。C++17 起 `noexcept` 成为函数类型的一部分。

要点:

- `noexcept` 说明符: `void f() noexcept;` 承诺不抛; 若仍抛异常会直接 `std::terminate`。

- 条件形式 `noexcept(expr)`： `expr` 为真才承诺不抛，常用于模板（如移动构造的条件化 `noexcept`）。
- `noexcept(expr)` 运算符：编译期布尔值，判断表达式是否会抛（常与 `std::declval` 配合）。
- 对移动构造的影响：`std::vector` 扩容时，仅当元素移动构造为 `noexcept` 才使用移动，否则退回拷贝以保证强保证。

示例：

```
#include <iostream>
#include <vector>
#include <type_traits>

// noexcept 说明符：承诺不抛异常
void f1() noexcept { /* 不抛异常 */ }

// 条件 noexcept：表达式为 true 时承诺不抛
template <typename T>
void f2() noexcept(noexcept(T{})) {}

int main() {
    std::cout << std::boolalpha;
    // noexcept 运算符：编译期判断表达式是否声明为不抛
    std::cout << "f1 不抛： " << noexcept(f1()) << "\n";           // true
    std::cout << "f2<int> 不抛： " << noexcept(f2<int>()) << "\n"; // true

    // 对移动构造的影响：noexcept 的移动构造可被 vector 扩容优先使用
    std::cout << "int 移动构造不抛： "
              << std::is_nothrow_move_constructible<int>::value << "\n"; // true
    std::cout << "vector<int> 移动构造不抛： "
              << std::is_nothrow_move_constructible<std::vector<int>>::value
              << "\n"; // true

    // C++17 起 noexcept 是函数类型的一部分，可用于重载区分
    static_assert(noexcept(f1()) == true, "f1 应为 noexcept");
    return 0;
}
```

易错点/注意：

- 声明 `noexcept` 的函数若真的抛异常，`std::terminate` 会被直接调用，`catch` 无法捕获。
- 移动构造/移动赋值、`swap`、析构应尽量标 `noexcept`，能显著提升标准容器性能与安全性。

标准异常体系

标准异常体系

概念：标准库定义了一套以 `std::exception` 为根的异常类层次，分为"逻辑错误"（`logic_error`，可在程序运行前发现）和"运行时错误"（`runtime_error`，只能运行时发现）两大分支，另有 `bad_alloc`、`bad_cast` 等独立分支。

要点：

- 所有标准异常都继承自 `std::exception`，核心接口是 `virtual const char* what() const noexcept`。
- `logic_error` 家族：`invalid_argument`、`domain_error`、`length_error`、`out_of_range`。

- `runtime_error` 家族: `range_error`、`overflow_error`、`underflow_error` (以及常用的 `system_error`) 。
- 独立分支: `bad_alloc` (`new` 失败)、`bad_cast` (`dynamic_cast` 引用转型失败)、`bad_typeid`、`bad_optional_access` 等。

示例:

```
#include <iostream>
#include <stdexcept>
#include <new>
#include <string>

// 标准异常层次 (简图):
// std::exception
// └─ std::logic_error          // 程序逻辑错误 (可提前避免)
//   └─ std::invalid_argument
//   └─ std::domain_error
//   └─ std::length_error
//   └─ std::out_of_range
// └─ std::runtime_error        // 运行时错误 (难以提前避免)
//   └─ std::range_error
//   └─ std::overflow_error
//   └─ std::underflow_error
// └─ std::bad_alloc            // new 分配失败
// └─ std::bad_cast             // dynamic_cast 引用转型失败

int main() {
    // logic_error 家族: out_of_range
    try {
        std::string s = "abc";
        s.at(10); // 越界访问, 抛出 out_of_range
    } catch (const std::out_of_range& e) {
        std::cout << "越界: " << e.what() << "\n";
    } catch (const std::logic_error& e) {
        std::cout << "逻辑错误: " << e.what() << "\n";
    }

    // runtime_error 家族
    try {
        throw std::runtime_error("运行时错误");
    } catch (const std::runtime_error& e) {
        std::cout << "运行时: " << e.what() << "\n";
    }

    // bad_alloc: new 分配失败
    try {
        int* p = new int[10000000000ULL]; // 极可能抛 bad_alloc
        delete[] p;
    } catch (const std::bad_alloc& e) {
        std::cout << "分配失败: " << e.what() << "\n";
    }
    return 0;
}
```

易错点/注意:

- 捕获顺序应先具体后一般：先 `catch(out_of_range)` 再 `catch(logic_error)`，最后 `catch(std::exception)`。
- 需区分 `logic_error`（程序 bug，应修代码）与 `runtime_error`（环境问题，应运行时处理）。

自定义异常类

概念：实际项目通常需要携带业务信息的异常，做法是自定义一个类继承 `std::exception`（或其派生类），并重写 `what()` 返回可读的错误描述。

要点：

- 继承 `std::exception` 并重写 `virtual const char* what() const noexcept`。
- 用 `std::string` 成员保存错误信息，`what()` 返回其 `c_str()`（注意生命期）。
- 可进一步派生细分异常类，形成自己的异常层次，便于上层分类捕获。
- 构造函数通常用 `const std::string&` 或 `std::string` 传参，拼接出完整信息。

示例：

```
#include <iostream>
#include <exception>
#include <string>

// 自定义异常：继承 std::exception 并重写 what()
class ConfigError : public std::exception {
    std::string msg_;
public:
    explicit ConfigError(const std::string& key)
        : msg_("缺少配置项: " + key) {}

    // 重写 what(), 返回可读错误信息
    const char* what() const noexcept override {
        return msg_.c_str();
    }
};

int main() {
    try {
        throw ConfigError("database.host");
    } catch (const std::exception& e) { // 通过基类捕获自定义异常
        std::cout << e.what() << "\n";
    }
    return 0;
}
```

易错点/注意：

- `what()` 返回的指针必须指向有效内存；若返回临时 `std::string` 的 `c_str()` 会悬垂，应存为成员。
- `what()` 必须声明为 `noexcept`（C++11 起基类即为 `noexcept`），否则签名不匹配无法正确重写。

异常与其他错误处理

异常 vs 错误码 (std::expected (C++23) 简介)

概念：错误码通过返回值表达失败，异常通过抛出表达失败；二者各有适用场景。C++23 引入 `std::expected<T, E>`，用返回类型同时携带"值或错误"，把错误变成可检查的返回值，兼具错误码的显式性和类型安全。

要点：

- 异常适合：错误罕见、调用栈深、错误需跨多层传播的场景；代价是 `throw` 开销较大。
- 错误码适合：错误常见、需显式逐个检查、性能敏感的热路径；代价是返回值与错误码混杂、易漏检。
- `std::expected<T, E>` (C++23)：返回 `T` 值或 `E` 错误，用 `has_value()` / `value()` / `error()` 访问，用 `std::unexpected` 构造错误。
- 选择建议：可预期、高频的错误用错误码/`expected`；真正"意外"、罕见的错误用异常。

示例：

```
#include <iostream>
#include <string>
#include <expected>      // C++23: 需 -std=c++23
#include <system_error>

// 用 std::expected 表达"值或错误"，无需抛出异常
std::expected<int, std::string> parseAge(const std::string& s) {
    try {
        int v = std::stoi(s);           // 可能抛异常
        if (v < 0) return std::unexpected("年龄不能为负");
        return v;                       // 返回"值"
    } catch (...) {
        return std::unexpected("不是合法整数"); // 返回"错误"
    }
}

int main() {
    auto r = parseAge("abc");
    if (r) {                             // 判断是否持有值
        std::cout << "年龄: " << *r << "\n";
    } else {
        std::cout << "错误: " << r.error() << "\n"; // 读取错误值
    }
    return 0;
}
```

易错点/注意：

- `std::expected` 是 C++23 特性，编译时需开启 C++23 标准；C++17/20 环境不可用。
- 传统错误码方案中，漏检错误码是常见 bug；`std::expected` 虽仍需手动检查，但类型系统能显著减少漏检。

性能与注意事项

性能与注意事项

概念：现代主流编译器采用“零开销异常”模型（表驱动），即“不抛异常时 try/catch 本身几乎无开销”，但真正抛出并展开时成本很高；因此异常应只用于真正的错误，不应作为正常控制流。

要点：

- 零开销模型：未抛出异常时，进入/离开 `try` 块几乎不产生额外指令开销。
- `throw` 与栈展开开销大：涉及查找处理表、析构沿途对象、可能的内存分配。
- 不要用异常做正常控制流（如用 `throw` 代替 `return`），这会严重拖慢性能并降低可读性。
- 将可能抛异常的代码集中、将 `noexcept` 用于不抛函数，有助于编译器优化与容器性能。

示例：

```
#include <iostream>
#include <vector>

// 反例：用异常做正常控制流（极低效、可读性差）
int indexOfBad(const std::vector<int>& v, int target) {
    try {
        for (size_t i = 0; i < v.size(); ++i)
            if (v[i] == target) throw i; // 用 throw 模拟 return, 极低效
    } catch (size_t i) {
        return static_cast<int>(i);
    }
    return -1;
}

// 正例：正常分支走返回值，异常只留给真正的错误
int indexOfGood(const std::vector<int>& v, int target) {
    for (size_t i = 0; i < v.size(); ++i)
        if (v[i] == target) return static_cast<int>(i); // 普通 return 即可
    return -1;
}

int main() {
    std::vector<int> v = {10, 20, 30};
    std::cout << indexOfBad(v, 20) << "\n"; // 1, 但内部走的是异常路径
    std::cout << indexOfGood(v, 20) << "\n"; // 1, 正常返回
    return 0;
}
```

易错点/注意：

- 不要把“找不到”这类正常结果用异常表达；应返回哨兵值或 `std::optional`。
- 在热循环、实时系统、嵌入式环境中应尽量避免依赖异常，可优先错误码或 `expected`。

本部分小结

主题	易忘点
抛出与捕获	<code>throw</code> 抛对象, <code>catch(const T&)</code> 按顺序匹配, <code>catch(...)</code> 兜底且必须放最后
栈展开	异常传播途中已构造局部对象按构造逆序自动析构, 是 RAII 安全的基础
重新抛出	<code>throw;</code> 保留原始类型; <code>throw e;</code> 抛拷贝且可能切片, 优先用 <code>throw;</code>
构造函数异常	失败抛异常; 已构造成员自动析构, 析构函数不执行; 用智能指针防泄漏
析构函数异常	默认 <code>noexcept(true)</code> , 抛异常即 <code>std::terminate</code> , 应吞掉或绝不抛出
异常安全保证	基本保证 (有效) /强保证 (不变) /不抛出 (<code>noexcept</code>), <code>copy-and-swap</code> 实现强保证
<code>noexcept</code>	说明符承诺不抛、运算符编译期判断; 移动构造 <code>noexcept</code> 才被 <code>vector</code> 优先使用
标准异常	根为 <code>std::exception</code> , 分 <code>logic_error</code> / <code>runtime_error</code> 两族, 另有 <code>bad_alloc</code> / <code>bad_cast</code>
自定义异常	继承 <code>std::exception</code> 重写 <code>what()</code> , 信息存成员, 返回 <code>c_str()</code>
异常 vs 错误码	罕见错误用异常、高频可预期错误用错误码/ <code>std::expected</code> (C++23)
性能	零开销模型: 不抛时几乎无成本; 真抛时代价高, 勿用异常做控制流

清单速记:

- 捕获顺序: 派生类 → 基类 → `catch(...)`。
- 构造函数失败: 抛异常; 成员清理靠 RAII, 不靠析构函数。
- 析构函数: 默认 `noexcept`, 绝不抛异常。
- 移动构造/赋值、`swap`: 尽量标 `noexcept`。
- 写库函数: 至少满足基本保证, 尽量做到强保证。

第八部分：标准库进阶

算法库基础

算法库总览

概念: 标准算法库由 `<algorithm>` (通用算法) 与 `<numeric>` (数值算法) 组成, 核心思想是"通过迭代器区间 `[first, last)` 操作元素", 因此算法与容器解耦, 可作用于数组、`vector`、`list`、`string` 等任何提供迭代器的序列。

要点:

- `<algorithm>`: 查找、排序、变换、去重、集合运算等通用算法; `<numeric>`: `accumulate`、`iota`、`gcd` 等数值算法。
- 区间约定为左闭右开 `[first, last)`, `last` 是"尾后迭代器", 不指向元素。

示例:

```

#include <algorithm>
#include <numeric>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {5, 1, 4, 2, 3};
    std::sort(v.begin(), v.end()); // 排序区间 [begin, end)
    int sum = std::accumulate(v.begin(), v.end(), 0); // 累加求和
    for (int x : v) std::cout << x << ' ';
    std::cout << "\n求和: " << sum << "\n"; // 1 2 3 4 5 / 15
}

```

易错点/注意:

- 区间始终左闭右开; 越界或跨容器的区间是未定义行为。

排序与查找

概念: 排序算法有 `sort` (不稳定快排)、`stable_sort` (稳定)、`partial_sort` (部分排序)、`nth_element` (第 n 小就位); 查找算法在**有序**区间上用
`lower_bound` / `upper_bound` / `binary_search` / `equal_range` 做二分查找, 均可传入自定义比较器。

要点:

- `sort` 平均 $O(n \log n)$ 且不稳定; `stable_sort` 稳定但更慢; `partial_sort(begin, mid, end)` 只把最小的若干元素排到 `[begin, mid)`。
- `nth_element(begin, nth, end)`: 使 `*nth` 等于完全排序后该位置的值, 两侧无须有序。
- 二分查找要求区间已有序: `lower_bound` 找第一个 $\geq$ 值、`upper_bound` 找第一个 $>$ 值、`equal_range` 返回 `[lower, upper)`、`binary_search` 只判存在。

示例:

```

#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {5, 3, 8, 1, 3, 9, 2};
    std::sort(v.begin(), v.end()); // 升序: 1 2 3 3 5 8 9
    std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; }); // 降序

    std::vector<int> w = {9, 1, 8, 2, 7, 3};
    std::partial_sort(w.begin(), w.begin() + 3, w.end()); // 前 3 个为最小 3 个

    std::vector<int> n = {9, 3, 7, 1, 5};
    std::nth_element(n.begin(), n.begin() + 2, n.end()); // n[2] 为第 3 小

    std::sort(v.begin(), v.end()); // 二分查找前须有序
    bool found = std::binary_search(v.begin(), v.end(), 3);
    auto lo = std::lower_bound(v.begin(), v.end(), 3); // 第一个 >= 3
    auto hi = std::upper_bound(v.begin(), v.end(), 3); // 第一个 > 3
    std::cout << found << ' ' << (lo - v.begin()) << ' '
              << (hi - v.begin()) << "\n";
}

```

易错点/注意:

- 二分查找要求区间有序, 否则是未定义行为; `sort` 需随机访问迭代器, 不能直接用于 `std::list`。

非修改算法

概念: 非修改算法遍历但不改变区间 (`for_each` 可通过引用修改), 用于查询与判定:

`find`/`find_if`、`count`/`count_if`、`all_of`/`any_of`/`none_of`、`for_each`、`max_element`/`min_element`。

要点:

- `find`/`find_if` 返回第一个匹配元素的迭代器, 找不到返回 `last`。
- `count`/`count_if` 返回满足条件的元素个数; `all_of`/`any_of`/`none_of` 做全称/存在/否定判断。

示例:

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6};
    auto it = std::find(v.begin(), v.end(), 3); // 找值 3
    auto it2 = std::find_if(v.begin(), v.end(),
        [](int x) { return x % 2 == 0; }); // 首个偶数
    long n = std::count_if(v.begin(), v.end(),
        [](int x) { return x > 3; }); // >3 的个数
    bool all = std::all_of(v.begin(), v.end(), [](int x) { return x > 0; });
    bool any = std::any_of(v.begin(), v.end(), [](int x) { return x > 5; });
    std::for_each(v.begin(), v.end(), [](int& x) { x *= 2; }); // 原地翻倍
    auto mx = std::max_element(v.begin(), v.end());
    std::cout << (it != v.end()) << ' ' << *it2 << ' ' << n << ' '
        << all << ' ' << any << ' ' << *mx << "\n";
}
```

易错点/注意:

- `find`/`max_element` 找不到时返回 `end()`, 解引用前务必先判断 `!= end()`。

修改算法

概念: 修改算法改写或生成元素: `transform` (逐元素变换)、`copy`/`copy_if` (拷贝)、`reverse`/`fill`/`generate` (反转/填充/生成)、`unique` (相邻去重); 其中 `unique`/`remove` 需配合 `erase` 形成 erase-remove 惯用法才真正删除元素。

要点:

- `transform` 把源区间经函数变换写入目标区间, 输出区间须够大或用 `back_inserter`。
- `copy_if` 按条件拷贝, 常配合 `std::back_inserter` 追加。
- erase-remove 惯用法: `v.erase(std::remove(v.begin(), v.end(), x), v.end());`。

示例:

```
#include <algorithm>
```

```

#include <vector>
#include <iostream>
#include <iterator>

int main() {
    std::vector<int> src = {1, 2, 3, 4, 5};
    std::vector<int> dst(src.size());
    std::transform(src.begin(), src.end(), dst.begin(),
        [](int x) { return x * x; }); // 1 4 9 16 25

    std::vector<int> even;
    std::copy_if(src.begin(), src.end(), std::back_inserter(even),
        [](int x) { return x % 2 == 0; }); // 2 4

    std::reverse(dst.begin(), dst.end());
    std::fill(dst.begin(), dst.end(), 0);
    std::generate(dst.begin(), dst.end(), [n = 0]() mutable { return n++; });

    std::vector<int> v = {1, 1, 2, 2, 2, 3, 4, 4};
    v.erase(std::unique(v.begin(), v.end()), v.end()); // 1 2 3 4
    std::vector<int> w = {1, 2, 3, 2, 4, 2};
    w.erase(std::remove(w.begin(), w.end(), 2), w.end()); // 1 3 4
    for (int x : v) std::cout << x << ' ';
}

```

易错点/注意:

- `unique` / `remove` 只移动元素不释放容量，必须接 `erase` 才真正删除。

集合与排列

概念: 集合算法 `set_union` / `set_intersection` / `set_difference` 在两个**有序**区间上求并/交/差；排列算法 `next_permutation` / `prev_permutation` 按字典序重排，用于枚举全排列。

要点:

- 集合算法要求两个输入区间有序，结果经输出迭代器（如 `back_inserter`）写出。
- `next_permutation` 改为字典序下一个排列，返回 `false` 表示已是最大排列并回卷到最小。
- `prev_permutation` 对称求上一个排列。

示例:

```

#include <algorithm>
#include <vector>
#include <iostream>
#include <iterator>

int main() {
    std::vector<int> a = {1, 2, 3, 4}, b = {3, 4, 5, 6}, out;
    std::set_union(a.begin(), a.end(), b.begin(), b.end(),
        std::back_inserter(out)); // 并集 1 2 3 4 5 6

    out.clear();
    std::set_intersection(a.begin(), a.end(), b.begin(), b.end(),
        std::back_inserter(out)); // 交集 3 4

    out.clear();
    std::set_difference(a.begin(), a.end(), b.begin(), b.end(),
        std::back_inserter(out)); // 差集 1 2
}

```

```

std::vector<int> p = {1, 2, 3};
do { for (int x : p) std::cout << x << ' '; std::cout << "| "; }
while (std::next_permutation(p.begin(), p.end())); // 枚举全排列
std::cout << "\n";
}

```

易错点/注意:

- 集合算法输入必须有序; 枚举全排列前应先把区间排序到最小排列。

函数对象

函数对象与适配器

概念: 函数对象 (仿函数) 是重载了 `operator()` 的类; 标准库提供 `std::greater` / `std::less` 预定义函数对象, 以及 `std::mem_fn`、`std::not_fn` 适配器把成员函数、谓词改造成算法可用的可调用对象; lambda 本质是编译器生成的匿名仿函数。

要点:

- `std::greater` / `std::less`: 标准比较函数对象, 可传给 `sort`、优先队列等。
- `std::mem_fn`: 把成员函数包装成普通可调用对象, 调用时对象作首参。

示例:

```

#include <algorithm>
#include <functional>
#include <vector>
#include <iostream>
#include <string>

struct Person { std::string name; int age; int getAge() const { return age; } };
struct AgeLess { bool operator()(const Person& a, const Person& b) const {
return a.age < b.age; } };

int main() {
    std::vector<int> v = {3, 1, 4, 1, 5};
    std::sort(v.begin(), v.end(), std::greater<int>()); // 标准函数对象降序

    std::vector<Person> ps = {{ "A", 30 }, { "B", 20 }, { "C", 25 }};
    std::sort(ps.begin(), ps.end(), AgeLess{}); // 仿函数
    std::sort(ps.begin(), ps.end(),
        [](const Person& a, const Person& b) { return a.age < b.age; });
    // lambda

    auto getAge = std::mem_fn(&Person::getAge); // 包装成员函数
    std::cout << getAge(ps[1]) << "\n";

    auto isOdd = [](int x) { return x % 2 == 1; };
    auto isEven = std::not_fn(isOdd); // C++17 取反
    std::cout << std::boolalpha << isEven(2) << "\n"; // true
}

```

易错点/注意:

- 比较器须严格弱序, 否则 `sort` 未定义; `std::not_fn` 是 C++17 (`std::not1` 已弃用)。

通用工具

<utility>: pair、tuple 与 swap/exchange

概念: <utility> 提供 `std::pair`、`std::swap`、`std::exchange`; <tuple> 提供 `std::tuple` 及 `make_tuple`、`tie`、`tuple_cat`, 配合 C++17 结构化绑定可简洁解包。

要点:

- `pair`: 二元组, 成员 `first/second`; `tuple` 任意元数, 用 `std::get<N>` 访问。
- `tie` 解包到已有变量 (`std::ignore` 忽略某项); `tuple_cat` 拼接 tuple。
- 结构化绑定 (C++17): `auto [x, y, z] = t;` 一行解包。
- `std::swap` 交换 (常 `noexcept`); `std::exchange(obj, new_val)` (C++14) 返回旧值并替换。

示例:

```
#include <utility>
#include <tuple>
#include <string>
#include <iostream>

int main() {
    std::pair<std::string, int> p = std::make_pair("age", 18); // pair
    std::cout << p.first << "=" << p.second << "\n";

    std::tuple<int, std::string, double> t = std::make_tuple(1, "hi", 3.14);
    std::cout << std::get<0>(t) << ' ' << std::get<1>(t) << "\n"; // 访问元素

    int a; std::string b; double c;
    std::tie(a, b, c) = t; // 解包到已有变量
    auto [x, y, z] = t; // 结构化绑定 (C++17)
    auto t2 = std::tuple_cat(t, std::make_tuple('A')); // 拼接 tuple

    int m = 1, n = 2;
    std::swap(m, n); // 交换
    int old = std::exchange(m, 100); // C++14: 读旧值并替换
    std::cout << x << ' ' << old << ' ' << m << "\n";
}
```

易错点/注意:

- `std::get<N>` 的 `N` 须是编译期常量; 结构化绑定变量数量须与元素数一致。

std::optional (C++17)

概念: `std::optional<T>` 表示"可能有值的 T", 替代哨兵值/输出参数的老写法, 语义清晰、类型安全, 典型场景是可能失败的查找、解析等返回值。

要点:

- `has_value()` / `operator bool` 判断是否有值; `value()` 取值 (无值时抛 `bad_optional_access`)。
- `value_or(default)` 有值返回值、无值给默认值; `std::nullopt` 表示无值。
- 适用场景: 返回可能缺失的单个值 (map 查找、字符串转数字)。

示例:

```

#include <optional>
#include <iostream>
#include <string>

std::optional<int> parseInt(const std::string& s) {
    try { return std::stoi(s); }
    catch (...) { return std::nullopt; } // 表示"无值"
}

int main() {
    auto a = parseInt("42"), b = parseInt("abc");
    if (a.has_value()) std::cout << "a=" << a.value() << "\n"; // 42
    std::cout << "b=" << b.value_or(-1) << "\n"; // -1 (无值给默认值)
    if (a) std::cout << "*a=" << *a << "\n";
}

```

易错点/注意:

- 无值调 `value()` 抛异常、解引用 `*opt` 是未定义行为；优先 `value_or` 或先判断。

std::variant (C++17)

概念: `std::variant<T...>` 是类型安全的联合体，任一时刻持有列表中的一种类型，大小约为最大类型加开销；用 `std::visit`/`std::get` 访问，适合"几种类型之一"的运行时多态，无需虚函数与堆分配。

要点:

- `std::get<T>/get<N>` 取值，类型不符抛 `bad_variant_access`。
- `std::get_if<T>(&v)` 返回指针，类型不符返回 `nullptr`，不抛异常。
- `std::visit(visitor, v)` 对当前值分派（泛型 lambda + `if constexpr`）。

示例:

```

#include <variant>
#include <iostream>
#include <string>
#include <type_traits>

int main() {
    std::variant<int, double, std::string> v;
    v = 42;
    std::cout << std::get<int>(v) << "\n"; // 42
    v = "hello";
    std::cout << std::get<std::string>(v) << "\n";

    if (auto p = std::get_if<int>(&v)) std::cout << "int: " << *p << "\n";
    else std::cout << "不是 int\n";

    std::visit([](auto&& val) { // 对当前持有的值分派处理
        using T = std::decay_t<decltype(val)>;
        if constexpr (std::is_same_v<T, int>) std::cout << "int " << val <<
"\n";
        else if constexpr (std::is_same_v<T, double>) std::cout << "double " <<
val << "\n";
        else std::cout << "string " << val << "\n";
    }, v);
}

```

```
}
```

易错点/注意:

- `get<T>` 类型不符抛异常, 热路径用 `get_if` + 判空; visitor 必须覆盖所有类型分支。

std::any (C++17)

概念: `std::any` 可持有任意可拷贝类型, 是类型擦除容器: 具体类型被隐藏, 用 `any_cast` 在运行时还原, 适合"类型完全未知"的通用配置、脚本桥接等场景。

要点:

- 赋值任意类型: `a = 42; a = 3.14; a = std::string("x");`。
- `any_cast<T>(a)` 转型, 类型不符抛 `bad_any_cast`; 指针版 `any_cast<T>(&a)` 返回 `nullptr`。
- 类型擦除思想: 把具体类型操作封装进虚函数/函数指针, 对外暴露统一接口。

示例:

```
#include <any>
#include <iostream>
#include <string>

int main() {
    std::any a;
    a = 42;
    a = std::string("hi");

    try { std::cout << std::any_cast<std::string>(a) << "\n"; }
    catch (const std::bad_any_cast& e) { std::cout << "转换失败\n"; }

    if (auto p = std::any_cast<int>(&a)) std::cout << "int: " << *p << "\n";
    else std::cout << "不是 int\n";

    std::cout << a.has_value() << "\n";
    a.reset(); // 清空
}
```

易错点/注意:

- `any_cast` 须用准确的原始类型; 能用 `variant` / 模板表达时优先, `any` 仅用于类型完全开放时。

文本与正则

std::regex

概念: `<regex>` 提供正则支持: `regex_match` (全串匹配)、`regex_search` (子串搜索)、`regex_replace` (替换), 配合 `std::regex` 模式与 `std::smatch` 结果, 用于验证、提取、替换文本。

要点:

- `regex_match` 要求整个字符串完全匹配; `regex_search` 查找任意子串; `regex_replace` 替换匹配子串。
- 模式常用原始字符串 `R"(...)"` 避免转义反斜杠。

示例：

```
#include <regex>
#include <iostream>
#include <string>

int main() {
    std::string text = "电话 138-1234-5678, 邮箱 abc@example.com";
    std::regex phone(R"(\d{3}-\d{4}-\d{4})");
    std::smatch m;
    if (std::regex_search(text, m, phone)) std::cout << m.str() << "\n"; // 提取电话

    std::string masked = std::regex_replace(text, phone, "***-***-***"); // 脱敏

    std::regex email(R"([\w.]+@[w.]+)");
    std::cout << std::boolalpha
              << std::regex_match("abc@example.com", email) << "\n"; // 全串匹配
}
```

常用元字符表：

元字符	含义
.	除换行外任意单个字符
\d \w \s	数字 / 字母数字下划线 / 空白
\D \W \S	上述反义
^ \$	行首 / 行尾
* + ?	0 次以上 / 1 次以上 / 0 或 1 次
{n} {n,} {n,m}	恰好 n / 至少 n / n 到 m 次
[...] [^...]	字符类 / 取反
(...)	捕获组
	或

易错点/注意：

- 验证整体格式用 `regex_match`，提取片段用 `regex_search`；同一模式多次使用应复用 `std::regex` 对象。

时间与随机

<chrono>：时间与计时

概念：<chrono> 提供类型安全时间库：`duration`（时长）、`time_point`（时间点）、`steady_clock` 等时钟；C++20 扩展了日历日期类型。

要点：

- `duration` 带单位，`count()` 取数值；`duration_cast<单位>` 显式换算（截断）。

- `steady_clock` 单调递增，适合计时；`clock::now()` 得时间点，相减得 `duration`。
- C++20 日历：`year_month_day{year{2024}, month{1}, day{15}}` 表达日期，可做日期运算。

示例：

```
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono;
    milliseconds ms(1500);
    seconds s = duration_cast<seconds>(ms); // 截断为 1 秒
    std::cout << ms.count() << ' ' << s.count() << "\n";

    auto start = steady_clock::now(); // 计时起点
    for (int i = 0; i < 1000000; ++i) { volatile int x = i * i; }
    auto end = steady_clock::now();
    auto el = duration_cast<microseconds>(end - start); // 时间点相减得时长
    std::cout << "耗时 " << el.count() << " 微秒\n";
}
```

易错点/注意：

- 计时用 `steady_clock`；`system_clock` 可能因校时回拨。`duration_cast` 是截断不是四舍五入。

<random>：随机数

概念： <random> 分层提供"引擎 + 分布"：引擎如 `mt19937`（梅森旋转），分布如 `uniform_int_distribution`；`random_device` 提供真随机种子。相比 `rand()` 质量更高、范围均匀、可复现。

要点：

- `random_device` 非确定性，作种子；`mt19937` 是通用首选引擎。
- `uniform_int_distribution(a, b)` 在 `[a, b]` 均匀取整。
- 与 `rand()` 对比：`rand() % n` 有模偏差、质量差、全局状态不安全。

示例：

```
#include <random>
#include <iostream>

int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(1, 6); // 骰子

    for (int i = 0; i < 10; ++i) std::cout << dist(gen) << ' ';
    std::cout << "\n";

    std::uniform_real_distribution<double> udist(0.0, 1.0);
    std::cout << udist(gen) << "\n";

    std::mt19937 fixed(42); // 固定种子可复现
    std::cout << dist(fixed) << "\n";
    // int r = rand() % 6 + 1; // 旧式写法，不推荐
}
```

```
}
```

易错点/注意:

- 引擎与分布要复用，别每次取数重建引擎；固定种子用于调试复现。

输入输出

文件流 (ifstream/ofstream/fstream)

概念: `ifstream` / `ofstream` / `fstream` 封装文件 I/O，支持文本与二进制模式；`getline` 逐行读文本，`read/write` 读写原始字节，用状态位判断成败。

要点:

- `ofstream` 默认截断写；`ifstream` 只读；二进制模式加 `std::ios::binary`。
- `getline(in, line)` 逐行读（不含换行）；`write/read` 传字节指针与长度。
- 状态位: `good()` / `fail()` / `bad()` / `eof()`；流可隐式转 `bool` 判断是否可用。

示例:

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    { // 文本写
        std::ofstream out("data.txt");
        out << "第一行\n" << 42 << "\n";
    }
    { // 文本读: getline 逐行
        std::ifstream in("data.txt");
        std::string line;
        while (std::getline(in, line)) std::cout << line << "\n";
    }
    { // 二进制读写
        std::ofstream out("data.bin", std::ios::binary);
        int arr[3] = {1, 2, 3};
        out.write(reinterpret_cast<char*>(arr), sizeof(arr));
        std::ifstream in("data.bin", std::ios::binary);
        int brr[3] = {};
        in.read(reinterpret_cast<char*>(brr), sizeof(brr));
        std::cout << brr[0] << ' ' << brr[1] << ' ' << brr[2] << "\n";
    }
    { // 错误状态位
        std::ifstream in("不存在.txt");
        std::cout << std::boolalpha << in.fail() << ' ' << in.eof() << "\n";
    }
}
```

易错点/注意:

- `getline` 返回流引用可作 `while` 条件；二进制 `read/write` 用 `reinterpret_cast<char*> + sizeof`。

字符串流 stringstream

概念: `<sstream>` 的 `stringstream`/`istringstream`/`ostringstream` 把字符串当流读写, 用于格式化拼接、从字符串解析、数值与字符串互转。

要点:

- `ostringstream` 用 `<<` 拼接, `str()` 取结果; `istringstream` 用 `>>` 解析。
- `stringstream` 双向, 可用于类型转换。

示例:

```
#include <sstream>
#include <iostream>
#include <string>

int main() {
    std::ostringstream oss;           // 格式化拼接
    oss << "价格: " << 12.5 << " 元";
    std::cout << oss.str() << "\n";

    std::istringstream iss("10 20 30"); // 从字符串解析
    int a, b, c;
    iss >> a >> b >> c;
    std::cout << a + b + c << "\n"; // 60

    std::stringstream ss;
    ss << "3.14";                     // 字符串 -> 数字
    double pi; ss >> pi;
    std::cout << pi * 2 << "\n";
}
```

易错点/注意:

- 复用同一流需 `str("")` 清内容 + `clear()` 清状态位; 解析失败会置 `fail` 位。

<filesystem> (C++17)

概念: `<filesystem>` (C++17) 提供跨平台文件系统操作, 核心 `std::filesystem::path` 表示路径, 配合 `exists`、`is_directory`、`create_directories`、`directory_iterator` 完成判断、遍历、增删。

要点:

- `path` 支持 `filename()`/`extension()`/`parent_path()`/`root_path()` 分解。
- `exists` 判断存在, `is_directory`/`is_regular_file` 判断类型。
- `create_directories` 递归建目录, `remove` 删除; `directory_iterator` 遍历 (不递归)。

示例:

```
#include <filesystem>
#include <iostream>
namespace fs = std::filesystem;

int main() {
    fs::path p = "C:/temp/test.txt";
    std::cout << p.filename() << ' ' << p.extension() << ' ' << '\n';
}
```

```

        << p.parent_path() << "\n"; // test.txt .txt C:/temp

fs::path dir = "C:/temp";
if (fs::exists(dir) && fs::is_directory(dir)) {
    for (const auto& e : fs::directory_iterator(dir)) // 遍历目录
        std::cout << " " << e.path().filename() << "\n";
}

fs::path sub = "C:/temp/sub/dir";
fs::create_directories(sub); // 递归创建
fs::remove("C:/temp/sub/dir");
fs::remove("C:/temp/sub");
}

```

易错点/注意：

- 操作可抛 `filesystem_error`，可用 `std::error_code` 重载避免异常；`directory_iterator` 不递归。

现代特性

C++20 ranges

概念： C++20 `<ranges>` 让算法从"迭代器对"升级为"作用于整个 range 的管道组合"；`views::filter` / `transform` / `take` 等视图惰性求值，用 `|` 管道写出声明式流水线。

要点：

- range 概念：有 `begin()` / `end()` 的都是 range（容器、数组、视图）。
- 视图惰性求值，遍历时才逐个计算，不产生中间容器。

示例：

```

#include <ranges>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // 传统写法需中间容器与 back_inserter; ranges 管道惰性求值、无中间容器
    auto result = v
        | std::views::filter([](int x) { return x % 2 == 0; }) // 过滤偶数
        | std::views::transform([](int x) { return x * x; })   // 平方
        | std::views::take(3);                                 // 取前 3 个
    for (int x : result) std::cout << x << ' '; // 4 16 36
    std::cout << "\n";
}

```

易错点/注意：

- 视图惰性，源容器被修改或销毁后遍历是未定义行为；需 C++20+ 支持 ranges 的编译器。

<numeric>：数值算法

概念： <numeric> 提供数值算法： `accumulate`（累加/折叠）、 `iota`（递增序列）、 `partial_sum`（前缀和）、以及 C++17 的 `gcd/lcm`（最大公约数/最小公倍数）。

要点：

- `accumulate(first, last, init)` 累加，带第 4 参可自定义运算（如累乘）。
- `iota(first, last, v)` 从 `v` 递增填充； `partial_sum` 输出前缀和。
- `gcd/lcm`（C++17）求最大公约数/最小公倍数。

示例：

```
#include <numeric>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    int sum = std::accumulate(v.begin(), v.end(), 0); // 15
    int prod = std::accumulate(v.begin(), v.end(), 1,
                               [](int a, int b) { return a * b; }); // 120

    std::vector<int> seq(5);
    std::iota(seq.begin(), seq.end(), 10); // 10 11 12 13 14

    std::vector<int> ps(v.size());
    std::partial_sum(v.begin(), v.end(), ps.begin()); // 1 3 6 10 15

    std::cout << sum << ' ' << prod << ' '
               << std::gcd(12, 18) << ' ' << std::lcm(4, 6) << "\n"; // 15 120 6
    12
}
```

易错点/注意：

- `accumulate` 初值决定结果类型（累加 `double` 用 `0.0`）； `gcd/lcm` 是 C++17 特性。

本部分小结

主题	易忘点
算法库	算法作用于迭代器区间 <code>[first, last)</code> ，与容器解耦
排序查找	<code>sort</code> 不稳定、 <code>stable_sort</code> 稳定；二分查找要求有序；比较器须严格弱序
非修改算法	<code>find/max_element</code> 找不到返回 <code>last</code> ，使用前先判 <code>!= end()</code>
修改算法	<code>unique/remove</code> 必须接 <code>erase</code> 才真正删除（erase-remove 惯用法）
集合排列	集合算法要求有序输入； <code>next_permutation</code> 前先排序到最小排列
函数对象	<code>greater/less</code> 、 <code>mem_fn</code> 、 <code>not_fn</code> (C++17)、 <code>lambda</code> 本质是匿名仿函数
utility	<code>pair/tuple</code> 、 <code>tie</code> 、结构化绑定(C++17)、 <code>swap</code> 、 <code>exchange</code> (C++14)
optional	<code>has_value/value_or</code> ；无值时 <code>value()</code> 抛异常、 <code>*opt</code> 未定义
variant	<code>get/get_if/visit/holds_alternative</code> ；visitor 须覆盖所有类型
any	<code>any_cast</code> 类型不符抛 <code>bad_any_cast</code> ；指针版返回 <code>nullptr</code>
regex	<code>match</code> 全串、 <code>search</code> 子串、 <code>replace</code> 替换；模式用 <code>R"(...)"</code>
chrono	<code>steady_clock</code> 计时； <code>duration_cast</code> 截断换算；C++20 有日历日期
random	<code>random_device</code> 种子 + <code>mt19937</code> 引擎 + <code>uniform_int_distribution</code> 分布
文件流	<code>getline</code> 逐行；二进制 <code>read/write</code> + <code>reinterpret_cast<char*></code>
stringstream	<code>ostringstream</code> 拼接、 <code>istringstream</code> 解析；复用需 <code>str("")+clear()</code>
filesystem	<code>path/exists/directory_iterator/create_directories</code> (C++17)
ranges	<code>views::filter/transform/take</code> 惰性求值， <code> </code> 管道组合 (C++20)
numeric	<code>accumulate/iota/partial_sum/gcd/lcm</code> (C++17)；初值决定累加类型

清单速记：

- 区间一律左闭右开，二分查找必须先排序。
- 现代 C++ 优先 `optional/variant/any` 表达"可能缺失/多态"，替代裸指针与哨兵值。

第九部分：并发与多线程

1. 并发基本概念

并发基本概念

概念：并发是指多个执行流在一段时间内交替或同时推进。进程是资源分配的基本单位，拥有独立地址空间；线程是 CPU 调度的基本单位，同一进程内的线程共享内存但各自拥有独立栈。多线程并发访问共享数据时，若不加同步，会产生数据竞争（data race）、原子性破坏与可见性问题。

要点：

- 进程：拥有独立内存空间，进程间通信需要 IPC（管道、共享内存、消息队列等）。
- 线程：同一进程内共享堆与全局数据，切换开销比进程小。

- 数据竞争：两个以上线程同时访问同一内存位置，且至少一个是写操作，且没有同步约束，结果是未定义行为。
- 原子性：一个操作要么完全执行、要么完全不执行，中途不被其他线程观察到中间状态。
- 可见性：一个线程对共享变量的修改，能否及时被其他线程看到（受缓存与编译器重排影响）。

示例：

```
#include <thread>    // 线程库
#include <iostream>
#include <vector>

// 共享计数器，多个线程同时累加
int counter = 0;

void add(int n) {
    for (int i = 0; i < n; ++i) {
        ++counter;    // 非原子操作，存在数据竞争
    }
}

int main() {
    std::vector<std::thread> threads;
    const int N = 100000;    // 每个线程累加次数
    for (int i = 0; i < 8; ++i) {    // 启动 8 个线程
        threads.emplace_back(add, N);
    }
    for (auto& t : threads) {
        t.join();    // 等待所有线程结束
    }
    // 理想结果是 800000，实际可能小于该值（丢失更新）
    std::cout << "counter = " << counter << '\n';
    return 0;
}
```

易错点/注意：

- `++counter` 在底层是「读-改-写」三步，多线程下会丢失更新，必须加锁或用 `std::atomic`。
- 不要假设线程执行顺序，调度顺序由操作系统决定，程序必须对任意交错都正确。

2. std::thread

std::thread（创建与传参、join/detach、析构注意事项）

概念：`std::thread` 是 C++11 引入的线程句柄，用于创建和管理一个执行线程。构造时传入可调用对象（函数、函数对象、lambda）及其参数即可启动线程；必须通过 `join()` 等待线程结束或 `detach()` 使其独立运行，否则线程对象析构时若仍处于可汇合（joinable）状态，会调用 `std::terminate`。

要点：

- 创建：`std::thread t(func, args...)`，参数以值形式拷贝到线程中，传递引用参数需用 `std::ref`。
- `join()`：阻塞当前线程，等待目标线程结束，之后线程对象变为不可汇合。
- `detach()`：让线程在后台独立运行，线程对象与执行流解耦，此后无法再 join。
- `joinable()`：判断线程对象是否可汇合，join 或 detach 前先判断可避免异常。

- 析构注意事项：一个 joinable 的 `std::thread` 对象析构会触发 `std::terminate`，必须先 join 或 detach。

示例：

```
#include <thread>
#include <iostream>
#include <functional>

// 以引用方式接收参数，必须配合 std::ref
void set_value(int& target, int value) {
    target = value;    // 直接修改调用方的变量
}

int main() {
    // 1. 普通传值：参数会被拷贝到新线程
    std::thread t1([](int x) {
        std::cout << "t1 收到 " << x << '\n';
    }, 42);

    // 2. 引用传参：必须用 std::ref 包裹，否则会编译错误或拷贝
    int result = 0;
    std::thread t2(set_value, std::ref(result), 100);
    t2.join();
    std::cout << "result = " << result << '\n';    // 输出 100

    // 3. detach：后台线程，主线程不等待
    std::thread t3([]() {
        std::cout << "detach 线程运行\n";
    });
    t3.detach();    // 分离后不再需要 join

    if (t1.joinable()) {
        t1.join();
    }
    return 0;
}
```

易错点/注意：

- 忘记 `join()` 或 `detach()` 会导致程序在退出时崩溃（`std::terminate`）。
- `detach` 后的线程不能引用已销毁的局部变量（悬空引用），否则是未定义行为。
- 传引用一定要 `std::ref`，直接写 `std::thread t(f, x)` 时 `x` 是被拷贝的。
- 线程函数若抛异常且未捕获，会导致 `std::terminate`。

成员函数作为线程入口的写法：

```
#include <thread>
#include <iostream>

struct Worker {
    void run(int times) {
        for (int i = 0; i < times; ++i) {
            std::cout << "工作 " << i << '\n';
        }
    }
};

};
```

```

int main() {
    worker w;
    // 调用成员函数: 传成员函数指针 + 对象地址 + 参数
    std::thread t(&worker::run, &w, 3);
    t.join();
    return 0;
}

```

3. 数据竞争示例与危害

数据竞争示例与危害

概念: 当两个或多个线程同时访问同一内存位置，其中至少一个是写操作，且这些访问之间不存在同步（锁、原子操作等）时，就发生数据竞争。数据竞争在 C++ 中属于未定义行为，可能导致结果错误、崩溃、死锁或难以复现的偶发 bug。

要点:

- 未定义行为: 编译器可能重排指令，导致行为与直觉不符。
- 典型危害: 丢失更新 (lost update)、读到中间状态、撕裂读 (torn read)。
- 难以复现: 数据竞争往往在特定时序才出现，调试困难。
- 解决手段: 加锁 (mutex)、原子类型 (atomic)、避免共享可变状态。

示例:

```

#include <thread>
#include <iostream>

long shared = 0; // 被多线程共享且无保护

void unsafe_increment() {
    for (int i = 0; i < 100000; ++i) {
        // 非原子读-改-写: 可能丢失更新
        shared = shared + 1;
    }
}

int main() {
    std::thread a(unsafe_increment);
    std::thread b(unsafe_increment);
    a.join();
    b.join();
    // 期望 200000, 但往往小于 200000 (丢失更新)
    std::cout << "shared = " << shared << '\n';
    return 0;
}

```

易错点/注意:

- 即使用 `volatile` 也不能解决数据竞争，`volatile` 只用于特殊内存（如硬件寄存器），不提供原子性与同步。
- 不要靠「看起来是单条语句」判断安全，`x += 1` 同样不是原子的。
- 修复此例应使用 `std::atomic<long>` 或互斥锁保护。

4. mutex (互斥锁)

mutex (lock/unlock、lock_guard、unique_lock、scoped_lock、recursive_mutex)

概念：互斥量（mutex）用于保证同一时刻只有一个线程能进入临界区。手动 `lock()` / `unlock()` 容易因异常或提前返回而忘记解锁，因此推荐使用 RAII 封装：`std::lock_guard`（轻量）、`std::unique_lock`（可延迟/手动加解锁、可转移）、`std::scoped_lock`（C++17，可同时锁多个互斥量）。`std::recursive_mutex` 允许同一线程重复加锁。

要点：

- `mutex`：普通互斥锁，不可重复加锁（同一线程二次 lock 会死锁）。
- `std::lock_guard`：构造即加锁、析构即解锁，不可手动解锁，开销最小。
- `std::unique_lock`：可延迟加锁（`std::defer_lock`）、可提前解锁、可转移所有权，配合条件变量使用。
- `std::scoped_lock`（C++17）：可变参数模板，可一次锁住多个互斥量且避免死锁，是 `lock_guard` 的增强版。
- `recursive_mutex`：同一线程可多次加锁，需配对多次解锁。

示例：

```
#include <mutex>
#include <thread>
#include <iostream>

std::mutex mtx;
int counter = 0;

void safe_add() {
    for (int i = 0; i < 100000; ++i) {
        std::lock_guard<std::mutex> lock(mtx); // RAII: 自动加锁与解锁
        ++counter;
    }
}

void manual_demo() {
    std::unique_lock<std::mutex> ul(mtx, std::defer_lock); // 延迟加锁
    // ... 可以做一些无需锁的准备 ...
    ul.lock(); // 手动加锁
    ++counter;
    ul.unlock(); // 提前解锁
}

int main() {
    std::thread a(safe_add);
    std::thread b(safe_add);
    a.join();
    b.join();
    std::cout << "counter = " << counter << '\n'; // 稳定输出 200000
    return 0;
}
```

易错点/注意：

- `lock_guard` 无法配合 `condition_variable::wait`（wait 需要临时解锁），应使用 `unique_lock`。

- 临界区尽量短，避免在持锁期间做耗时操作或调用可能再次加锁的函数。
- 不要在持锁时跨线程等待，容易造成死锁。

5. 死锁与避免

死锁与避免

概念：死锁指两个或多个线程互相持有对方需要的锁并等待对方释放，导致所有相关线程永久阻塞。产生死锁的四个必要条件是：互斥、持有并等待、不可剥夺、循环等待；破坏其中任意一个即可避免死锁。常用手段有 `std::lock` 同时加锁和按固定顺序加锁。

要点：

- 四个必要条件：互斥、持有并等待、不可剥夺、循环等待。
- `std::lock(a, b)`：同时锁定多个互斥量，内部避免死锁（若部分失败会回滚已锁定的锁）。
- 固定顺序加锁：所有线程按相同顺序获取锁，破坏循环等待。
- 尽量缩小锁的粒度与持有时间，降低冲突概率。

示例：

```
#include <mutex>
#include <thread>
#include <iostream>

std::mutex m1, m2;

// 错误示范：两个线程以相反顺序加锁，可能死锁
void bad_order() {
    std::lock_guard<std::mutex> l1(m1);
    std::lock_guard<std::mutex> l2(m2);
}

// 方案一：std::lock 同时加锁
void safe_with_std_lock() {
    std::lock(m1, m2); // 原子地同时锁定
    std::lock_guard<std::mutex> l1(m1, std::adopt_lock); // 接管已锁定的锁
    std::lock_guard<std::mutex> l2(m2, std::adopt_lock);
    // 临界区操作
}

// 方案二：C++17 std::scoped_lock 更简洁
void safe_with_scoped_lock() {
    std::scoped_lock lock(m1, m2); // 同时锁定且异常安全
    // 临界区操作
}

int main() {
    std::thread a(safe_with_std_lock);
    std::thread b(safe_with_scoped_lock);
    a.join();
    b.join();
    std::cout << "无死锁完成\n";
    return 0;
}
```

易错点/注意：

- `std::adopt_lock` 表示「锁已被锁定，lock_guard 只负责解锁」，不要重复加锁。
- 与 `std::lock` 配合时若再手动 `lock()` 会二次加锁导致死锁。
- 固定顺序加锁要求所有代码路径严格一致，容易在维护中破坏。

6. condition_variable (条件变量)

condition_variable (wait 谓词版本、notify、生产者-消费者、虚假唤醒)

概念：条件变量用于在线程间等待某个条件成立，常与 `std::unique_lock<std::mutex>` 配合。

`wait` 会释放锁并阻塞线程，直到被 `notify_one` 或 `notify_all` 唤醒后重新加锁继续执行。由于存在虚假唤醒 (spurious wakeup)，必须用带谓词的 `wait` 重载或循环检查条件。

要点：

- `wait(lock, pred)`：谓词为假时释放锁并等待，被唤醒后重新加锁并再次检查谓词，天然防止虚假唤醒。
- `notify_one()`：唤醒一个等待线程；`notify_all()`：唤醒所有等待线程。
- 虚假唤醒：线程可能在未被通知时被唤醒，因此条件必须用谓词反复检查。
- 经典应用：生产者-消费者模型（使用队列做缓冲区）。

示例：

```
#include <condition_variable>
#include <mutex>
#include <thread>
#include <queue>
#include <iostream>
#include <chrono>

std::mutex mtx;
std::condition_variable cv;
std::queue<int> buffer;           // 共享缓冲区
constexpr int kMaxSize = 5;

void producer(int id) {
    for (int i = 0; i < 10; ++i) {
        std::unique_lock<std::mutex> lock(mtx);
        // 缓冲区满则等待消费者消费
        cv.wait(lock, [] { return buffer.size() < kMaxSize; });
        buffer.push(i);
        std::cout << "生产者 " << id << " 生产 " << i << '\n';
        lock.unlock();
        cv.notify_all();         // 通知消费者
    }
}

void consumer(int id) {
    for (int i = 0; i < 10; ++i) {
        std::unique_lock<std::mutex> lock(mtx);
        // 缓冲区空则等待生产者生产
        cv.wait(lock, [] { return !buffer.empty(); });
        int value = buffer.front();
        buffer.pop();
        std::cout << "消费者 " << id << " 消费 " << value << '\n';
        lock.unlock();
    }
}
```

```

        cv.notify_all();           // 通知生产者
    }
}

int main() {
    std::thread p(producer, 1);
    std::thread c(consumer, 2);
    p.join();
    c.join();
    return 0;
}

```

易错点/注意:

- `wait` 必须使用 `unique_lock`, 不能用 `lock_guard` (因为需要临时解锁)。
- 判断条件的谓词必须在持锁状态下读取, 条件相关数据也应同一把锁保护。
- 丢失唤醒: 若在 `wait` 之前条件已满足且未用谓词检查, 线程可能永久等待。
- 生产/消费次数不对称时, 上面的简单示例可能因一方提前结束而卡住, 实际工程需用结束标记或两阶段通知。

7. std::atomic (原子操作)

std::atomic (原子操作、fetch_add、CAS、内存序)

概念: `std::atomic<T>` 提供对单个对象的无锁原子操作, 保证「读-改-写」在多线程下是原子且可见的, 常用于计数器、标志位等简单共享数据, 避免使用重量级互斥锁。其成员函数如 `fetch_add`、`compare_exchange_weak/strong` (CAS) 在底层对应硬件原子指令。

要点:

- 原子性: 对 `atomic` 对象的操作不可分割, 不会被其他线程看到中间状态。
- `fetch_add(n)`: 原子地把当前值加 `n`, 返回旧值; `operator++/+=` 也是原子的。
- `store/load`: 原子地写入/读取。
- CAS: `compare_exchange_strong(expected, desired)`, 若当前值等于 `expected` 则写入 `desired` 并返回 `true`, 否则把当前值写回 `expected` 并返回 `false`。
- 内存序: 默认 `std::memory_order_seq_cst` (顺序一致, 最严格最直观); 另有 `relaxed`、`acquire`、`release` 等更弱序, 需谨慎使用。

示例:

```

#include <atomic>
#include <thread>
#include <iostream>

std::atomic<int> counter{0}; // 原子计数器

void add() {
    for (int i = 0; i < 100000; ++i) {
        counter.fetch_add(1); // 原子自增, 返回旧值
    }
}

void cas_demo() {
    int expected = 100; // 期望值
    int desired = 200; // 目标值
    // 若 counter 当前等于 expected, 则写入 desired
}

```

```

    if (counter.compare_exchange_strong(expected, desired)) {
        std::cout << "CAS 成功\n";
    } else {
        // 失败时 expected 被更新为实际当前值
        std::cout << "CAS 失败，实际值为 " << expected << '\n';
    }
}

int main() {
    std::thread a(add);
    std::thread b(add);
    a.join();
    b.join();
    std::cout << "counter = " << counter.load() << '\n'; // 稳定输出 200000
    return 0;
}

```

易错点/注意:

- `atomic` 适合单个变量，多个相关变量的原子性需要锁。
- `compare_exchange` 失败时会修改 `expected`，循环重试时记得用局部变量承载期望值。
- `weak` 版本可能「伪失败」（即使值相等也可能返回 `false`），通常配合循环使用，单次判断用 `strong`。

内存序 (memory_order) 简介示例:

```

#include <atomic>
#include <thread>
#include <iostream>

std::atomic<bool> ready{false};
int data = 0;

void producer() {
    data = 42; // 普通写
    // release: 保证之前的写对后续 acquire 可见
    ready.store(true, std::memory_order_release);
}

void consumer() {
    // acquire: 读到 true 后, producer 之前的写对本线程可见
    while (!ready.load(std::memory_order_acquire)) {
        std::this_thread::yield();
    }
    std::cout << "data = " << data << '\n'; // 保证输出 42
}

int main() {
    std::thread p(producer);
    std::thread c(consumer);
    p.join();
    c.join();
    return 0;
}

```

说明：默认的 `std::memory_order_seq_cst` 提供全局顺序一致、最容易推理；`relaxed` 只保证原子性不保证顺序；`acquire/release` 用于成对的「发布-订阅」同步。除非性能瓶颈明确，优先用默认序。

8. `std::async` / `future` / `promise` / `packaged_task`

`std::async` / `std::future` / `std::promise` / `std::packaged_task`

概念： `std::async` 异步启动一个任务并返回 `std::future`，用于获取任务返回值或传播异常。

`std::promise` 是「生产者」端，通过 `set_value/set_exception` 提供结果；

`std::packaged_task` 把可调用对象包装成可异步执行并能取回结果的任务。`future` 通过 `get()`（阻塞等待）或 `wait_for`（限时等待）取结果。

要点：

- `std::async(launch_policy, f, args...)`： `std::launch::async` 强制新线程，`std::launch::deferred` 延迟到 `get` 时执行。
- `future::get()`： 阻塞直到结果就绪，只能调用一次。
- `future::wait_for(duration)`： 返回 `future_status::ready/timeout/deferred`，可用于超时判断。
- `promise` 与 `future` 通过 `get_future()` 配对，一个 `promise` 只能关联一个 `future`。
- `packaged_task` 通过 `get_future()` 取结果，再调用 `task(args...)` 执行。

示例：

```
#include <future>
#include <thread>
#include <iostream>
#include <chrono>

int compute(int n) {
    int sum = 0;
    for (int i = 1; i <= n; ++i) sum += i;
    return sum;
}

int main() {
    // 1. async 异步计算
    std::future<int> f = std::async(std::launch::async, compute, 100);

    // 2. 限时等待：忙等 100ms 后检查是否完成
    auto status = f.wait_for(std::chrono::milliseconds(100));
    if (status == std::future_status::ready) {
        std::cout << "async 结果 = " << f.get() << '\n';    // 5050
    } else {
        std::cout << "尚未完成，继续阻塞获取\n";
        std::cout << "async 结果 = " << f.get() << '\n';
    }

    // 3. promise 手动设置结果
    std::promise<int> prom;
    std::future<int> f2 = prom.get_future();
    std::thread t([&prom] {
        prom.set_value(42);    // 提供结果
    });
    std::cout << "promise 结果 = " << f2.get() << '\n';
}
```

```

t.join();

// 4. packaged_task 包装可调用对象
std::packaged_task<int(int)> task(compute);
std::future<int> f3 = task.get_future();
task(10); // 执行任务
std::cout << "packaged_task 结果 = " << f3.get() << '\n'; // 55
return 0;
}

```

易错点/注意:

- `future` 的析构若关联 `std::async` 且尚未取结果，会阻塞直到任务完成（这是 `async` 的特殊行为，避免异步任务变成孤儿）。
- `get()` 只能调用一次，再次调用是未定义行为；可用 `valid()` 检查。
- `promise` 若在设置结果前析构，关联 `future` 的 `get()` 会抛 `std::future_error` (broken promise)。

9. std::call_once 与 once_flag

std::call_once 与 once_flag

概念： `std::call_once` 保证某段初始化代码在多线程环境下只被执行一次，配合 `std::once_flag` 使用。它比「双检锁」更安全简洁，常用于惰性初始化、单例等场景，且异常安全（若初始化抛异常，标志保持未执行，可重试）。

要点:

- `once_flag`：记录「是否已执行」状态的标志。
- `call_once(flag, f, args...)`：无论多少线程同时调用，`f` 只执行一次。
- 线程安全且阻塞：第一个进入的线程执行，其余线程等待其完成。
- 异常安全：`f` 抛异常则视为未完成，下一次调用会重试。

示例:

```

#include <mutex>
#include <thread>
#include <iostream>
#include <vector>

std::once_flag init_flag;
int shared_config = 0;

void init() {
    std::cout << "执行初始化（仅一次）\n";
    shared_config = 42;
}

void worker(int id) {
    std::call_once(init_flag, init); // 多线程只初始化一次
    std::cout << "线程 " << id << " 读取配置 " << shared_config << '\n';
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 5; ++i) {

```

```

        threads.emplace_back(worker, i);
    }
    for (auto& t : threads) t.join();
    return 0;
}

```

易错点/注意：

- 每个「一次性初始化」需要独立的 `once_flag`，多个 `call_once` 共用一个 flag 则只会执行一次。
- 初始化函数抛异常时标志不会置位，后续线程会再次尝试执行。

10. std::jthread (C++20)

std::jthread (自动 join、stop_token 简介)

概念： `std::jthread` 是 C++20 引入的线程类，析构时自动调用 `join()`，解决了 `std::thread` 忘记 join 导致 `std::terminate` 的问题。它还内置停止机制：通过 `request_stop()` 请求停止，线程内可用 `stop_token` 检测停止请求，实现协作式取消。

要点：

- 自动 join: `jthread` 析构时若仍 joinable 会自动 join，不会崩溃。
- `request_stop()`：向线程发送停止请求，非强制，需线程主动响应。
- `get_stop_token()`：获取停止令牌，`stop_token::stop_requested()` 检测是否请求停止。
- 线程函数签名可为 `void(std::stop_token st)`，自动接收停止令牌。

示例：

```

#include <thread>
#include <iostream>
#include <chrono>

int main() {
    // 1. 自动 join: 无需显式 join
    {
        std::jthread t([] {
            std::cout << "jthread 自动 join 演示\n";
        });
    } // 离开作用域自动 join, 不会崩溃

    // 2. stop_token 协作式取消
    std::jthread worker([](std::stop_token st) {
        while (!st.stop_requested()) { // 检测停止请求
            std::cout << "工作中...\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
        std::cout << "收到停止请求, 退出\n";
    });

    std::this_thread::sleep_for(std::chrono::milliseconds(350));
    worker.request_stop(); // 请求停止
    worker.join(); // 等待线程响应并结束
    return 0;
}

```

易错点/注意：

- `request_stop()` 是「请求」而非强制，线程必须主动检查 `stop_requested()` 才能退出。
- `jthread` 不可拷贝、可移动，与 `std::thread` 语义一致。
- 自动 `join` 意味着持有 `jthread` 的对象的析构可能阻塞，需注意析构时机。

11. 线程安全的单例模式

线程安全的单例模式（局部 `static` 初始化）

概念：C++11 起，函数内局部 `static` 变量的初始化是线程安全的：多个线程同时首次进入时，只有一个线程执行初始化，其余线程阻塞等待，且由标准保证只构造一次。利用这一点可实现「迈耶斯单例」（Meyers Singleton），无需手动加锁。

要点：

- 局部 `static` 初始化线程安全：编译器负责加一次锁，保证单次构造。
- 写法简洁：`static Singleton& get() { static Singleton instance; return instance; }`。
- 相比双检锁（DCLP）：无锁竞争开销、无内存序陷阱、异常安全。
- 惰性初始化：首次调用 `get()` 时才构造。

示例：

```
#include <iostream>

class Singleton {
public:
    // 禁止拷贝与赋值
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;

    static Singleton& get() {
        static Singleton instance;    // C++11 起线程安全，只构造一次
        return instance;
    }

    void show() const {
        std::cout << "单例地址: " << this << '\n';
    }

private:
    Singleton() = default;    // 私有构造，禁止外部实例化
};

int main() {
    Singleton& a = Singleton::get();
    Singleton& b = Singleton::get();
    a.show();
    b.show();    // 与 a 地址相同，证明是同一对象
    return 0;
}
```

易错点/注意：

- 局部 `static` 的线程安全仅指「初始化」过程，对象内部方法若访问可变共享数据仍需自行同步。

- C++11 之前的编译器不保证此线程安全性，需用 `call_once` 或双检锁（现代代码应避免 DCLP）。

本部分小结

易忘点清单：

主题	易忘/易错点
<code>std::thread</code>	引用传参用 <code>std::ref</code> ；joinable 对象析构会 <code>terminate</code> ；线程函数抛异常会 <code>terminate</code>
数据竞争	<code>volatile</code> 不解决数据竞争； <code>x += 1</code> 非原子
mutex	<code>lock_guard</code> 不能用于 <code>condition_variable::wait</code> ； <code>unique_lock</code> 可延迟/提前解锁
<code>scoped_lock</code>	(C++17) 可一次锁多个互斥量且避免死锁
死锁	四条件：互斥、持有并等待、不可剥夺、循环等待；用 <code>std::lock</code> 或固定顺序破坏循环等待
条件变量	必须用带谓词的 <code>wait</code> 防虚假唤醒；谓词在持锁下检查
<code>atomic</code>	默认 <code>seq_cst</code> ；CAS 失败会改写 <code>expected</code> ；只保证单变量原子
<code>future</code>	<code>get()</code> 只能一次； <code>wait_for</code> 返回 <code>future_status</code>
<code>call_once</code>	每个一次性初始化用独立 <code>once_flag</code> ；异常可重试
<code>jthread</code>	(C++20) 析构自动 join； <code>request_stop</code> 是协作式取消
单例	局部 <code>static</code> 初始化 (C++11) 线程安全，避免双检锁

第十部分：现代 C++ 特性 (C++11/14/17/20/23)

1. C++11 特性

auto、decltype、nullptr 与 static_assert

概念：`auto` 由初始化表达式推导变量类型（剥离顶层 `const`/引用）；`decltype` 在编译期获取表达式的声明类型且不求值；`nullptr` 是类型安全的空指针字面量（`std::nullptr_t`），消除 `NULL`（实为 `0`）的重载歧义；`static_assert` 在编译期断言常量条件，失败即报编译错误。

要点：

- `auto` 剥离顶层 `const` 与引用，需保留时写 `const auto&` / `auto&`；`decltype((x))` 加括号得引用。
- `nullptr` 可区分 `f(int)` 与 `f(void*)` 重载，不要再用 `NULL` / `0` 表示空指针。
- `static_assert(常量, "信息")` 编译期检查，C++17 起提示可省略。

示例：

```
#include <iostream>
```

```

void f(int) { std::cout << "int 版本\n"; }
void f(void*) { std::cout << "指针版本\n"; }
int main() {
    const int x = 10;
    auto a = x;           // int (const 被剥离)
    const auto& b = x;     // const int&
    decltype(x) c = 20;   // const int
    int y = 5;
    decltype((y)) r = y;  // int& (加括号得引用)
    r = 100;              // 修改 r 即修改 y
    f(nullptr);           // 指针版本: 明确匹配 void*
    static_assert(sizeof(int) >= 4, "int 至少 4 字节"); // 编译期断言
    std::cout << y << '\n'; // 100
    return 0;
}

```

易错点/注意:

- `auto s = "hi";` 推导为 `const char*`, 不是 `std::string`。

lambda 表达式

概念: lambda 是匿名函数对象, 语法 `[捕获列表](参数) -> 返回类型 { 函数体 }` (返回类型可省略)。捕获列表决定如何访问外层变量: 按值 `=`、按引用 `&`、指定 `[x, &y]`、`this`; C++14 起支持初始化捕获与泛型 lambda。

要点:

- `[]` 不捕获; `[=]` 按值捕获全部; `[&]` 按引用捕获全部; `[x, &y]` 混合捕获。
- `mutable` 允许修改按值捕获的副本 (不影响原变量)。
- 按引用捕获的变量在 lambda 存活期内必须有效, 避免悬空。

示例:

```

#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{5, 1, 4, 2, 3};
    std::sort(v.begin(), v.end(), [](int a, int b) { return a < b; });
    int threshold = 3;
    int cnt = std::count_if(v.begin(), v.end(),
        [&threshold](int x) { return x > threshold; }); // 引用捕获
    std::cout << "大于 " << threshold << " 的有 " << cnt << " 个\n";
    int n = 0;
    auto inc = [n]() mutable { return ++n; }; // 修改的是副本
    std::cout << inc() << ' ' << inc() << " 外层 n=" << n << '\n'; // 1 2 0
    return 0;
}

```

易错点/注意:

- 按值捕获发生在 lambda 创建时, 之后变量变化不影响副本。

右值引用与移动语义

概念：右值引用 `T&&` 绑定临时对象（右值），实现移动语义：把资源从即将销毁的对象「转移」给新对象，避免深拷贝。`std::move` 只做「左值转右值」的类型转换，真正的转移发生在移动构造/移动赋值中；`std::forward` 配合万能引用实现完美转发。

要点：

- 左值：可取地址的持久对象；右值：临时对象、字面量、`std::move(x)` 的结果。
- 移动构造 `T(T&&)`、移动赋值 `T& operator=(T&&)`：接管资源后把源对象置为「有效但未指定」。
- 移动操作应标记 `noexcept`，否则 `std::vector` 扩容可能退化用拷贝；返回值上不要写 `std::move`。

示例：

```
#include <iostream>
#include <vector>
class Buffer {
public:
    explicit Buffer(size_t n) : data_(new int[n]) { std::cout << "构造\n"; }
    Buffer(Buffer&& other) noexcept : data_(other.data_) { // 移动构造
        other.data_ = nullptr; // 源对象置空
        std::cout << "移动构造\n";
    }
    ~Buffer() { delete[] data_; }
    Buffer(const Buffer&) = delete; // 禁止拷贝
private:
    int* data_;
};
int main() {
    std::vector<Buffer> bufs;
    bufs.reserve(3);
    bufs.push_back(Buffer(100)); // 临时对象触发移动构造
    std::cout << "未发生深拷贝\n";
    return 0;
}
```

易错点/注意：

- 被移动后的对象一般只能安全析构或重新赋值，不要继续使用其资源。

智能指针（`unique_ptr` / `shared_ptr` / `weak_ptr`）

概念：智能指针用 RAII 管理堆内存。`unique_ptr` 独占所有权、只可移动；`shared_ptr` 用引用计数共享所有权；`weak_ptr` 弱引用观察 `shared_ptr`，不增加计数，用于打破循环引用。

要点：

- `make_unique` (C++14)、`make_shared` 创建，避免裸 `new` 与二次分配。
- `weak_ptr::lock()` 返回 `shared_ptr`，对象已销毁则返回空。
- 循环引用（互相持有 `shared_ptr`）会泄漏，一侧改用 `weak_ptr`；勿用同一裸指针构造多个 `shared_ptr`。

示例：

```
#include <iostream>
```

```

#include <memory>
struct Node {
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> prev;    // 弱引用打破循环
    ~Node() { std::cout << "Node 析构\n"; }
};
int main() {
    auto up = std::make_unique<int>(42);
    auto up2 = std::move(up);    // unique_ptr 只能移动
    auto sp1 = std::make_shared<int>(10);
    {
        auto sp2 = sp1;
        std::cout << "计数 = " << sp1.use_count() << '\n';    // 2
    }
    std::weak_ptr<int> wp = sp1;
    if (auto sp3 = wp.lock()) std::cout << "值 = " << *sp3 << '\n';
    return 0;
}

```

易错点/注意:

- `shared_ptr` 的引用计数线程安全，但所指向的对象本身不是。

范围 for、统一初始化列表与 constexpr 增强

概念: 范围 for 简化容器遍历（自动调用 begin/end）；统一初始化列表用 `{}` 初始化任意对象，配合 `std::initializer_list` 支持自定义类型并禁止窄化转换；`constexpr` (C++11) 让函数可编译期求值，C++14 放宽为可含循环、分支、局部变量等多语句。

要点:

- 范围 for: `auto` 按值、`auto&` 可改、`const auto&` 只读；循环内不要增删容器。
- `{}` 统一初始化: 基础类型、数组、容器、聚合体、自定义类型；禁止窄化转换。
- `constexpr` 结果用于数组大小、模板参数等必须常量的场合；运行期也能调用。

示例:

```

#include <iostream>
#include <vector>
#include <initializer_list>
constexpr int factorial(int n) {    // C++14: 可多语句
    int r = 1;
    for (int i = 1; i <= n; ++i) r *= i;
    return r;
}
class Bag {
public:
    Bag(std::initializer_list<int> l) { for (int x : l) d_.push_back(x); }
    void print() const { for (int x : d_) std::cout << x << ' '; std::cout << '\n'; }
private:
    std::vector<int> d_;
};
int main() {
    std::vector<int> v{1, 2, 3, 4};    // 统一初始化
    for (auto& x : v) x *= 2;        // 范围 for 引用修改
    Bag bag{10, 20, 30};            // initializer_list 构造
}

```

```

    bag.print();
    std::cout << "5! = " << factorial(5) << '\n';
    // int a{3.14};    // 错误：窄化转换被禁止
    return 0;
}

```

易错点/注意：

- `initializer_list` 元素只读；构造同时存在 `initializer_list` 重载时，`{}` 优先匹配它。

override/final、using 别名、=default/=delete 与 explicit 转换运算符

概念： `override` 显式声明重写虚函数，签名不符即编译错误；`final` 禁止虚函数再被重写、类再被继承。`using` 作类型别名（支持模板别名），比 `typedef` 清晰。`=default` 要求编译器生成默认特殊成员函数；`=delete` 禁用函数。`explicit` 可用于转换运算符，实现「安全 bool」惯用法。

要点：

- `override/final` 置于函数声明末尾；`final` 也可放类名后禁止继承。
- `using New = Old;` 与模板别名 `template<class T> using Vec = std::vector<T>;`。
- `=default` 保留默认构造等；`=delete` 禁拷贝等；`explicit operator bool()` 让 `if (obj)` 可用。

示例：

```

#include <iostream>
#include <vector>
class Base { public: virtual void draw() const { } };
class Derived : public Base {
public:
    void draw() const override { }           // 显式重写
};
class Final final : public Derived { };      // final 类禁止继承
template<class T> using Vec = std::vector<T>; // 模板别名
class NonCopyable {
public:
    NonCopyable() = default;
    NonCopyable(const NonCopyable&) = delete; // 禁止拷贝
    NonCopyable& operator=(const NonCopyable&) = delete;
};
class SafeBool {
public:
    explicit operator bool() const { return ok_; } // 安全 bool
    void set(bool v) { ok_ = v; }
private:
    bool ok_ = false;
};
int main() {
    Vec<int> v{1, 2, 3};
    NonCopyable a;
    SafeBool s;
    if (s) std::cout << "s 为真\n";
    std::cout << "v.size = " << v.size() << '\n';
    return 0;
}

```

易错点/注意:

- 只写 `override` 不写 `virtual` 会编译报错; `explicit operator bool` 避免 `obj + 1` 隐式转换。

可变参数模板、`std::tuple` 与 `thread` 库

概念: 可变参数模板用参数包 `typename... Args` 接受任意数量类型参数, 通过递归展开处理, 常用于 `tuple`、`emplace_back` 等。`std::tuple` 是异构定长集合, 用 `std::get<N>` 取值、`std::tie` 解包。`thread` 库 (`std::thread`、`mutex`、`atomic`、条件变量等) 已在第九部分详述。

要点:

- `template<class... Args>` 声明参数包, `Args...` 展开, `sizeof...(Args)` 取个数。
- 递归模式: 基函数处理最后一个参数 + 递归函数每次剥一个; C++17 可用折叠表达式替代。
- `std::tuple` + `std::get<0>`、`std::tie`、`std::make_tuple`。

示例:

```
#include <iostream>
#include <tuple>
#include <string>
template<class T> T sum(T v) { return v; } // 递归终止
template<class T, class... Args> T sum(T first, Args... rest) {
    return first + sum(rest...); // 递归展开
}
std::tuple<int, std::string> info() { return std::make_tuple(42, "hi"); }
int main() {
    std::cout << "sum = " << sum(1, 2, 3, 4, 5) << '\n'; // 15
    auto t = info();
    std::cout << std::get<0>(t) << ' ' << std::get<1>(t) << '\n';
    int id; std::string name;
    std::tie(id, name) = info(); // tie 解包
    std::cout << "id=" << id << ", name=" << name << '\n';
    return 0;
}
```

易错点/注意:

- 递归展开必须有终止条件 (基函数); `std::get` 索引越界会编译错误。

2. C++14 特性

泛型 lambda 与返回类型推导

概念: C++14 允许 lambda 参数用 `auto` (生成模板 `operator()`) , 函数可用 `auto` 推导返回类型 (C++11 需尾置返回) , 让泛型代码更简洁; `decltype(auto)` 可保留返回表达式的引用与 `const` 性质。

要点:

- 泛型 lambda: `[](auto a, auto b) { return a + b; }` 可用于多种类型。
- `auto` 返回推导要求函数内所有 `return` 类型一致。

示例:

```
#include <iostream>
#include <string>
auto add(int a, int b) { return a + b; } // 返回类型推导为 int
int main() {
    auto plus = [](auto a, auto b) { return a + b; }; // 泛型 lambda
    std::cout << plus(1, 2) << '\n'; // 3
    std::cout << plus(std::string("a"), std::string("b")) << '\n'; // ab
    std::cout << "add = " << add(3, 4) << '\n';
    return 0;
}
```

易错点/注意：

- 返回类型推导函数多个 return 类型不同会导致编译错误。

make_unique 与变量模板

概念： `std::make_unique<T>(args...)` (C++14) 安全创建 `unique_ptr`，避免 `new` 的异常泄漏。变量模板 `template<class T> T name = value;` 定义「模板化的变量」，如 `pi<T>`，是 C++17 `std::is_same_v` 等萃取变量的基础。

要点：

- `make_unique` 返回 `unique_ptr<T>`，支持数组 `make_unique<T[]>(n)`。
- 变量模板实例化时给出类型实参，可被特化。

示例：

```
#include <iostream>
#include <memory>
#include <type_traits>
template<class T> constexpr T pi = T(3.1415926535897932385L); // 变量模板
int main() {
    auto p = std::make_unique<int>(42);
    std::cout << *p << '\n';
    auto arr = std::make_unique<int[]>(3); // 动态数组
    for (int i = 0; i < 3; ++i) arr[i] = i;
    std::cout << "pi<double> = " << pi<double> << '\n';
    static_assert(std::is_same_v<int, int>); // is_same_v 即变量模板
    return 0;
}
```

易错点/注意：

- 优先用 `make_unique` / `make_shared` 而非裸 `new`。

二进制字面量、数字分隔符、constexpr 多语句与 std::exchange

概念： C++14 引入二进制字面量 `0b...` 与数字分隔符 `'`（只影响可读性），并放宽 `constexpr` 可含多语句。`std::exchange(obj, new)` 用新值替换对象并返回旧值，常用于「取出并置空」与状态切换。

要点：

- `0b1010` 表示十进制 10；分隔符不能放数值开头/结尾或紧邻小数点、指数符号。
- C++14 `constexpr` 函数可含局部变量、循环、分支。
- `std::exchange`： `obj = new` 并返回旧值。

示例:

```
#include <iostream>
#include <utility>
#include <string>
constexpr int f(int n) {                // C++14 constexpr 多语句
    int s = 0;
    for (int i = 1; i <= n; ++i) s += i;
    return s;
}
int main() {
    int mask = 0b1010'1100;              // 二进制 + 分隔符 = 172
    int big  = 1'000'000;
    static_assert(f(10) == 55);
    std::string s = "hello";
    std::string old = std::exchange(s, "world"); // s="world", 返回旧值
    std::cout << mask << ' ' << big << " old=" << old << " s=" << s << '\n';
    return 0;
}
```

易错点/注意:

- `std::exchange` 先取旧值再赋值, 注意副作用顺序; `constexpr` 函数运行期也能调用。

3. C++17 特性

结构化绑定

概念: 结构化绑定用 `auto [a, b] = expr;` 一次性解包 `pair`、`tuple`、数组、聚合体到多个变量, 替代 `std::tie`, 配合 `std::map` 遍历非常方便。

要点:

- `auto [x, y]` 绑定值, `auto& [x, y]` 绑定引用 (可修改原对象)。
- 支持 `pair`、`tuple`、数组、公有成员结构体; 变量个数必须匹配; 必须用 `auto`。

示例:

```
#include <iostream>
#include <map>
#include <string>
struct Point { int x, y; };
int main() {
    std::map<std::string, int> m{"a", 1}, {"b", 2};
    for (auto& [k, v] : m) std::cout << k << "=" << v << ' '; // 遍历 map
    std::cout << '\n';
    auto [a, b] = std::make_pair(10, 20); // 解包 pair
    Point p{3, 4};
    auto& [px, py] = p; // 引用绑定
    px = 100; // 修改原对象
    std::cout << a << ' ' << b << " p.x=" << p.x << '\n';
    return 0;
}
```

易错点/注意:

- 绑定引用的生命周期与被解包对象一致, 对象销毁后引用悬空。

if constexpr 与 if/switch 初始化语句

概念: `if constexpr` 在编译期根据常量条件选择分支，未命中分支不参与实例化，实现「按类型分派」而无需 SFINAE。`if (init; cond) / switch (init; expr)` 把初始化变量的作用域限制在语句内。

要点:

- `if constexpr` 条件必须编译期常量；模板中不同实例化只编译对应分支。
- 普通 `if` 判断模板参数时未匹配分支也必须合法，`if constexpr` 解决此问题。
- `if (auto it = m.find(k); it != m.end())`: `it` 作用域限于 `if`。

示例:

```
#include <iostream>
#include <string>
#include <type_traits>
template<class T> std::string name() {
    if constexpr (std::is_integral_v<T>) return "整数";
    else if constexpr (std::is_floating_point_v<T>) return "浮点";
    else return "其他";
}
int main() {
    std::cout << name<int>() << ' ' << name<double>() << ' ' <<
name<std::string>() << '\n';
    std::string s = "hello";
    if (auto pos = s.find('e'); pos != std::string::npos) {    // if 初始化语句
        std::cout << "e 在位置 " << pos << '\n';
    }
    return 0;
}
```

易错点/注意:

- `if constexpr` 在非模板语境条件也必须是编译期常量。

std::optional / std::variant / std::any

概念: C++17 三个可选/变体类型。`optional<T>` 表示「可能有值或无值」，替代哨兵值表达缺失；`variant<A,B,...>` 是类型安全联合体，任一时刻持有一种类型；`any` 可存任意可拷贝类型，用 `any_cast` 取出。

要点:

- `optional`: `has_value()` / `operator bool` 判断，`value()` 取值（无值抛异常），`value_or(default)` 兜底。
- `variant`: `std::get<T>` / `get<idx>` 取值，`std::visit` 泛型访问，`holds_alternative<T>` 判断。
- `any`: `any_cast<T>` 取出，类型不匹配抛 `std::bad_any_cast`。

示例:

```
#include <iostream>
#include <optional>
#include <variant>
#include <any>
#include <string>
```

```

std::optional<int> divide(int a, int b) {
    if (b == 0) return std::nullopt;
    return a / b;
}

int main() {
    auto r = divide(10, 2);
    if (r) std::cout << "10/2 = " << *r << '\n';
    std::cout << "默认值 = " << divide(1, 0).value_or(-1) << '\n';
    std::variant<int, std::string> v = "hello";
    std::cout << std::get<std::string>(v) << '\n';
    v = 42;
    std::visit([](auto&& x) { std::cout << "值 = " << x << '\n'; }, v);
    std::any a = 3.14;
    std::cout << "any = " << std::any_cast<double>(a) << '\n';
    return 0;
}

```

易错点/注意:

- `optional::value()` 无值抛 `bad_optional_access`; `variant::get<T>` 类型不符抛 `bad_variant_access`。

std::string_view 与 std::filesystem

概念: `string_view` 是字符串的只读视图 (指针 + 长度), 不拥有数据、`substr` 为 $O(1)$, 适合只读参数与子串。 `filesystem` (C++17) 提供跨平台文件操作: `path`、目录遍历、状态查询、创建删除等。

要点:

- `string_view` 引用的数据必须在视图使用期存活, 勿绑定临时对象; 结尾未必有 `\0`。
- `filesystem::path` 支持 / 拼接、`filename()`、`extension()`; `directory_iterator` 遍历目录; `exists()` / `is_directory()` 查询。

示例:

```

#include <iostream>
#include <string>
#include <string_view>
#include <filesystem>
namespace fs = std::filesystem;
void print(std::string_view sv) { std::cout << sv << " len=" << sv.size() << '\n'; }
int main() {
    std::string s = "hello world";
    std::string_view sub = std::string_view(s).substr(0, 5);    // O(1) 子串
    print(sub);          // hello
    print("字面量");
    fs::path dir = fs::current_path();
    std::cout << "当前目录: " << dir << '\n';
    for (const auto& e : fs::directory_iterator(dir)) {
        std::cout << (e.is_directory() ? "[目录] " : "[文件] ")
            << e.path().filename() << '\n';
    }
    return 0;
}

```

易错点/注意:

- 不要 `string_view sv = std::string("临时");`（悬空）；遍历目录时增删项会使迭代器失效。

折叠表达式、inline 变量与 CTAD

概念：折叠表达式 (C++17) 用 `(args + ...)` 等把参数包折叠成单个表达式，替代递归模板。`inline` 变量允许在头文件定义变量而不违反 ODR，`inline static` 成员可类内初始化。CTAD（类模板实参推导）让 `std::vector v{1,2,3}` 免写模板实参。

要点：

- 一元左折叠 `(... op pack)`、右折叠 `(pack op ...)`；二元折叠 `(init op ... op pack)` 可给初值。
- `inline int config = 10;` 可写进头文件；CTAD 从构造实参推导模板实参。

示例：

```
#include <iostream>
#include <vector>
#include <utility>
inline int global = 100;           // inline 变量
class C { public: inline static int ver = 3; }; // 类内初始化 static
template<class... Args> auto sum(Args... args) { return (args + ...); } // 折叠求和
template<class... Args> void print(Args... args) { (std::cout << ... << args) << '\n'; }
int main() {
    std::vector v{1, 2, 3};        // CTAD: vector<int>
    std::pair p{42, "answer"};     // CTAD: pair<int, const char*>
    std::cout << "sum=" << sum(1, 2, 3, 4) << " ver=" << C::ver << " global=" << global << '\n';
    print("a", 1, 2.5, "b");
    return 0;
}
```

易错点/注意：

- 折叠方向影响减法/除法结果；空包的一元 `+` 折叠不合法，需二元折叠给初值。

std::clamp、属性与 std::scoped_lock

概念：`std::clamp(v, lo, hi)` 把值限制在区间内。属性提供额外语义：`[[nodiscard]]` 警告忽略返回值、`[[fallthrough]]` 显式标注 switch 落空、`[[maybe_unused]]` 抑制未使用告警。

`std::scoped_lock` (C++17) 可同时锁多个互斥量并避免死锁（第九部分已详述）。

要点：

- `clamp` 要求 `lo <= hi`，否则未定义行为。
- `[[nodiscard]]` 用于函数/类型；`[[fallthrough]]` 放 case 间；`[[maybe_unused]]` 用于变量/参数。
- `scoped_lock` 可变参数、一次锁多把锁、异常安全。

示例：

```
#include <iostream>
#include <algorithm>
[[nodiscard]] int compute() { return 42; }
void classify(int x) {
```

```

switch (x) {
case 1:
    [[fallthrough]];          // 显式落空
case 2:
    std::cout << "1 或 2\n"; break;
default:
    std::cout << "其他\n";
}
}
int main() {
    std::cout << std::clamp(5, 0, 10) << ' ' << std::clamp(50, 0, 10) << ' '
        << std::clamp(-5, 0, 10) << '\n';    // 5 10 0
    [[maybe_unused]] int unused = 1;
    classify(1);
    compute();    // 忽略返回值 → nodiscard 告警
    return 0;
}

```

易错点/注意:

- `clamp` 传 `lo > hi` 是未定义行为; `nodiscard` 只是警告而非强制错误。

4. C++20 特性

concepts (概念与约束)

概念: concepts 用 `concept` 与 `requires` 定义类型约束, 让模板参数满足特定「概念」才能实例化, 编译报错更清晰, 且能参与重载决议, 是现代泛型编程方式。

要点:

- 定义: `template<class T> concept Addable = requires(T a, T b) { a + b; };`
- 使用: `template<Addable T> void f(T)` 或 `requires` 子句; 概念可 `&&/||/!` 组合。

示例:

```

#include <iostream>
#include <concepts>
template<class T> concept Comparable = requires(T a, T b) { a < b; };
template<Comparable T> T my_max(T a, T b) { return (a < b) ? b : a; }
int main() {
    std::cout << my_max(3, 5) << '\n';    // 5
    std::cout << my_max(3.5, 1.2) << '\n'; // 3.5
    return 0;
}

```

易错点/注意:

- `requires` 只检查语法合法性, 不保证运行时语义。

ranges (范围库)

概念: `<ranges>` 提供「范围 + 视图 + 算法」管道式编程, `std::ranges::sort(v)` 直接传容器, 视图适配器 `filter/transform/take` 用 `|` 组合、惰性求值、不产生中间容器。

要点:

- `std::ranges::sort(v)` 无需 `begin/end`。

- 视图惰性：遍历时才计算，底层容器须存活且不被修改。

示例：

```
#include <iostream>
#include <vector>
#include <ranges>
#include <algorithm>
int main() {
    std::vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    auto view = v
        | std::views::filter([](int x) { return x % 2 == 0; })
        | std::views::transform([](int x) { return x * x; })
        | std::views::take(3);
    for (int x : view) std::cout << x << ' ';    // 4 16 36
    std::cout << '\n';
    std::ranges::sort(v, std::greater{});
    for (int x : v) std::cout << x << ' ';
    return 0;
}
```

易错点/注意：

- 视图对象轻量，但多次遍历会重复计算底层操作。

三路比较 <=> 与默认比较

概念：三路比较运算符 <=>（太空船）返回

strong_ordering/weak_ordering/partial_ordering，一次表达小于/等于/大于；auto operator<=>(const T&) const = default; 让编译器按成员字典序自动生成全部六种比较运算符。

要点：

- a <=> b 与 0 比较判断 </==/>。
- strong_ordering/weak_ordering 可合成 ==，partial_ordering（含 NaN）不能；自定义 <=> 后不会自动生成 ==。

示例：

```
#include <iostream>
#include <compare>
struct Point {
    int x, y;
    auto operator<=>(const Point&) const = default;    // 自动生成全部比较
};
int main() {
    Point a{1, 2}, b{1, 3}, c{1, 2};
    if (a < b) std::cout << "a < b\n";
    if (a == c) std::cout << "a == c\n";
    auto r = a <=> b;
    if (r < 0) std::cout << "a 小于 b\n";
    return 0;
}
```

易错点/注意：

- 浮点比较用 partial_ordering，不合成 ==。

std::span 与 std::jthread

概念： `std::span<T>` 是连续内存的轻量视图（指针 + 长度），可引用数组、vector、array 等而不拥有数据，比裸指针安全。`std::jthread` 析构自动 `join`，并内置 `stop_token` 协作式取消（详见第九部分）。

要点：

- `span` 不拥有数据，引用的容器须存活；只适用连续内存（不能是 list/deque）。
- 支持 `subspan`、`first/last`；动态 `span<T>` 与固定 `span<T,N>`。
- `jthread`：`request_stop()` 请求停止，线程内 `stop_requested()` 检测。

示例：

```
#include <iostream>
#include <span>
#include <vector>
#include <thread>
void print(std::span<const int> s) { for (int x : s) std::cout << x << ' ';
std::cout << '\n'; }
int main() {
    int arr[] = {1, 2, 3, 4};
    std::vector<int> v{5, 6, 7, 8};
    print(arr);          // 传 C 数组
    print(v);            // 传 vector
    std::span<int> s(arr);
    s[0] = 100;          // 通过 span 修改原数组
    std::jthread t([](std::stop_token st) { // 自动 join + 协作取消
        while (!st.stop_requested()) { }
    });
    t.request_stop();
    return 0;
}
```

易错点/注意：

- `span` 引用的容器必须在 `span` 使用期存活；`request_stop` 是协作式，线程需主动检查。

协程与 modules 简述

概念： 协程用 `co_await/co_yield/co_return` 实现可挂起恢复的函数，编译器转为状态机；C++20 只提供底层设施（`coroutine_handle` 等），高层 `std::generator` 到 C++23。模块用 `export module/import` 替代 `#include`，减少重复编译、隔离宏污染、提升编译速度。

要点：

- 协程：`co_await` 挂起、`co_yield` 产出、`co_return` 返回；局部状态存堆上。
- 模块：`export module m;` 声明，`import m;` 导入；不泄漏宏与实现细节。

示例：

```
#include <coroutine>
#include <iostream>
struct Task { // 极简协程骨架
    struct promise_type {
        Task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
```

```
std::suspend_never final_suspend() noexcept { return {}; }
void return_void() {}
void unhandled_exception() {}

};

};

Task demo() {
    co_await std::suspend_always{};    // 挂起点
    std::cout << "恢复执行\n";
    co_return;
}

// ---- 模块（另文件）----
// export module math;
// export int add(int a, int b) { return a + b; }
// import math;    // 使用方导入
int main() {
    std::cout << "协程/modules 骨架示例\n";
    return 0;
}
```

易错点/注意:

- 协程直接使用复杂，生产常用 `cppcoro` 或 C++23 `std::generator`；模块扩展名因编译器而异（`.ixx/.cppm`）。

日历与、constexpr/constinit、指定初始化器与 bit_cast

概念: `<chrono>` 日历提供 `year_month_day` 等日期类型; `<format>` 提供 `std::format` 类型安全格式化。 `constexpr` 声明必须编译期求值的立即函数; `constexpr` 保证变量编译期初始化 (避免静态初始化顺序问题)。指定初始化器按成员名赋初值; `std::bit_cast` 安全按位重解释类型。

要点:

- `std::format("{} ", x)` 支持 `{:04d}/{:02f}`; `year_month_day` 用 `2024y/3/15` 构造。
- `constexpr` 不能运行期调用; `constexpr` 变量运行期可变 (只约束初始化)。
- 指定初始化器须按成员声明顺序; `bit_cast<To>(from)` 要求两端等大小且 trivially copyable。

示例：

```
#include <iostream>
#include <format>
#include <chrono>
#include <bit>
#include <cstdint>

constexpr int square(int x) { return x * x; } // 必须编译器
constexpr int global = square(5);           // 编译期初始化

struct Point { int x, y; };

int main() {
    std::cout << std::format("pi={:.3f} n={:04d}\n", 3.14159, 42);
    using namespace std::chrono;
    year_month_day d = 2024y / 3 / 15;
    std::cout << std::format("{}-{}-{}\n", (int)d.year(), (unsigned)d.month(),
(unsigned)d.day());
    Point p{x = 1, .y = 2}; // 指定初始化器
    std::uint32_t bits = std::bit_cast<std::uint32_t>(1.0f);
    std::cout << p.x << ' ' << std::hex << bits << '\n';
    global = 100;           // constexpr 变量可改
    return 0;
}
```

```
}
```

易错点/注意:

- `constexpr` 不能用于运行期变量实参; 指定初始化器乱序/跳序会编译错误。

5. C++23 特性

`std::expected`

概念: `std::expected<T, E>` 表示「要么成功值 T、要么错误 E」, 比异常更适合可预期错误 (解析失败、查找缺失), 比 `optional` 多携带错误信息, 并强制调用方处理。

要点:

- `has_value()` 判断; `value()` 取值、`error()` 取错误; 失败态调 `value()` 抛 `bad_expected_access`。
- 单子操作 `and_then`/`transform`/`or_else` 链式处理; 替代「输出参数 + bool 返回值」模式。

示例:

```
#include <iostream>
#include <expected>
#include <string>

std::expected<int, std::string> parse(const std::string& s) {
    try { return std::stoi(s); }
    catch (...) { return std::unexpected("解析失败"); }
}

int main() {
    auto ok = parse("123");
    if (ok) std::cout << "成功: " << *ok << '\n';
    auto bad = parse("abc");
    if (!bad) std::cout << "失败: " << bad.error() << '\n';
    return 0;
}
```

易错点/注意:

- 需 C++23 与较新标准库; 先用 `has_value()` 判断再取 `value()`。

`std::print` / `std::println`

概念: `std::print`/`std::println` 提供比 `std::cout` 简洁的类型安全格式化输出, 内部基于 `std::format`, 支持占位符与格式说明符, `println` 自动补换行。

要点:

- `std::print("格式", args...)`; `std::println` 末尾加换行。
- 支持 `{}`、位置参数 `{0}`、格式说明符; 头文件 `<print>`, 需较新编译器。

示例:

```
#include <print>
int main() {
    std::print("不换行 ");
    std::println("自动换行");
    int x = 42;
    std::println("x={}, pi={:.2f}, 重复 {0}", x, 3.14159);
    return 0;
}
```

易错点/注意:

- `std::print` 不自动刷新缓冲（遇换行或显式 `flush` 才刷新）。

mdspan 与 ranges 增强 (views::zip / enumerate)

概念: `std::mdspan` 是多维数组视图，对连续内存做多维索引（矩阵/张量）而不拥有数据。ranges 增强新增 `views::zip`（把多个范围打包成元组序列）、`views::enumerate`（附带下标）等惰性视图。

要点:

- `mdspan<T, extents<...>>` 描述形状与步长，`operator[]` 多维访问。
- `views::zip(v1, v2)` 同时遍历多范围；`views::enumerate(v)` 附带索引；视图均惰性。

示例:

```
#include <iostream>
#include <vector>
#include <ranges>
#include <mdspan>
int main() {
    std::vector<int> a{1, 2, 3}, b{10, 20, 30};
    for (auto [i, x] : std::views::enumerate(a)) std::cout << "a[" << i << "]="
    << x << ' ';
    std::cout << '\n';
    for (auto [x, y] : std::views::zip(a, b)) std::cout << x + y << ' '; // 11
    22 33
    std::cout << '\n';
    int raw[6] = {1, 2, 3, 4, 5, 6};
    std::mdspan<int, std::extents<int, 2, 3>> mat(raw); // 2 行 3 列
    std::cout << "mat[1][2]=" << mat[1, 2] << '\n'; // 6
    return 0;
}
```

易错点/注意:

- 需 C++23 与较新标准库；视图不拥有数据。

if constexpr 与 std::generator 简介

概念: `if constexpr` 判断当前调用是否处于编译期，可为同一函数提供编译期/运行期两套实现。

`std::generator<T>` 是基于协程的同步生成器，用 `co_yield` 逐个产出值，消费者用范围 `for` 惰性遍历。

要点:

- `if constexpr { } else { }`：编译期分支，`else` 分支在编译期求值上下文不实例化。
- `std::generator<T>`：内部 `co_yield v` 产出序列，挂起-恢复实现惰性。

示例：

```
#include <iostream>
constexpr int twice(int x) {
    if constexpr { return x * 2; } // 编译期实现
    else { return x * 2; }         // 运行期实现
}
int main() {
    constexpr int c = twice(21);    // 编译期
    std::cout << "编译期=" << c << " 运行期=" << twice(5) << '\n';
    // std::generator<int> seq() {    // 需 #include <generator> 与 C++23 协程
    //     for (int i = 0; i < 5; ++i) co_yield i;
    // }
    // for (int v : seq()) std::cout << v << ' ';
    return 0;
}
```

易错点/注意：

- `if constexpr` 的 `else` 分支在编译期上下文中不实例化；`generator` 需完整 C++23 协程支持。

6. 版本特性速查表

特性	引入版本
auto / decltype / nullptr / lambda / 右值引用与移动	C++11
智能指针 / 范围 for / 统一初始化列表 / constexpr	C++11
override / final / static_assert / using 别名 / =default / =delete	C++11
explicit 转换运算符 / 可变参数模板 / tuple / thread 库	C++11
泛型 lambda / 返回类型推导 / make_unique / 变量模板	C++14
二进制字面量 / 数字分隔符 / constexpr 多语句 / exchange	C++14
结构化绑定 / if constexpr / if-init 语句	C++17
optional / variant / any / string_view / filesystem	C++17
折叠表达式 / inline 变量 / CTAD	C++17
clamp / nodiscard / fallthrough / maybe_unused / scoped_lock	C++17
concepts / ranges / 三路比较 <=> / span	C++20
jthread / 协程 / modules / 日历 /	C++20
constexpr / constexpr / 指定初始化器 / bit_cast	C++20
expected / print / println / mdspan	C++23
views::zip / enumerate / if constexpr / generator	C++23

7. 升级代码时的兼容性注意

要点：

- 工具链版本：<format>、<print>、<generator>、<mdspan> 依赖较新编译器与标准库，升级前先确认支持。
- 已移除/废弃 API：std::auto_ptr（废弃）、std::result_of（移除，改用 invoke_result）、std::random_shuffle（移除，改用 shuffle）。
- 语义变化：C++20 起 u8 字面量类型从 char 变为 char8_t；C++17 起 std::string 不再保证 COW。
- ABI 兼容：不同标准编译的动态库可能 ABI 不兼容，混编需谨慎。
- 迁移建议：逐步引入更高 -std=，用 [[deprecated]] 标记旧接口，配合警告逐步替换。

示例：

```
#include <iostream>
#include <type_traits>
template<class F, class... Args> using R = std::invoke_result_t<F, Args...>; //
替代 result_of
[[deprecated("请使用 new_api")]] void old_api() { }
void new_api() { }
int main() {
    auto s = u8"hello"; // C++20 起为 const char8_t*
    std::cout << "兼容性示例\n";
    old_api();           // 触发 deprecated 警告
    new_api();
    return 0;
}
```

易错点/注意：

- 升级后先全量编译关注弃用警告，逐个替换已移除 API；不要把编译器扩展当标准。

本部分小结

易忘点清单：

主题	易忘/易错点
auto	剥离 const/引用; <code>auto s="x"</code> 是 <code>const char*</code>
lambda	按引用捕获注意生命周期; <code>mutable</code> 改的是副本
移动语义	移动构造应 <code>noexcept</code> ; 被移动对象状态未指定
智能指针	勿裸指针构造多个 <code>shared_ptr</code> ; 循环引用用 <code>weak_ptr</code>
范围 for	循环内别增删容器
constexpr	C++11 单 return、C++14 多语句; 运行期也能调用
结构化绑定	必须 <code>auto</code> ; C++17 不能指定单个类型
if constexpr	未命中分支不实例化
string_view	不拥有内存, 别引用临时对象; <code>substr</code> 是 $O(1)$
clamp	<code>lo > hi</code> 是未定义行为
ranges/views	惰性求值, 底层容器须存活
<code><=></code>	<code>partial_ordering</code> 不合成 <code>==</code>
span	只适用连续内存
constexpr/constexpr	<code>constexpr</code> 必须编译期; <code>constexpr</code> 只约束初始化
bit_cast	两端等大小且 trivially copyable
升级兼容	注意已移除 API (<code>result_of/random_shuffle</code>) 与 <code>u8</code> 类型变化

第十一部分：预处理、命名空间与工程实践

预处理概述

预处理阶段做什么

概念：预处理是编译过程的第一个阶段，它在真正的词法分析、语法分析和代码生成之前运行。预处理器处理以 `#` 开头的指令（如 `#include`、`#define`、`#if`），完成文本层面的替换、条件选择和文件包含，最终产出一个“纯 C++ 源码”的翻译单元交给编译器。

要点：

- 预处理是**纯文本操作**，不理解 C++ 语法，不做类型检查。
- 主要工作：包含头文件、宏展开、条件编译、删除注释、行号标记。
- 可以用 `g++ -E` 单独查看预处理后的结果。
- 预处理后的结果仍是一个合法的源码文本（.i 文件）。

示例：

```
// 假设源码 main.cpp 内容如下
#include <iostream>           // 会被替换为 iostream 文件的完整内容
#define MAX(a,b) ((a)>(b)?(a):(b)) // 定义宏
int main() {
    std::cout << MAX(1+2, 3) << "\n"; // MAX 在预处理阶段展开为 ((1+2)>(3)?(1+2):(3))
    return 0;
}
// 在命令行执行: g++ -E main.cpp -o main.i 可查看展开后的文本
```

易错点/注意：预处理不检查语义，因此宏导致的错误往往要等到编译或链接阶段才暴露，报错信息里的行号也可能因宏展开而难以理解。

#include

include 尖括号与双引号的区别

概念：`#include` 用于把一个头文件的内容原样插入到当前文件中。尖括号 `<>` 与双引号 `" "` 的区别在于头文件的搜索路径不同。

要点：

- `#include <header>`：优先在**系统/标准库目录**中查找。
- `#include "header"`：优先在**当前源文件所在目录**中查找，找不到再回退到系统目录。
- 现代项目通常用 `-I` 指定额外的用户头文件搜索路径。
- 头文件通常以 `.h`、`.hpp` 或 `.hh` 结尾。

示例：

```
#include <vector>           // 尖括号：搜索标准库目录，如 /usr/include/c++
#include "my_vector.h"      // 双引号：先搜索当前目录，再搜索 -I 指定的目录
#include <iostream>         // 标准库头文件应使用尖括号

int main() {
    std::vector<int> v{1, 2, 3}; // 使用标准库 vector
    return 0;
}
```

易错点/注意：自定义头文件一般用双引号，标准库和第三方库安装的头文件用尖括号；混用会导致找不到文件或引入错误的同名头文件。

include guard 与 #pragma once

概念：头文件可能被多次包含（直接或间接），导致类型重复定义。`include guard`（头文件保护宏）与 `#pragma once` 都是为了防止同一头文件在一个翻译单元中被重复包含。

要点：

- `include guard` 写法：文件开头 `#ifndef XXX_H + #define XXX_H`，结尾 `#endif`。
- 宏名通常用文件名全大写加下划线，保证全局唯一。
- `#pragma once` 是编译器扩展，几乎所有主流编译器都支持，写法更简洁。
- 二者可以只选其一；`include guard` 是标准写法，可移植性最好。

示例：

```
// my_vector.h — 经典 include guard 写法
```

```

#ifndef MY_VECTOR_H      // 若宏未定义才进入
#define MY_VECTOR_H      // 立即定义，防止再次进入

struct Vec {
    int x, y;
};

#endif // MY_VECTOR_H

// 另一种等价的简洁写法（大多数编译器支持）
#pragma once             // 编译器保证本文件只被包含一次
struct Point {
    double x, y;
};

```

易错点/注意：guard 宏名若与其它头文件撞名，会导致其中一个头文件内容被"吞掉"；`#pragma once` 依赖编译器对文件身份的判断，在软链接/硬链接等极端场景下行为可能不一致。

#define 宏

对象宏（简单常量替换）

概念：对象宏（object-like macro）把某个标识符替换为固定的文本，常见用途是定义常量或简单的开关。

要点：

- 语法：`#define 名称 替换文本`。
- 预处理阶段进行**纯文本替换**，不带类型。
- 现代 C++ 更推荐用 `constexpr` 或 `const` 定义常量，宏仅用于必须场合。
- 用 `#undef` 可以取消定义。

示例：

```

#include <iostream>
#define PI 3.14159          // 对象宏：所有 PI 被替换为 3.14159
#define APP_NAME "MyApp"    // 字符串宏

int main() {
    double r = 2.0;
    std::cout << PI * r * r << "\n";    // 展开为 3.14159 * 2.0 * 2.0
    std::cout << APP_NAME << "\n";
    // 现代写法对比：constexpr 有类型、可调试、参与作用域
    constexpr double pi = 3.14159;
    std::cout << pi * r * r << "\n";
    return 0;
}

```

易错点/注意：宏没有类型也没有作用域，可能污染全局命名空间；能用 `constexpr` / `const` / `enum` / `class` 时就别用宏。

函数宏与副作用

概念：函数宏（function-like macro）形如 `#define MAX(a,b) ...`，调用时进行带参文本替换。由于它是文本替换而非真正的函数调用，参数若含副作用表达式会被**求值多次**，产生意外结果。

要点：

- 函数宏的"参数"在替换文本中每次出现都会原样代入。
- 副作用表达式（如 `i++`、`f()`）会被重复求值。
- 现代 C++ 优先用 `inline` 函数、`constexpr` 函数或模板替代函数宏。

示例：

```
#include <iostream>
#define SQUARE(x) ((x) * (x))    // 参数 x 出现了两次

int main() {
    int i = 2;
    int a = SQUARE(++i);         // 展开为 ((++i) * (++i))，++i 执行两次，结果未定义！
    std::cout << a << "\n";      // 行为未定义，不同编译器结果可能不同

    // 正确做法：用函数/模板，参数只求值一次
    auto square = [](int v) { return v * v; };
    int j = 2;
    std::cout << square(++j) << "\n"; // 输出 9，++j 只执行一次
    return 0;
}
```

易错点/注意：函数宏传 `i++`、`++i`、函数调用等副作用表达式会导致多次求值；如果必须用宏，不要把有副作用的表达式传进去。

参数加括号与整体加括号

概念：宏替换是文本级别的，若不把参数和整个表达式用括号包起来，在复杂表达式中会因**运算符优先级**改变语义。

要点：

- 替换文本中的每个参数都要用括号包裹：`(x)` 而非 `x`。
- 整个替换结果也要用括号包裹，避免被外层表达式"吞并"。
- 这条规则是函数宏安全性的最低要求，但即便加括号也无法解决多次求值问题。

示例：

```
#include <iostream>
// 错误写法：不加括号
#define BAD_ADD(a,b) a + b
// 正确写法：参数加括号 + 整体加括号
#define GOOD_ADD(a,b) ((a) + (b))

int main() {
    // BAD_ADD(1,2) * 3 展开为 1 + 2 * 3 = 7，而非期望的 (1+2)*3 = 9
    std::cout << BAD_ADD(1,2) * 3 << "\n";    // 输出 7，错误！
    std::cout << GOOD_ADD(1,2) * 3 << "\n";    // 输出 9，正确
    return 0;
}
```

易错点/注意：写函数宏时"每个参数加括号 + 整个式子加括号"是铁律；漏掉任何一个括号都可能引入隐蔽的优先级 bug。

多行宏

概念：当宏的替换文本太长时，可以用反斜杠 `\` 续行，把一个宏写在多行上。注意 `\` 必须是该行的最后一个字符，后面不能有空格。

要点：

- 每行（除最后一行）末尾加 `\` 表示续行。
- 整个宏实际上仍是一条逻辑指令。
- 复杂多行宏建议改用 `do { ... } while(0)` 包裹，使其能像单条语句一样使用。

示例：

```
#include <iostream>
// 多行宏 + do-while(0) 包裹，保证宏能安全地作为"一条语句"使用
#define LOG_AND_INC(x) do { \
    std::cout << "before: " << (x) << "\n"; \
    ++(x); \
    std::cout << "after:  " << (x) << "\n"; \
} while (0)

int main() {
    int n = 5;
    if (n > 0)
        LOG_AND_INC(n);    // 展开后是完整的一条语句，安全
    return 0;
}
```

易错点/注意：反斜杠后若有空格会导致续行失效；多语句宏若不用 `do-while(0)` 包裹，在 `if/else` 无花括号时会匹配错位。

与 ## 运算符

概念：`#`（字符串化运算符）把宏参数转换为字符串字面量；`##`（连接运算符）把两个记号拼接成一个新记号。二者是函数宏专属能力。

要点：

- `#x` 把参数 `x` 变成 `"x"`（带引号的字符串）。
- `a ## b` 把 `a` 与 `b` 拼接为一个标识符，可用于生成函数名、变量名。
- 拼接结果需是合法记号，否则编译报错。

示例：

```
#include <iostream>
#define STR(x) #x           // 字符串化
#define CONCAT(a,b) a##b    // 记号连接
#define DEBUG_VAR(name) std::cout << #name << " = " << (name) << "\n"

int main() {
    int xy = 42;
    std::cout << STR(hello world) << "\n"; // 输出 "hello world"
    std::cout << CONCAT(x, y) << "\n";     // 展开为 xy, 即输出 42
    DEBUG_VAR(xy);                         // 输出 "xy = 42"
    return 0;
}
```

易错点/注意：## 拼接出的必须是合法标识符或记号；字符串化不会展开嵌套宏参数（若需先展开再字符串化，要用两层宏间接实现）。

可变参数宏

概念：可变参数宏用 ... 和 __VA_ARGS__ 接收任意数量的参数，常用于日志、调试打印等场景。

要点：

- 语法：#define LOG(...) printf(__VA_ARGS__)。
- __VA_ARGS__ 代表所有可变参数。
- ##__VA_ARGS__ 是 GNU 扩展，用于在可变参数为空时吞掉前面的逗号。
- 也可给可变参数命名：#define LOG(fmt, args...) ... (GNU 风格) 或 ... + 具名参数。

示例：

```
#include <cstdio>
#define LOG(fmt, ...) std::printf("[LOG] " fmt "\n", ##__VA_ARGS__)
// ##__VA_ARGS__: 当无可变参数时，自动去掉前面多余的逗号

int main() {
    LOG("启动"); // 无可变参数，正常
    LOG("x=%d y=%d", 1, 2); // 有可变参数，正常
    return 0;
}
```

易错点/注意：##__VA_ARGS__ 并非标准 C++ 写法（GCC/Clang 扩展，MSVC 亦支持）；若追求严格标准可写两个宏分别处理有无参数的情况。

宏的优缺点

与 constexpr / inline / using 的对比

概念：宏诞生于早期 C 时代，能做的事（常量、函数替代、类型别名）如今几乎都能被现代 C++ 的 constexpr、inline、using 等机制更安全地替代。理解宏的缺点有助于在工程中克制地使用它。

要点：

- 宏的**优点**：可用于条件编译、#/## 字符串化与连接、与具体类型无关的文本生成、访问 __FILE__ / __LINE__。
- 宏的**缺点**：无类型检查、无作用域、参数多次求值、难以调试、命名污染。
- constexpr 常量有类型且可参与重载与模板推导。
- inline / 模板函数有类型检查且参数只求值一次。

- `using` 别名更清晰，且能配合模板。

示例：

```
#include <iostream>
#define MAX_MACRO(a,b) ((a)>(b)?(a):(b))    // 宏：无类型检查

// constexpr 函数：类型安全、可编译期求值
constexpr int max_int(int a, int b) { return a > b ? a : b; }

// 函数模板：对任意可比较类型通用
template <typename T>
constexpr T max_t(T a, T b) { return a > b ? a : b; }

// using 别名替代 typedef，更清晰且支持模板
template <typename T>
using Vec = std::vector<T>;

int main() {
    std::cout << max_int(3, 4) << "\n";        // 7 -> 4
    std::cout << max_t(2.5, 1.5) << "\n";      // 2.5
    Vec<int> v{1, 2, 3};
    std::cout << v.size() << "\n";
    return 0;
}
```

易错点/注意：宏唯一的“不可替代”领域是预处理阶段能力（条件编译、字符串化、`__LINE__` 等）；其余场景一律优先现代特性。

宏的坑

概念：宏除了多次求值、优先级问题外，还有若干经典陷阱，如宏名与函数/变量重名、宏展开的“粘性”、以及宏不遵守作用域导致的意外替换。

要点：

- 宏是全局的，可能替换掉无关代码中的同名标识符。
- 宏展开后可能产生语法错误或语义变化。
- 定义宏时避免与标准库、第三方库中的名字冲突（常见做法是全大写 + 前缀）。
- 宏无法参与重载、命名空间、模板等语言机制。

示例：

```
#include <iostream>
#define min(a,b) ((a)<(b)?(a):(b))    // 灾难：覆盖了 std::min 语义

// 使用 std:: 前缀仍可能被宏污染：
// std::min(1,2) 若写成 min(1,2) 会被宏展开

int main() {
    int a = 1, b = 2;
    std::cout << min(a, b) << "\n";    // 调用了宏，而非 std::min
    // 解除宏定义后可恢复：
    #undef min
    std::cout << std::min(a, b) << "\n"; // 现在才是真正的 std::min
    return 0;
}
```

易错点/注意：不要在头文件中定义通用短名宏（如 `min`、`max`、`ERROR`），否则会污染所有包含者；确需宏时用项目前缀。

条件编译

`#if` / `#ifdef` / `#ifndef` / `#elif` / `#else` / `#endif`

概念：条件编译让编译器只保留满足条件的代码分支，用于跨平台代码、调试开关、特性开关等。`#if` 判断常量表达式，`#ifdef` / `#ifndef` 判断宏是否已定义。

要点：

- `#if 表达式`：表达式为真则保留；表达式必须是预处理期可求值的常量。
- `#ifdef NAME` 等价于 `#if defined(NAME)`；`#ifndef NAME` 等价于 `#if !defined(NAME)`。
- `#elif`、`#else` 与运行时 if-else 结构类似。
- 每个条件块必须以 `#endif` 结束。

示例：

```
#include <iostream>
#define VERSION 2

int main() {
    #if VERSION == 1
        std::cout << "版本 1 功能\n";
    #elif VERSION == 2
        std::cout << "版本 2 功能\n";    // 只编译这一条
    #else
        std::cout << "未知版本\n";
    #endif
    return 0;
}
```

易错点/注意：`#if` 里不能使用运行时变量（如函数参数）；未定义的标识符在 `#if` 中被当作 0 处理，但需谨慎，容易误判。

`#if defined` 用法与调试开关

概念：`defined` 运算符在预处理期判断某个宏是否已定义，可配合 `&&`、`||`、`!` 组合复杂条件，是编写调试开关和特性检测的标准手法。

要点：

- `#if defined(DEBUG)` 比 `#ifdef DEBUG` 更灵活，可与逻辑运算符组合。
- 常见调试开关：`#if defined(DEBUG) && !defined(NDEBUG)`。
- 编译时用 `-DDEBUG` 定义宏开启调试代码。
- `NDEBUG` 是标准规定的宏，定义后 `assert` 失效。

示例：

```

#include <iostream>
#include <cassert>
// 编译命令: g++ -std=c++17 -DDEBUG main.cpp
// 不带 -DDEBUG 时, 调试日志完全不参与编译

int main() {
    int x = 10;
    #if defined(DEBUG) && !defined(NDEBUG)
        std::cout << "[调试] x = " << x << "\n";    // 仅在 DEBUG 下编译
    #endif
    assert(x > 0);                                // 定义 NDEBUG 后失效
    return 0;
}

```

易错点/注意: `defined` 只能用于 `#if/#elif`, 不能出现在宏展开或普通代码中; 忘记 `-D` 时调试代码会被静默排除。

其他指令

#undef、#error、#warning、#line、#pragma

概念: 这些是预处理器的辅助指令: `#undef` 取消宏定义; `#error` 强制编译报错; `#warning` 给出编译警告 (非标准但被广泛支持); `#line` 修改行号与文件名; `#pragma` 触发编译器特定行为。

要点:

- `#undef NAME` 取消宏, 之后 `NAME` 不再被替换。
- `#error "message"` 在预处理阶段终止编译, 常用于平台不支持时提示。
- `#warning "message"` 产生警告但不终止 (GCC/Clang 支持)。
- `#line 100 "file.cpp"` 改变后续代码的 `__LINE__` 和 `__FILE__` 起始值。
- `#pragma` 种类繁多, 如 `#pragma pack`、`#pragma message`。

示例:

```

#include <iostream>
#define FEATURE_X 1
#undef FEATURE_X    // 取消定义

#if !defined(FEATURE_X)
#warning "FEATURE_X 未定义, 将使用回退实现"    // 编译警告
#endif

#if defined(WIN32)
#error "当前平台不支持 WIN32"                // 直接报错终止
#endif

int main() {
    std::cout << "line=" << __LINE__ << "\n";    // 正常行号
    #line 500 "virtual.cpp"
    std::cout << "line=" << __LINE__ << " file=" << __FILE__ << "\n"; // 行号被改
    写
    return 0;
}

```

易错点/注意：`#error` 会导致编译立即失败，只用于"必须停止"的硬约束；`#line` 常用于代码生成工具，手写代码几乎用不到。

预定义宏

FILE、LINE、FUNCTION、__cplusplus

概念：编译器预定义了若干宏用于获取当前位置信息：`__FILE__`（当前文件名）、`__LINE__`（当前行号）、`__FUNCTION__`（当前函数名）、`__cplusplus`（C++ 标准版本号）。

要点：

- `__FILE__` 和 `__LINE__` 是标准宏，`__FUNCTION__` 是编译器扩展（等价于标准的 `__func__`）。
- `__cplusplus` 取值：199711L(C++98)、201103L(C++11)、201402L(C++14)、201703L(C++17)、202002L(C++20)、202302L(C++23)。
- 常用来做日志、断言、条件编译（按标准版本启用特性）。
- 预定义宏由编译器提供，无法用 `#undef` 取消（实际上某些编译器允许，但不应依赖）。

示例：

```
#include <iostream>
void report() {
    // __FUNCTION__ 是编译器扩展，等价的标准写法是 __func__
    std::cout << "函数: " << __FUNCTION__ << "\n";
    std::cout << "文件: " << __FILE__ << "\n";
    std::cout << "行号: " << __LINE__ << "\n";
}

int main() {
    report();
    #if __cplusplus >= 201703L
        std::cout << "C++17 或更高\n";        // 按标准版本条件编译
    #else
        std::cout << "低于 C++17\n";
    #endif
    return 0;
}
```

易错点/注意：`__FUNCTION__` 不是 C++ 标准的一部分（标准只有 `__func__`，它是变量而非宏，不能用于字符串拼接）；判断 `__cplusplus` 时应使用 `>=` 比较而非 `==`。

命名空间

定义与嵌套

概念：命名空间（namespace）用于把全局作用域中的名字分组，避免不同库之间的命名冲突。命名空间可以嵌套，可以用 `::` 逐级访问。

要点：

- 定义：`namespace name { ... }`。
- 嵌套：`namespace A { namespace B { ... } }`，C++17 可写作 `namespace A::B { ... }`。
- 使用：`A::B::func()` 全限定名访问。
- 同一命名空间可分多次打开定义（跨文件追加）。

示例：

```
#include <iostream>
namespace A {
    int value = 1;
    void f() { std::cout << "A::f\n"; }
    namespace B {                // 嵌套命名空间
        int value = 2;
    }
}
// C++17 起可写成 namespace A::B { ... } 直接嵌套定义

int main() {
    std::cout << A::value << "\n";    // 1
    std::cout << A::B::value << "\n"; // 2
    A::f();
    return 0;
}
```

易错点/注意：命名空间内定义的函数若在外实现，必须用 `返回类型 命名空间::函数名(...)` 的形式，且声明与定义要在同一命名空间。

using 声明 vs using 指令

概念：`using 声明` 把单个名字引入当前作用域；`using 指令` 把整个命名空间的所有名字引入当前作用域。二者粒度和风险完全不同。

要点：

- `using 声明`：`using std::cout;`，只引入 `cout`。
- `using 指令`：`using namespace std;`，引入 `std` 下全部名字。
- 声明比指令更安全，污染面小。
- 头文件中**严禁**使用 `using namespace` 指令，会污染所有包含者。

示例：

```
#include <iostream>
// using 声明：只引入需要的名字
using std::cout;
using std::endl;

// using 指令：引入整个 std（不推荐放头文件、尽量少用）
// using namespace std;

int main() {
    cout << "hello" << endl;    // 因 using 声明，可直接写 cout/endl
    return 0;
}
```

易错点/注意：`using namespace std;` 在大型项目中易造成名字冲突与可读性下降；在源文件内部可用，头文件里坚决不用。

匿名命名空间（取代 static 全局）

概念：匿名命名空间 `namespace { ... }` 使内部名字具有**内部链接**，即仅在当前翻译单元可见，等价于 C 时代的 `static` 全局变量/函数，且是 C++ 推荐的写法。

要点：

- 语法：`namespace { int x; }`，名字无需命名。
- 内部名字相当于该翻译单元私有，其它 .cpp 文件无法引用。
- 取代旧的 `static` 全局用法。
- 匿名命名空间内可以定义类、函数、变量。

示例：

```
#include <iostream>
namespace {
    const int kInternal = 100;    // 匿名命名空间
    void helper() {               // 仅本文件可见，内部链接
        std::cout << "内部函数\n"; // 本文件私有函数
    }
}

int main() {
    helper();
    std::cout << kInternal << "\n";
    return 0;
}
```

易错点/注意：匿名命名空间的名字在多个 .cpp 中各有一份独立实体，不会冲突也不会共享；不要再额外用 `static` 修饰其成员（C++ 已不推荐 `static` 全局）。

内联命名空间 (C++11)

概念：内联命名空间（inline namespace）中的名字会被“提升”到外层命名空间，外层可直接访问，主要用于版本化：库升级时保留旧版本符号兼容 ABI。

要点：

- 语法：`inline namespace v1 { ... }`。
- 外层命名空间可无前缀访问内联命名空间的成员。
- 常用于库的版本管理，例如 `namespace std { inline namespace __1 { ... } }`。
- 多个内联命名空间可同时存在，但同名符号可能产生二义性。

示例：

```
#include <iostream>
namespace Lib {
    inline namespace v2 {           // 内联命名空间：默认版本
        void run() { std::cout << "v2 实现\n"; }
    }
    namespace v1 {                  // 普通命名空间：旧版本
        void run() { std::cout << "v1 实现\n"; }
    }
}

int main() {
    Lib::run();                     // 直接访问到内联的 v2
}
```

```
Lib::v1::run();    // 显式访问旧版本
return 0;
}
```

易错点/注意：内联命名空间主要服务于库的 ABI 版本管理，普通业务代码较少用到；升级内联版本时旧符号路径仍可用，便于二进制兼容。

命名空间别名

概念：命名空间别名给一个很长的嵌套命名空间起一个短名字，方便书写，语法为 `namespace 短名 = 长名;`。

要点：

- 语法： `namespace fs = std::filesystem;`。
- 只创建别名，不复制内容。
- 可嵌套命名空间别名，也可给别名再起别名。
- 常用于简化深层嵌套或第三方库命名空间。

示例：

```
#include <iostream>
namespace Very { namespace Long { namespace Deep {
    void hello() { std::cout << "hello deep\n"; }
}}}
namespace Short = Very::Long::Deep;    // 命名空间别名

int main() {
    Short::hello();                    // 等价于 Very::Long::Deep::hello()
    return 0;
}
```

易错点/注意：别名在声明点之后才生效；别名不能与已有名字冲突。

头文件与源文件组织

声明与定义分离

概念：C++ 传统工程把**声明**放在头文件（.h/.hpp），把**定义**放在源文件（.cpp），实现接口与实现分离，支持多文件编译和增量构建。

要点：

- 头文件放：类声明、函数声明、常量声明、模板（必须）、内联函数。
- 源文件放：非内联函数定义、静态变量定义。
- 头文件必须加 include guard。
- 一个头文件只声明一个类/一组相关功能，职责单一。

示例：

```
// shape.h — 只有声明
#ifndef SHAPE_H
#define SHAPE_H
class Shape {
public:
    double area() const;    // 声明，不定义
};
```

```
#endif

// shape.cpp -- 定义
#include "shape.h"
double Shape::area() const {
    return 0.0;          // 定义
}
```

易错点/注意：非内联函数若定义在头文件且被多个 .cpp 包含，会导致**重复定义**链接错误；普通函数定义应放 .cpp。

类模板必须放头文件

概念：类模板（和函数模板）在实例化时编译器需要看到**完整的模板定义**，因此模板的声明与实现通常必须放在头文件中（或通过 .tpp 文件被头文件包含）。

要点：

- 模板不是真正的代码，实例化时才生成具体类型版本。
- 编译器在编译使用处需要模板全部定义。
- 常见做法：模板类声明+成员函数实现都写在头文件。
- 也可用"头文件 + 末尾 include .tpp"的方式分离，但 .tpp 本质仍被包含进头文件。

示例：

```
// mypair.h -- 模板必须完整放在头文件
#ifndef MYPAIR_H
#define MYPAIR_H
template <typename T>
class MyPair {
    T a_, b_;
public:
    MyPair(T a, T b) : a_(a), b_(b) {}    // 定义必须在此可见
    T sum() const { return a_ + b_; }
};
#endif

// main.cpp
#include <iostream>
#include "mypair.h"
int main() {
    MyPair<int> p(3, 4);
    std::cout << p.sum() << "\n";    // 7
    return 0;
}
```

易错点/注意：把模板成员函数实现单独放 .cpp 会导致**未定义引用**错误（除非显式实例化 `template class MyPair<int>;`）。

inline 函数/变量与 ODR 一次定义规则

概念：ODR（One Definition Rule，一次定义规则）要求每个非内联实体在程序中只能有一个定义。`inline` 函数/变量是例外，它们可出现在多个翻译单元而只保留一份，因此可以安全地定义在头文件中。

要点：

- ODR: 非 inline 函数/变量全程序只能定义一次。
- `inline` 函数: 允许在每个使用它的翻译单元各有一份相同定义。
- `inline` 变量 (C++17): 可在头文件中定义而不会重复定义。
- 类内定义的成员函数自动隐式 `inline`。
- `constexpr` 变量/函数也隐式 `inline` (C++17 起 `constexpr` 变量隐式 `inline`) 。

示例:

```
// util.h
#ifndef UTIL_H
#define UTIL_H
inline int add(int a, int b) { return a + b; } // inline 函数可放头文件
inline int g_counter = 0; // C++17 inline 变量, 全程序共享一份
constexpr int kMax = 100; // constexpr 变量隐式 inline
#endif

// main.cpp 与 a.cpp 都 include "util.h", 不会出现重复定义
```

易错点/注意: `inline` 函数在多个翻译单元中定义必须**文本相同**, 否则是未定义行为; 普通全局变量想跨文件共享要用 `extern` 声明, 而非直接在头文件定义。

static 全局变量的内部链接

概念: 用 `static` 修饰的全局变量/函数具有**内部链接**, 只在当前翻译单元可见, 每个 .cpp 各有一份独立实体, 与匿名命名空间效果相同。

要点:

- `static` 全局变量: 内部链接, 文件私有。
- `static` 函数: 内部链接, 仅本文件可调用。
- 现代 C++ 推荐用匿名命名空间替代 `static` 全局。
- `static` 类成员含义不同 (属于类而非对象), 注意区分。

示例:

```
// a.cpp
static int s_counter = 0; // 仅 a.cpp 可见, 内部链接
static void reset() { s_counter = 0; } // 内部链接函数

// b.cpp 中同名 static 变量是另一个独立实体, 互不影响
// static int s_counter = 100; // 合法, 与 a.cpp 的互不冲突
```

易错点/注意: `static` 在类内表示静态成员 (类级别共享), 在全局/函数外表示内部链接, 在函数内表示局部静态变量 (生命周期跨调用), 三种含义要区分清楚。

链接基础

extern 与跨文件共享变量

概念: `extern` 用于声明一个"在别处定义"的变量或函数, 使多个翻译单元可以共享同一个实体。
`extern` 是声明而非定义, 不分配存储。

要点:

- `extern int g_x;` 声明外部变量, 定义在另一个 .cpp 中。

- 函数默认具有外部链接，跨文件调用无需额外 `extern`（但显式写 `extern` 也可）。
- `extern` 变量必须在某处有一次定义（不含 `extern` 的初始化）。
- C++17 起可用 `inline` 变量直接放头文件实现共享。

示例：

```
// globals.h
#ifndef GLOBALS_H
#define GLOBALS_H
extern int g_counter;    // 声明：定义在 globals.cpp
void increment();        // 函数声明，默认外部链接
#endif

// globals.cpp
#include "globals.h"
int g_counter = 0;        // 定义（唯一一次）
void increment() { ++g_counter; }

// main.cpp
#include <iostream>
#include "globals.h"
int main() {
    increment();
    std::cout << g_counter << "\n";    // 1, 跨文件共享同一变量
    return 0;
}
```

易错点/注意：`extern` 声明不能带初始化（否则变成定义）；若在头文件写 `int g_counter = 0;` 且被多个 .cpp 包含，会触发重复定义链接错误。

extern "C" 与 C 互操作

概念：C++ 会对函数名进行**名称修饰 (name mangling)**，导致与 C 库的函数符号不匹配。`extern "C"` 告诉编译器按 C 的规则生成符号，实现 C/C++ 互操作。

要点：

- `extern "C"` 关闭名称修饰，符号名保持 C 风格。
- 用 `#ifdef __cplusplus` 包裹，使头文件同时兼容 C 和 C++ 编译器。
- 可以修饰单个函数或整个代码块。
- 重载函数不能同时使用 `extern "C"`（C 不支持重载）。

示例：

```
// c_api.h -- 同时兼容 C 与 C++
#ifndef C_API_H
#define C_API_H
#ifdef __cplusplus
extern "C" {                // C++ 编译时关闭名称修饰
#endif
    int c_add(int a, int b);
#ifdef __cplusplus
}
#endif
#endif
```

```
// c_api.cpp -- 实现
#include "c_api.h"
int c_add(int a, int b) { return a + b; }

// main.cpp
#include <iostream>
#include "c_api.h"
int main() {
    std::cout << c_add(3, 4) << "\n"; // 7, 可链接到 c 实现
    return 0;
}
```

易错点/注意：`extern "C"` 只影响符号命名，不影响语言语义；两个重载函数都标记 `extern "C"` 会因符号同名而冲突。

静态库/动态库概念与名称修饰简介

概念：静态库（.a/.lib）在链接期把目标代码拷贝进可执行文件；动态库（.so/.dll/.dylib）在运行期加载、多个程序共享。名称修饰（name mangling）是 C++ 为支持重载、命名空间、模板而生成的唯一符号名。

要点：

- 静态库：链接期合并，产物更大，部署简单。
- 动态库：运行期加载，可共享、可热更新，但有版本依赖问题。
- 名称修饰：`void f(int)` 与 `void f(double)` 生成不同符号，编码了函数签名。
- 修饰规则因编译器而异，导致不同编译器/版本的 C++ 库二进制不兼容（ABI 问题）。

示例：

```
// 函数重载依赖名称修饰：两个 f 生成不同符号
void f(int) {}
void f(double) {}
namespace ns { void g() {} } // 命名空间也参与符号名编码

// 查看符号：g++ -c demo.cpp && nm -C demo.o
// 可看到类似 _Z1fi、_Z1fd、_ZN2ns1gEv 的被修饰符号
```

易错点/注意：链接 C++ 库时必须保证库与程序用相同编译器、相同标准库版本（ABI 兼容），否则会出现 undefined symbol 或崩溃；跨语言调用 C 接口必须用 `extern "C"`。

编译与调试实践

g++ 常用编译选项

概念：g++ 是 GNU 的 C++ 编译器，通过命令行选项控制语言标准、优化级别、调试信息和警告策略。掌握常用选项是工程实践的基本功。

要点：

- `-std=c++17`：指定语言标准（c++11/14/17/20/23）。
- `-O0`（默认，不优化，便于调试）、`-O2`（常规优化）、`-O3`（激进优化）。
- `-g`：生成调试信息，供 gdb 使用。
- `-Wall -Wextra`：开启常用/额外警告；`-Werror`：把警告当错误。
- `-fsanitize=address,undefined`：开启 ASan/UBSan 内存与未定义行为检测。
- `-D` 定义宏、`-I` 指定头文件路径、`-c` 只编译、`-o` 指定输出。

示例：

```
# 单文件直接编译
g++ -std=c++17 -O2 -Wall -Wextra -o app main.cpp

# 调试构建：关优化 + 调试信息 + 消毒器
g++ -std=c++17 -O0 -g -fsanitize=address,undefined -o app_debug main.cpp

# 分步编译 + 指定头文件路径 + 定义宏
g++ -std=c++17 -c -Iinclude -DDEBUG -o main.o main.cpp
g++ -o app main.o
```

易错点/注意：`-O3` 可能触发激进优化导致依赖未定义行为的代码出问题；`-fsanitize` 与 `-O2` 可搭配，但通常调试时用 `-O0/-O1` 便于定位；发布构建记得去掉 `-g` 或用 `-g` 配合符号剥离。

gdb 基础命令

概念：gdb 是 GNU 调试器，用于单步执行、设置断点、查看变量和调用栈。调试程序需用 `-g` 编译。

要点：

- `run`：启动程序。
- `break 文件名:行号` 或 `break 函数名`：设断点。
- `next`（下一行，不进入函数）、`step`（进入函数）。
- `print 表达式`：打印变量值。
- `backtrace`：查看调用栈。
- `continue`：继续运行到下一断点。

示例：

```
g++ -std=c++17 -O0 -g -o demo demo.cpp
gdb ./demo
# (gdb) break main           # 在 main 处设断点
# (gdb) run                  # 运行
# (gdb) next                 # 单步，不进入函数
# (gdb) step                 # 单步，进入函数
# (gdb) print x              # 打印变量 x
# (gdb) backtrace            # 查看调用栈
# (gdb) quit                 # 退出
```

易错点/注意：`next` 与 `step` 的区别是面试常考：`next` 把函数调用当作一步执行完，`step` 会进入被调函数内部；调试前必须用 `-g`，否则没有符号信息。

CMake 基础

最小 CMakeLists.txt 与常用指令

概念：CMake 是跨平台构建系统生成器，通过 `CMakeLists.txt` 描述项目结构，生成 Makefile、VS 工程等。最小项目只需声明 cmake 最低版本、项目名和可执行目标。

要点：

- `cmake_minimum_required(VERSION 3.15)`：声明最低 CMake 版本。
- `project(MyApp)`：定义项目名。
- `add_executable(app main.cpp)`：生成可执行目标。

- `add_library(mylib STATIC src.cpp)`: 生成静态库; `SHARED` 为动态库。
- `target_include_directories(app PUBLIC include)`: 给目标加头文件路径。
- `set(CMAKE_CXX_STANDARD 17)`: 设置 C++ 标准。

示例:

```
cmake_minimum_required(VERSION 3.15)
project(MyApp CXX)

set(CMAKE_CXX_STANDARD 17)           # 使用 C++17
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_library(mylib STATIC src/lib.cpp) # 静态库
target_include_directories(mylib PUBLIC include) # 库的头文件路径

add_executable(app src/main.cpp)      # 可执行文件
target_include_directories(app PRIVATE include)
target_link_libraries(app PRIVATE mylib) # 链接静态库
```

易错点/注意: `target_include_directories` 的 `PUBLIC/PRIVATE/INTERFACE` 决定了头文件路径是否传递给依赖者; 用 `target_*` 而非全局 `include_directories` 是现代推荐做法。

find_package 与常见构建流程

概念: `find_package` 用于查找并引入第三方库 (如 Boost、OpenCV、Threads), 找到后会提供相应的导入目标供 `target_link_libraries` 使用。

要点:

- `find_package(Package REQUIRED)`: 查找库, `REQUIRED` 表示找不到就报错。
- 找到后通常可用 `Package::xxx` 导入目标链接。
- 常见构建流程: `cmake -B build` 配置, `cmake --build build` 构建。
- 可用 `-DCMAKE_BUILD_TYPE=Release/Debug` 指定构建类型。

示例:

```
cmake_minimum_required(VERSION 3.15)
project(Demo CXX)
set(CMAKE_CXX_STANDARD 17)

find_package(Threads REQUIRED) # 查找线程库

add_executable(demo main.cpp)
target_link_libraries(demo PRIVATE Threads::Threads) # 链接导入目标
```

```
# 常见构建流程 (out-of-source 构建)
cmake -B build -DCMAKE_BUILD_TYPE=Release # 配置到 build 目录
cmake --build build                       # 编译
./build/demo                             # 运行
```

易错点/注意: `find_package` 依赖库提供的 `*-config.cmake` 或 `Find*.cmake` 模块; 第三方库需先安装到系统或通过 `CMAKE_PREFIX_PATH` 指定路径。

代码风格与命名规范建议

命名、缩进与注释规范

概念：统一的代码风格能显著提升可读性和维护性。核心原则：命名表达意图、风格全项目一致、注释解释"为什么"而非"做什么"。

要点：

- 类名：PascalCase（如 HttpClient）；函数/变量：camelCase 或 snake_case（全项目统一）。
- 常量：k 前缀或全大写；宏：全大写加下划线。
- 成员变量：加后缀 _ 或前缀 m_，便于区分局部变量。
- 缩进：4 空格（或 2 空格），不用 Tab 混用。
- 用 clang-format 统一格式化。

示例：

```
#include <string>
class HttpClient {                // 类名 PascalCase
public:
    void sendRequest(const std::string& url); // 函数 camelCase
private:
    std::string base_url_;         // 成员变量带下划线后缀
};

constexpr int kMaxRetries = 3;    // 常量 k 前缀
#define HTTP_TIMEOUT_MS 5000      // 宏全大写

// 好注释：解释为什么（而非复述代码）
// 超时设为 5s，因为上游网关在高峰期最长响应约 3s
```

易错点/注意：风格本身无绝对对错，关键是一致性；不要混用 Tab 和空格，提交前用 clang-format 或项目配置统一处理。

本部分小结

易忘点	说明
预处理是纯文本替换	不理解类型，不做语法检查
函数宏参数加括号 + 整体加括号	否则优先级出错；副作用参数会被多次求值
# 字符串化、## 记号连接	仅函数宏可用
#ifdef x == #if defined(x)	defined 可组合逻辑运算
__cplusplus 取值	201703L(C++17)、202002L(C++20)，判断用 >=
using 声明 vs 指令	声明只引入单个名字，指令引入全部，头文件禁用指令
匿名命名空间	内部链接，取代 static 全局
类模板必须放头文件	实例化需要完整定义，否则未定义引用
inline 函数/变量	可跨翻译单元各有一份定义，违反 ODR 之外
extern "C"	关闭名称修饰，用于 C 互操作
g++ 调试选项	-g、-O0、-fsanitize=address,undefined、-Wall -Wextra -Werror
CMake 构建流程	cmake -B build 配置，cmake --build build 构建

- 核心心法：能用语言特性就别用宏（constexpr/inline/using 更安全）；头文件只放声明、模板和内联；命名空间 + 规范风格是大型工程的基石；g++/gdb/CMake 是日常三件套。

第十二部分：常见陷阱、性能优化与面试高频题

常见陷阱清单

数组越界与未定义行为

概念：访问数组下标超出 `[0, size-1]` 范围是**未定义行为 (UB)**，可能崩溃、可能读到脏数据、也可能“看起来正常”，这正是其危险之处。

要点：

- C 风格数组和 `std::vector` 的 `operator[]` 都不做越界检查。
- `std::vector::at()` 会抛 `std::out_of_range` 异常，可作安全替代。
- 越界是 UB，编译器可能据此做意外优化。

示例：

```
#include <iostream>
#include <vector>
int main() {
    int a[3] = {1, 2, 3};
    // 错误: a[3] 越界，未定义行为
    // std::cout << a[3] << "\n";    // UB!
```

```

// 修正：用 at() 或确保下标合法
std::vector<int> v{1, 2, 3};
try {
    std::cout << v.at(3) << "\n"; // 抛异常，可捕获
} catch (const std::out_of_range& e) {
    std::cout << "越界： " << e.what() << "\n";
}
return 0;
}

```

易错点/注意：面试/工程中尽量用 `at()` 或 range-for 遍历；用 `operator[]` 时必须自己保证下标合法。

空指针解引用

概念：对 `nullptr` 或悬垂指针解引用是未定义行为。C++ 中 `nullptr` (C++11) 是空指针字面量，比旧的 `NULL` / `0` 类型更安全。

要点：

- 解引用空指针：`*p` 或 `p->member`，其中 `p == nullptr`，是 UB。
- 使用前判空，或用引用（引用不能为空）规避。
- 智能指针 `std::unique_ptr` / `std::shared_ptr` 提供所有权语义，但仍需判空。

示例：

```

#include <iostream>
#include <memory>
void print(int* p) {
    if (p) { // 修正：先判空
        std::cout << *p << "\n";
    } else {
        std::cout << "空指针\n";
    }
}
int main() {
    int* p = nullptr;
    // std::cout << *p << "\n"; // 错误：解引用空指针，崩溃/UB
    print(p); // 正确
    std::unique_ptr<int> up; // 智能指针默认为空
    if (up) std::cout << *up << "\n";
    return 0;
}

```

易错点/注意：`delete` 后把指针置 `nullptr` 可减少悬垂误用；优先用智能指针彻底避免手动 `delete`。

有符号整数溢出与类型提升

概念：有符号整数溢出（如 `INT_MAX + 1`）是未定义行为；而整数运算前的小类型会先被**整型提升**为 `int`，可能导致预期外的结果。

要点：

- `char`、`short` 参与运算时先提升为 `int`。
- 有符号溢出是 UB，无符号溢出则按模回绕（定义良好）。
- 大循环、累加、乘法要警惕溢出。

示例：

```
#include <iostream>
#include <limits>
int main() {
    int x = std::numeric_limits<int>::max();
    // std::cout << x + 1 << "\n";      // 错误：有符号溢出，UB

    unsigned int u = std::numeric_limits<unsigned>::max();
    std::cout << u + 1 << "\n";        // 0，无符号按模回绕，定义良好

    // 类型提升示例：两个 char 相加结果类型是 int
    char a = 100, b = 100;
    auto c = a + b;                    // c 是 int, 值 200
    std::cout << sizeof(c) << " " << c << "\n";
    return 0;
}
```

易错点/注意：判断 `a < b + c` 等混合类型比较时留意隐式转换；需要可靠溢出检测可用编译器内置 `__builtin_add_overflow`。

浮点数比较 (== 的坑、epsilon)

概念：浮点数以二进制近似存储，很多十进制小数（如 0.1）无法精确表示，直接 `==` 比较会因舍入误差失败。应使用误差容限（epsilon）比较。

要点：

- `0.1 + 0.2 != 0.3` 在 double 下成立（不等于）。
- 用 `std::abs(a - b) < epsilon` 判断近似相等。
- epsilon 选取要相对数值量级：绝对值比较只适合接近 0 的数。
- 大数比较用相对误差更稳妥。

示例：

```
#include <iostream>
#include <cmath>
bool nearly_equal(double a, double b, double eps = 1e-9) {
    return std::fabs(a - b) < eps;
}
int main() {
    double x = 0.1 + 0.2;
    // std::cout << (x == 0.3) << "\n";    // 错误：输出 0(false)
    std::cout << nearly_equal(x, 0.3) << "\n"; // 1(true)
    return 0;
}
```

易错点/注意：循环里用浮点做计数器（`for(double i=0; i<1; i+=0.1)`）会累积误差，改用整数计数再换算。

迭代器失效后继续使用

概念： `vector` 等容器在 `push_back`（触发扩容）、`erase`、`insert` 等操作后，之前的迭代器可能失效，继续使用是未定义行为。

要点：

- `vector::push_back` 扩容会使所有迭代器/指针/引用失效。
- `erase` 使被删元素及之后（vector）的迭代器失效。
- `erase` 返回指向被删元素之后的有效迭代器。
- 删除前 `reserve` 充足容量可避免扩容失效。

示例：

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v{1, 2, 3, 4, 5};
    // 修正：用 erase 返回的新迭代器
    for (auto it = v.begin(); it != v.end(); ) {
        if (*it % 2 == 0) {
            it = v.erase(it);    // erase 返回下一个有效迭代器
        } else {
            ++it;
        }
    }
    for (int n : v) std::cout << n << " ";    // 1 3 5
    std::cout << "\n";
    return 0;
}
```

易错点/注意： 保存 `v.end()` 后插入元素再比较会导致失效；删除/插入后重新获取 `end()`，不要复用旧迭代器。

返回局部变量引用/指针（悬垂）

概念： 函数返回指向局部变量的指针或引用，函数结束后局部变量销毁，返回值变成**悬垂引用/指针**，使用它即未定义行为。

要点：

- 局部变量在栈上，函数返回后生命周期结束。
- 返回 `int&`、`int*` 指向局部变量是经典错误。
- 可返回值（拷贝）或返回有所有权的对象（`std::string`、智能指针）。
- `static` 局部变量生命周期跨调用，可安全返回引用。

示例：

```
#include <iostream>
#include <string>
// 错误：返回局部变量的引用，悬垂
// int& bad() { int x = 10; return x; }

// 正确：按值返回，发生拷贝/移动
std::string good() {
    std::string s = "hello";
    return s;    // 移动或 NRVO 优化，安全
}
```

```

}
int main() {
    std::cout << good() << "\n";
    return 0;
}

```

易错点/注意：`std::string` 等容器按值返回是安全的（有移动语义/NRVO）；但裸指针、引用返回局部对象永远是错误。

静态初始化顺序问题

概念：不同翻译单元中的全局对象初始化顺序是未定义的（静态初始化顺序 fiasco），一个全局对象若在构造中访问另一个尚未初始化的全局对象，会读到未定义值。

要点：

- 跨 .cpp 的全局对象构造顺序不确定。
- 用**函数内局部 static**（Meyers 单例）延迟初始化，保证首次使用时构造。
- C++11 起局部 static 初始化是线程安全的。
- 避免全局对象之间相互依赖。

示例：

```

#include <iostream>
#include <string>
// 正确：Meyers 单例，延迟且线程安全初始化
std::string& config() {
    static std::string s = "loaded"; // 首次调用才初始化
    return s;
}
// 错误示范（多文件场景）：a.cpp 全局对象 A 构造时调用 b.cpp 的 B，
// 若 B 尚未初始化，读取到空/未定义状态 — 即静态初始化顺序问题。

int main() {
    std::cout << config() << "\n"; // loaded, 安全
    return 0;
}

```

易错点/注意：把全局对象改为“函数返回静态局部对象的引用”是最常用的解法；同时减少全局可变状态。

double 精度与金额处理

概念：`double` 是二进制浮点，无法精确表示所有十进制小数，直接用它存金额会累积舍入误差。金额等精确计算应使用整数（分）或十进制类型。

要点：

- 用 `long long` 存“分”（最小货币单位），运算后再格式化。
- 或用支持十进制精度的库（如 `boost::multiprecision::cpp_dec_float`）。
- 不要用 `double` 做金额累加与等值判断。
- 输出金额时用整数运算保证精度。

示例：

```
#include <iostream>
// 错误: double 存金额
// double price = 0.1; price += 0.2; // 0.30000000000000004

// 正确: 用整数分存储
long long cents = 10;           // 0.10 元
cents += 20;                     // 0.30 元
int yuan = cents / 100;          // 元
int fen = cents % 100;           // 分
int main() {
    std::cout << yuan << "." << (fen < 10 ? "0" : "") << fen << "\n"; // 0.30
    return 0;
}
```

易错点/注意: 金额比较、累加、四舍五入都要在整数分上完成, 只在最终展示时转成小数字符串。

位运算与逻辑运算优先级 (& 与 &&)

概念: `&` (按位与) 优先级低于 `==`, 而 `&&` (逻辑与) 优先级也低于 `==`; 不加括号时 `a & b == c` 会被解析为 `a & (b == c)`, 产生错误。

要点:

- 关系/相等运算符优先级高于按位 `&`。
- 写位运算判断时务必加括号: `(a & mask) == mask`。
- `&` 是位运算 (两操作数都求值), `&&` 是逻辑运算 (短路)。

示例:

```
#include <iostream>
int main() {
    int flags = 0b1010;
    int mask = 0b1000;
    // 错误: flags & mask == mask 被解析为 flags & (mask == mask)
    // std::cout << (flags & mask == mask) << "\n"; // 语义错误

    // 正确: 加括号
    if ((flags & mask) == mask) {
        std::cout << "位被设置\n";
    }
    // && 会短路: 右侧可能不求值; & 两侧都求值
    int x = 0;
    (false && ++x); // 短路, x 不变
    std::cout << x << "\n"; // 0
    return 0;
}
```

易错点/注意: 位运算判断一律加括号; `&&/||` 有短路特性 (可用于判空后访问), `&/|` 无短路。

switch 忘记 break

概念: `switch` 中每个 `case` 执行完若不写 `break`, 会贯穿 (fall-through) 到下一个 `case` 继续执行, 这通常是 bug 来源。

要点:

- 每个 `case` 结尾写 `break` (或 `return`)。

- 确实需要贯穿时，用 `[[fallthrough]]` (C++17) 显式标注意图。
- 多个 `case` 共享同一段代码是合法的，但也要注意 `break` 位置。

示例：

```
#include <iostream>
int main() {
    int day = 2;
    switch (day) {
        case 1:
            std::cout << "周一\n";
            break; // 必须有 break
        case 2:
            std::cout << "周二\n";
            [[fallthrough]]; // C++17: 显式声明贯穿是有意为之
        case 3:
            std::cout << "周三\n";
            break;
        default:
            std::cout << "其他\n";
            break;
    }
    return 0;
}
```

易错点/注意：忘记 `break` 是最常见的 `switch` 坑；用 `[[fallthrough]]` 标注有意贯穿，能让编译器与读者都明确意图。

vector 遍历中修改容器

概念：在 `range-for` 或普通循环遍历 `vector` 时插入/删除元素，会使迭代器失效或跳过元素，导致未定义行为或逻辑错误。

要点：

- 遍历中删除元素：用 `erase` 返回的迭代器，或先收集再删除。
- 遍历中插入元素：预留容量或从后往前处理。
- 可用“擦除-移除”惯用法 `std::remove_if + erase`。

示例：

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{1, 2, 3, 4, 5, 6};
    // 正确: erase-remove 惯用法, 删除偶数
    v.erase(std::remove_if(v.begin(), v.end(),
        [](int n) { return n % 2 == 0; }), v.end());
    for (int n : v) std::cout << n << " "; // 1 3 5
    std::cout << "\n";
    return 0;
}
```

易错点/注意：不要在 `range-for` 内直接 `v.erase(...)` (迭代器失效)；用 `std::remove_if / std::erase_if` (C++20) 最安全。

字符串拼接误区 ("a"+"b")

概念：C 字符串字面量 `"a"` 是 `const char*`（指向字符数组），两个指针相加无意义（UB）。字符串拼接必须先用 `std::string` 包装。

要点：

- `"a" + "b"` 是 `const char*` 相加，非法/UB。
- 正确写法：`std::string("a") + "b"` 或 `std::string s = "a"; s += "b";`。
- C++14 起可用 `"a"s + "b"`（需 `using namespace std::string_literals;`）。
- `std::string` 与 `const char*` 相加时，重载运算符会正确转换。

示例：

```
#include <iostream>
#include <string>
using namespace std::string_literals; // 启用 "x"s 字面量
int main() {
    // 错误: const char* + const char*, 无此运算符
    // std::cout << "a" + "b" << "\n"; // 编译错误/UB

    // 正确写法 1: std::string 包装
    std::string s1 = std::string("a") + "b";
    // 正确写法 2: 字面量后缀 (C++14)
    std::string s2 = "a"s + "b";
    std::cout << s1 << " " << s2 << "\n"; // ab ab
    return 0;
}
```

易错点/注意：`"a" + "b"` 是面试常见陷阱；牢记字符串字面量是 `const char[]`，加号需 `std::string` 参与。

成员初始化顺序与初始化列表顺序不一致

概念：类成员按**声明顺序**初始化，而非初始化列表书写顺序。若初始化列表顺序与声明顺序不一致，且成员初始化相互依赖，会用到未初始化的值。

要点：

- 成员初始化顺序 == 声明顺序（与初始化列表无关）。
- 初始化列表顺序与声明顺序不一致时，编译器会警告 `-Wreorder`。
- 依赖其它成员的初始化必须保证声明顺序在前。
- 尽量让初始化列表顺序与声明顺序一致，避免混淆。

示例：

```
#include <iostream>
class Demo {
    int a; // 声明顺序: a 先, b 后
    int b;
public:
    // 错误: 初始化列表写 b(a), a(10), 但实际先初始化 a (此时是垃圾值)
    // Demo() : b(a), a(10) {} // b 先于 a 初始化, 读到 a 的垃圾值
    Demo() : a(10), b(a) {} // 正确: 顺序一致, b = 10
    void show() { std::cout << a << " " << b << "\n"; }
};
int main() {
```

```
Demo d;
d.show();    // 10 10
return 0;
}
```

易错点/注意：让初始化列表顺序**严格等于**声明顺序；打开 `-Wreorder` 让编译器帮你抓这类 bug。

悬垂 else 问题

概念：`else` 总是与**最近的、未配对的** `if` 匹配，当嵌套 `if` 缺少花括号时，`else` 可能绑定到错误的 `if`，导致逻辑错误。

要点：

- `else` 就近匹配原则。
- 嵌套 `if` 一律用花括号 `{}` 明确边界。
- 即使单语句也建议加花括号，避免缩进误导。

示例：

```
#include <iostream>
int main() {
    int x = 1, y = 0;
    // 错误: else 匹配到内层 if(y), 而非外层 if(x)
    // if (x > 0)
    //     if (y > 0)
    //         std::cout << "A\n";
    // else
    //     std::cout << "B\n";    // 实际绑定内层 if

    // 正确: 加花括号明确归属
    if (x > 0) {
        if (y > 0) {
            std::cout << "A\n";
        }
    } else {
        std::cout << "B\n";    // 明确绑定外层 if
    }
    return 0;
}
```

易错点/注意：永远用花括号包裹嵌套 `if`，不要让缩进代替语法；`else` 就近匹配是语法规则，与缩进无关。

最令人烦恼的解析 most vexing parse

概念：C++ 语法歧义：`Widget w();` 被解析为**声明一个返回 `Widget` 的函数**，而非定义一个默认构造的对象。用花括号初始化 `Widget w{};` 可规避。

要点：

- `T obj();` 是函数声明，不是对象定义。
- 用 `T obj{};` 或 `T obj = T();` 定义对象。
- 该歧义源于“声明优先”的语法规则。
- C++11 统一初始化 `{}` 是推荐解法。

示例：

```

#include <iostream>
#include <vector>
int main() {
    // 错误：被解析为函数声明
    // std::vector<int> v();    // 声明了函数 v，返回 vector<int>

    // 正确：花括号或等号初始化
    std::vector<int> v1{};      // 空 vector
    std::vector<int> v2 = std::vector<int>();
    std::cout << v1.size() << " " << v2.size() << "\n"; // 0 0
    return 0;
}

```

易错点/注意：定义对象优先用 `{}`；遇到“看起来像声明但行为不符”的编译/运行问题，先怀疑 most vexing parse。

性能优化基础

避免不必要的拷贝

概念：拷贝大对象（如 `std::string`、`std::vector`）代价高昂。通过按引用传参、按引用返回值、使用拷贝消除等手法避免无谓拷贝。

要点：

- 函数参数用 `const T&` 传大对象；仅读不写时不拷贝。
- 返回值依赖 RVO/NRVO（返回值优化），现代编译器默认开启。
- 局部变量传给容器用 `std::move` 或 `emplace` 就地构造。

示例：

```

#include <iostream>
#include <vector>
#include <string>
// 按 const 引用传参，避免拷贝
void print(const std::string& s) { std::cout << s << "\n"; }

std::string make_big() {
    std::string s(1000, 'x');
    return s;          // NRVO 优化，通常零拷贝
}

int main() {
    std::vector<std::string> v;
    v.reserve(4);
    std::string s = "data";
    v.emplace_back(std::move(s)); // 移动而非拷贝
    print(make_big());
    return 0;
}

```

易错点/注意：`std::move` 后原对象处于有效但未指定状态，不要再使用其内容；`emplace_back` 直接把构造参数转发，减少一次临时对象。

移动语义 (C++11)

概念: 移动语义通过"窃取"临时对象（右值）的内部资源（如堆指针），避免深拷贝。配合右值引用 `T&&`、`std::move`、移动构造函数/移动赋值实现。

要点:

- 右值引用 `T&&` 绑定临时对象，可安全转移资源。
- 移动构造/赋值把源对象的指针"偷"过来，再把源置空。
- 移动后源对象必须仍可析构、可赋值。
- 五法则：定义了拷贝/移动/析构任一，通常要一起定义。

示例:

```
#include <iostream>
#include <cstring>
class Buffer {
    char* data_;
public:
    explicit Buffer(const char* s) : data_(new char[std::strlen(s)+1]) {
        std::strcpy(data_, s);
    }
    // 移动构造：偷走资源
    Buffer(Buffer&& other) noexcept : data_(other.data_) {
        other.data_ = nullptr;
    }
    // 移动赋值
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            data_ = other.data_;
            other.data_ = nullptr;
        }
        return *this;
    }
    ~Buffer() { delete[] data_; }
    const char* get() const { return data_ ? data_ : "(空)"; }
};
int main() {
    Buffer a("hello");
    Buffer b(std::move(a)); // 移动构造
    std::cout << b.get() << " / " << a.get() << "\n"; // hello / (空)
    return 0;
}
```

易错点/注意: 移动构造函数应标记 `noexcept`（容器扩容时更倾向移动而非拷贝）；移动后源对象不可再解引用其资源。

reserve 预留容量

概念: `std::vector` / `std::string` 动态扩容会重新分配内存并搬移元素。提前 `reserve(n)` 预留容量，避免多次扩容的开销。

要点:

- `reserve(n)` 只分配容量，不改变 `size()`。
- `resize(n)` 改变元素个数，`reserve` 只改容量。

- 已知元素数量时先 `reserve` 再 `push_back`，性能显著提升。
- `capacity()` 查看当前容量。

示例：

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v;
    v.reserve(1000);           // 一次性预留，避免反复扩容
    for (int i = 0; i < 1000; ++i) {
        v.push_back(i);       // 无扩容搬移
    }
    std::cout << "size=" << v.size()
               << " capacity=" << v.capacity() << "\n"; // size=1000
    capacity=1000
    return 0;
}
```

易错点/注意：`reserve` 后 `size()` 仍为 0，不要用下标访问"预留"的元素；`reserve` 不会初始化元素。

内联与函数调用开销

概念：`inline` 提示编译器在调用点展开函数体，消除函数调用的栈帧开销。但对大函数、递归函数强制内联反而增大代码体积、降低缓存命中。

要点：

- `inline` 是**建议**，编译器可忽略。
- 小、频繁调用的函数（getter/setter）内联收益大。
- 现代编译器在 `-O2` 下会自动内联简单函数，无需手动标太多。
- 大函数内联会增大指令缓存压力，得不偿失。

示例：

```
#include <iostream>
// 小函数适合内联
inline int square(int x) { return x * x; }

int main() {
    int total = 0;
    for (int i = 0; i < 1000000; ++i) {
        total += square(i);    // 内联后无函数调用开销
    }
    std::cout << total << "\n";
    return 0;
}
```

易错点/注意：`inline` 关键字与"链接"语义绑定（允许头文件多份定义），不保证一定展开；真正优化交给编译器。

减少动态分配

概念： `new/malloc` 动态分配涉及系统调用和内存管理开销，频繁分配/释放会拖慢程序。尽量用栈上对象、复用缓冲区、对象池等手段减少动态分配。

要点：

- 小对象优先栈分配，别动不动 `new`。
- 复用容器/缓冲区，避免循环内反复分配。
- 用 `std::array` 替代小 `std::vector`（栈上定长）。
- 智能指针 `make_unique/make_shared` 一次分配，减少碎片。

示例：

```
#include <iostream>
#include <array>
#include <vector>
int main() {
    // 错误：循环内反复 new/delete
    // for (int i = 0; i < 1000000; ++i) { int* p = new int(i); delete p; }

    // 正确：栈上定长数组，零动态分配
    std::array<int, 100> buf{};
    for (int i = 0; i < 100; ++i) buf[i] = i * i;
    std::cout << buf[99] << "\n";
    return 0;
}
```

易错点/注意： `make_shared` 把控制块与对象一起分配（一次 `new`）；`std::array` 是栈上定长，大小必须编译期已知。

SSO 短字符串优化

概念： 多数标准库实现的 `std::string` 对短字符串（通常 15 字符以内，含 `\0`）采用**短字符串优化**（SSO）：直接在对象内部存储，不分配堆内存。

要点：

- 短字符串不触发堆分配，拷贝/移动成本极低。
- 超过阈值才分配堆内存。
- 阈值与实现相关（GCC 约 15 字节，MSVC 约 15 字节）。
- 移动一个“已用 SSO”的字符串仍需拷贝内部字符（不是零成本）。

示例：

```
#include <iostream>
#include <string>
int main() {
    std::string s = "hi";           // 短字符串，走 SSO，无堆分配
    std::cout << s.capacity() << "\n"; // GCC 通常输出 15
    std::string big(1000, 'x');     // 长字符串，堆分配
    std::cout << big.capacity() << "\n";
    return 0;
}
```

易错点/注意：SSO 意味着"移动短字符串"并非零成本（仍要拷贝内部字符）；理解 SSO 可解释某些"移动后为何还快/慢"的疑问。

缓存友好遍历

概念：CPU 按缓存行（通常 64 字节）读取内存。按**行优先**连续访问内存（如二维数组按行遍历）能最大化缓存命中，随机跳跃访问则频繁缓存失效。

要点：

- 二维数组按行遍历（内存连续）快于按列遍历。
- `std::vector` 连续内存，顺序遍历缓存友好。
- 链表节点分散在堆上，遍历缓存不友好。
- 数据布局（SoA vs AoS）也影响缓存性能。

示例：

```
#include <iostream>
#include <vector>
#include <chrono>
int main() {
    const int N = 2048;
    std::vector<std::vector<int>> m(N, std::vector<int>(N, 1));
    long long sum = 0;
    // 行优先遍历：内存连续，缓存友好
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            sum += m[i][j]; // 连续访问
    std::cout << sum << "\n";
    return 0;
}
```

易错点/注意：交换二维循环内外层（列优先）会显著变慢；对性能敏感的热点循环，保持内存访问局部性。

用算法代替手写循环

概念：标准库算法（`<algorithm>`/`<numeric>`）比手写循环更清晰、更不易出错，且编译器可对它们做优化。优先用 `std::find`、`std::copy`、`std::accumulate`、`std::sort` 等。

要点：

- 算法表达意图，减少边界错误。
- 可与 lambda 结合表达自定义逻辑。
- 编译器对标准算法有成熟优化路径。
- 并行版本 (C++17) `std::for_each(std::execution::par, ...)` 可并行化。

示例：

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
int main() {
    std::vector<int> v{3, 1, 4, 1, 5};
    // 求和：算法替代手写循环
    int sum = std::accumulate(v.begin(), v.end(), 0);
}
```

```

// 排序
std::sort(v.begin(), v.end());
// 查找
auto it = std::find(v.begin(), v.end(), 4);
std::cout << "sum=" << sum
          << " found=" << (it != v.end()) << "\n";
return 0;
}

```

易错点/注意：算法区间都是左闭右开 `[first, last)`；自定义比较器需满足严格弱序，否则 `sort` 行为未定义。

设计模式与 C++ 实现要点

单例模式（线程安全）

概念：单例保证一个类只有一个实例并提供全局访问点。C++11 起，函数内局部 `static` 对象的初始化是线程安全的，可用 Meyers 单例简洁实现。

要点：

- Meyers 单例：函数内 `static` 局部对象，首次调用才构造。
- C++11 起局部 `static` 初始化线程安全，无需双重检查锁。
- 删除拷贝构造与拷贝赋值，防止复制。
- 返回引用而非指针，语义更清晰。

示例：

```

#include <iostream>
class Singleton {
public:
    static Singleton& instance() {
        static Singleton inst;    // C++11 线程安全初始化
        return inst;
    }
    void work() { std::cout << "单例工作\n"; }
    Singleton(const Singleton&) = delete;    // 禁止拷贝
    Singleton& operator=(const Singleton&) = delete;
private:
    Singleton() = default;    // 私有构造
};
int main() {
    Singleton::instance().work();
    return 0;
}

```

易错点/注意：不要返回局部 `static` 的指针给外界持有后手动 `delete`；多线程下无需自己加锁，局部 `static` 初始化已安全。

工厂模式

概念：工厂模式把对象的创建逻辑封装起来，调用方不直接 `new` 具体类，而是通过工厂接口创建，便于解耦和扩展。

要点：

- 简单工厂：一个静态方法按参数返回不同子类。

- 工厂方法：基类声明纯虚工厂，子类实现创建。
- 返回 `std::unique_ptr<Base>` 转移所有权，避免裸指针泄漏。
- 配合虚析构函数保证正确析构。

示例：

```
#include <iostream>
#include <memory>
struct Shape {
    virtual void draw() = 0;
    virtual ~Shape() = default;    // 虚析构
};
struct Circle : Shape { void draw() override { std::cout << "画圆\n"; } };
struct Square : Shape { void draw() override { std::cout << "画方\n"; } };

struct Factory {
    static std::unique_ptr<Shape> create(int type) {
        if (type == 0) return std::make_unique<Circle>();
        return std::make_unique<Square>();
    }
};
int main() {
    auto s = Factory::create(0);
    s->draw();    // 画圆
    return 0;
}
```

易错点/注意：多态基类必须有虚析构函数，否则通过基类指针 delete 派生对象会资源泄漏/UB。

观察者模式

概念：观察者模式定义一对多依赖：主题（Subject）状态变化时通知所有观察者（Observer）。C++ 中常用接口 + `std::vector` 存储观察者指针实现。

要点：

- Subject 维护观察者列表，提供 attach/detach/notify。
- Observer 提供统一 update 接口。
- 用 `std::function` 回调可简化观察者。
- 注意观察者生命周期，避免悬垂指针。

示例：

```
#include <iostream>
#include <vector>
#include <functional>
class Subject {
    std::vector<std::function<void(int)>> observers_;    // 回调列表
public:
    void attach(std::function<void(int)> fn) { observers_.push_back(fn); }
    void notify(int state) {
        for (auto& fn : observers_) fn(state);    // 通知所有观察者
    }
};
int main() {
    Subject subj;
    subj.attach([](int s) { std::cout << "观察者A收到: " << s << "\n"; });
}
```

```

    subj.attach([](int s) { std::cout << "观察者B收到: " << s << "\n"; });
    subj.notify(42);
    return 0;
}

```

易错点/注意：观察者若被销毁而未 detach，回调会触发悬垂引用；`std::function` 捕获裸指针/引用时要特别注意生命周期。

策略模式（std::function 或模板实现）

概念：策略模式把算法族封装为可互换的策略对象，运行时可动态替换。C++ 可用 `std::function`（运行时多态）或模板（编译期多态）实现。

要点：

- `std::function` 方式：算法作为函数对象注入，灵活、运行时切换。
- 模板方式：策略作为模板参数，零虚函数开销，编译期绑定。
- 策略接口用 lambda、函数指针、仿函数均可。

示例：

```

#include <iostream>
#include <functional>
#include <vector>
#include <algorithm>
// 方式一: std::function 运行时策略
class Sorter {
public:
    template <typename T>
    static void sort(std::vector<T>& v, std::function<bool(T,T)> cmp) {
        std::sort(v.begin(), v.end(), cmp);
    }
};

int main() {
    std::vector<int> v{3, 1, 2};
    Sorter::sort<int>(v, [](int a, int b) { return a < b; }); // 升序
    for (int n : v) std::cout << n << " ";
    std::cout << "\n";
    // 换策略: 降序
    Sorter::sort<int>(v, [](int a, int b) { return a > b; });
    for (int n : v) std::cout << n << " ";
    std::cout << "\n";
    return 0;
}

```

易错点/注意：`std::function` 有类型擦除开销，性能敏感场景用模板/lambda 直接传比较器更优；策略对象需是可调用的。

面试高频手写题

手写 strlen / strcpy / strcmp

概念：三个 C 字符串基础函数的实现考察对指针、`'\0'` 结束符和逐字节操作的理解。

要点：

- `strlen` 数到 `'\0'` 为止，不含 `'\0'`。

- `strcpy` 要把 `'\0'` 一并拷贝。
- `strcmp` 返回 `unsigned char` 差值，避免符号扩展问题。

示例：

```
#include <cstddef>
#include <cassert>
size_t my_strlen(const char* s) {
    const char* p = s;
    while (*p) ++p;           // 走到 '\0'
    return p - s;             // 指针差即长度
}
char* my_strcpy(char* dest, const char* src) {
    char* p = dest;
    while ((*p++ = *src++)) {} // 连同 '\0' 一起拷贝
    return dest;
}
int my_strcmp(const char* a, const char* b) {
    while (*a && (*a == *b)) { ++a; ++b; }
    return *(const unsigned char*)a - *(const unsigned char*)b; // 转 unsigned 防
符号问题
}
int main() {
    char buf[16];
    my_strcpy(buf, "hello");
    assert(my_strlen(buf) == 5);
    assert(my_strcmp("abc", "abd") < 0);
    assert(my_strcmp("abc", "abc") == 0);
    return 0;
}
```

易错点/注意：`strcmp` 若直接用 `*a - *b` 且 `char` 为有符号时，结果可能错误，应转 `unsigned char`。

手写 string 类（深拷贝 + 移动）

概念：实现一个管理堆内存的简化 `string`，必须正确处理**深拷贝**（拷贝构造/赋值）和**移动**（移动构造），并遵守三/五法则。

要点：

- 深拷贝：新分配内存，复制内容，避免浅拷贝双 free。
- 移动：偷走资源并把源置空。
- 拷贝赋值需处理自赋值；析构释放资源。

示例：

```
#include <cstring>
#include <iostream>
class MyString {
    char* data_;
    size_t size_;
public:
    MyString(const char* s = "") : size_(std::strlen(s)) {
        data_ = new char[size_ + 1];
        std::strcpy(data_, s);
    }
}
```

```

MyString(const MyString& o) : size_(o.size_) { // 深拷贝构造
    data_ = new char[size_ + 1];
    std::strcpy(data_, o.data_);
}
MyString(MyString&& o) noexcept : data_(o.data_), size_(o.size_) { // 移动构造
    o.data_ = nullptr;
    o.size_ = 0;
}
MyString& operator=(const MyString& o) { // 拷贝赋值（含自赋值保护）
    if (this != &o) {
        char* tmp = new char[o.size_ + 1];
        std::strcpy(tmp, o.data_);
        delete[] data_;
        data_ = tmp;
        size_ = o.size_;
    }
    return *this;
}
~MyString() { delete[] data_; }
const char* c_str() const { return data_; }
size_t size() const { return size_; }
};

int main() {
    MyString a("hello");
    MyString b = a; // 深拷贝
    MyString c = std::move(a); // 移动
    std::cout << b.c_str() << " " << c.c_str() << "\n"; // hello hello
    return 0;
}

```

易错点/注意：浅拷贝会让两个对象指向同一块内存，析构时 double free；拷贝赋值必须先分配成功再释放旧内存（保证异常安全）。

手写简化版 shared_ptr / unique_ptr

概念：智能指针用 RAII 管理资源。shared_ptr 用引用计数共享所有权；unique_ptr 独占所有权、禁止拷贝。

要点：

- shared_ptr：拷贝时计数 +1，析构时 -1，归零时释放。
- unique_ptr：移动转移所有权，拷贝被 delete。
- 实现 operator*、operator-> 使行为像指针。

示例：

```

#include <iostream>
// 简化 shared_ptr: 引用计数
template <typename T>
class SharedPtr {
    T* ptr_;
    int* ref_;
public:
    explicit SharedPtr(T* p = nullptr) : ptr_(p), ref_(new int(1)) {}
    SharedPtr(const SharedPtr& o) : ptr_(o.ptr_), ref_(o.ref_) { ++(*ref_); }
    SharedPtr& operator=(const SharedPtr& o) {
        if (this != &o) { release(); ptr_ = o.ptr_; ref_ = o.ref_; ++(*ref_); }
    }
};

```

```

        return *this;
    }
    ~SharedPtr() { release(); }
    T* operator->() { return ptr_; }
    T& operator*() { return *ptr_; }
    int use_count() const { return *ref_; }
private:
    void release() { if (--(*ref_) == 0) { delete ptr_; delete ref_; } }
};

// 简化 unique_ptr: 独占所有权
template <typename T>
class UniquePtr {
    T* ptr_;
public:
    explicit UniquePtr(T* p = nullptr) : ptr_(p) {}
    UniquePtr(UniquePtr&& o) noexcept : ptr_(o.ptr_) { o.ptr_ = nullptr; }
    UniquePtr& operator=(UniquePtr&& o) noexcept {
        if (this != &o) { delete ptr_; ptr_ = o.ptr_; o.ptr_ = nullptr; }
        return *this;
    }
    UniquePtr(const UniquePtr&) = delete; // 禁止拷贝
    UniquePtr& operator=(const UniquePtr&) = delete;
    ~UniquePtr() { delete ptr_; }
    T* operator->() { return ptr_; }
    T& operator*() { return *ptr_; }
};

int main() {
    SharedPtr<int> sp(new int(10));
    SharedPtr<int> sp2 = sp; // 计数 = 2
    std::cout << sp.use_count() << " " << *sp << "\n"; // 2 10
    UniquePtr<int> up(new int(20));
    UniquePtr<int> up2 = std::move(up); // 移动, up 变空
    std::cout << *up2 << "\n"; // 20
    return 0;
}

```

易错点/注意: `shared_ptr` 的引用计数必须是**共享的** (`int*`)，不能是普通 `int` 成员，否则各对象各管各的计数。

手写简化版 vector

概念: 实现动态数组核心: `push_back` 时若容量不足则**扩容 (通常翻倍)** 并搬移元素, 维护 `size` 与 `capacity`。

要点:

- 扩容: 分配更大内存, 拷贝旧元素, 释放旧内存。
- 容量翻倍保证均摊 $O(1)$ 的 `push_back`。
- 析构释放内存; `operator[]` 返回引用。

示例:

```

#include <iostream>
#include <cstdint>
template <typename T>
class MyVector {

```

```

T* data_;
size_t size_;
size_t cap_;
public:
MyVector() : data_(nullptr), size_(0), cap_(0) {}
~MyVector() { delete[] data_; }
void push_back(const T& v) {
    if (size_ == cap_) { // 需要扩容
        size_t nc = (cap_ == 0) ? 1 : cap_ * 2;
        T* nd = new T[nc];
        for (size_t i = 0; i < size_; ++i) nd[i] = data_[i]; // 拷贝旧元素
        delete[] data_;
        data_ = nd;
        cap_ = nc;
    }
    data_[size_++] = v;
}
T& operator[](size_t i) { return data_[i]; }
size_t size() const { return size_; }
size_t capacity() const { return cap_; }
};

int main() {
    MyVector<int> v;
    for (int i = 0; i < 5; ++i) v.push_back(i);
    for (size_t i = 0; i < v.size(); ++i) std::cout << v[i] << " "; // 0 1 2 3 4
    std::cout << "\ncapacity=" << v.capacity() << "\n"; // 8
    return 0;
}

```

易错点/注意：扩容用 `new T[n]` 只适用于可默认构造的 `T`；真正的 vector 用 allocator + placement new 处理非平凡类型。

单例模式 (C++11 线程安全)

概念：C++11 起函数内局部 `static` 初始化线程安全，Meyers 单例无需双重检查锁即可保证唯一实例与线程安全。

要点：

- 局部 `static` 首次执行到才初始化，且仅一次。
- 私有构造 + 删除拷贝。
- 返回引用。

示例：

```

#include <iostream>
class Singleton {
public:
    static Singleton& get() {
        static Singleton inst; // C++11 线程安全，懒初始化
        return inst;
    }
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;
private:
    Singleton() = default;
};

```

```
int main() {
    std::cout << (&Singleton::get() == &Singleton::get()) << "\n"; // 1, 同一实例
    return 0;
}
```

易错点/注意：别再手写双重检查锁 + 裸指针，既复杂又易错；局部 static 已是标准最优解。

快速排序 / 归并排序

概念：快排基于**分治 + 分区**（平均 $O(n \log n)$ ），归并排序稳定地 $O(n \log n)$ 但需 $O(n)$ 额外空间。二者都是面试高频。

要点：

- 快排：选 pivot，分区，递归；最坏 $O(n^2)$ 。
- 归并：拆半递归，合并两个有序段；稳定。
- 快排注意边界与 pivot 选取；归并注意临时数组。

示例：

```
#include <iostream>
#include <algorithm>
// 快速排序 (Hoare 分区思路的简化 Lomuto 版)
int partition_q(int a[], int lo, int hi) {
    int pivot = a[hi];
    int i = lo;
    for (int j = lo; j < hi; ++j)
        if (a[j] < pivot) std::swap(a[i++], a[j]);
    std::swap(a[i], a[hi]);
    return i;
}
void quicksort(int a[], int lo, int hi) {
    if (lo < hi) {
        int p = partition_q(a, lo, hi);
        quicksort(a, lo, p - 1);
        quicksort(a, p + 1, hi);
    }
}
// 归并排序
void merge_m(int a[], int lo, int mid, int hi) {
    int n = hi - lo + 1;
    int* tmp = new int[n];
    int i = lo, j = mid + 1, k = 0;
    while (i <= mid && j <= hi) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi) tmp[k++] = a[j++];
    for (k = 0; k < n; ++k) a[lo + k] = tmp[k];
    delete[] tmp;
}
void mergesort(int a[], int lo, int hi) {
    if (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        mergesort(a, lo, mid);
        mergesort(a, mid + 1, hi);
        merge_m(a, lo, mid, hi);
    }
}
```

```
int main() {
    int a[] = {3, 1, 4, 1, 5, 9, 2};
    int n = 7;
    quicksort(a, 0, n - 1);
    for (int i = 0; i < n; ++i) std::cout << a[i] << " "; // 1 1 2 3 4 5 9
    std::cout << "\n";
    return 0;
}
```

易错点/注意：快排分区边界和递归出口 (`lo < hi`) 写错会导致死循环或越界；归并 `mid` 用 `lo + (hi - lo)/2` 防溢出。

二分查找（左闭右开）

概念：二分查找在有序区间内 $O(\log n)$ 定位元素。推荐采用**左闭右开** `[lo, hi)` 的写法，边界清晰、不易出错。

要点：

- 区间 `[lo, hi)`，循环条件 `lo < hi`。
- `mid = lo + (hi - lo) / 2` 防溢出。
- 找第一个 `>= target` (`lower_bound`) 是最通用的写法。
- 收缩边界时保持左闭右开不变式。

示例：

```
#include <iostream>
// 返回第一个 >= target 的下标 (lower_bound), [lo, hi) 左闭右开
int lower_bound(int a[], int n, int target) {
    int lo = 0, hi = n; // [0, n)
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1; // 目标在右半
        else hi = mid; // 目标在左半 (含 mid)
    }
    return lo; // 第一个 >= target 的位置
}
int main() {
    int a[] = {1, 3, 5, 5, 7, 9};
    std::cout << lower_bound(a, 6, 5) << "\n"; // 2, 第一个 5 的下标
    std::cout << lower_bound(a, 6, 8) << "\n"; // 5, 不存在则返回插入点
    return 0;
}
```

易错点/注意：`hi = mid` (不是 `mid-1`) 配合 `lo < hi` 才不会漏元素；`lo + (hi-lo)/2` 避免 `lo+hi` 溢出。

链表基础与反转链表

概念：链表节点含数据与 `next` 指针。反转链表用**三指针** (`prev/cur/next`) 迭代翻转指针方向，是链表题的基础。

要点：

- 反转：保存 `next`，翻转 `cur->next = prev`，再前移。
- 迭代法 $O(n)$ 、 $O(1)$ 空间；也可递归。
- 注意空链表与单节点边界。

示例:

```
#include <iostream>
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v) : val(v), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* cur = head;
    while (cur) {
        ListNode* next = cur->next; // 先保存后继
        cur->next = prev;           // 翻转指针
        prev = cur;                 // prev 前移
        cur = next;                 // cur 前移
    }
    return prev; // 新的头
}

int main() {
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    ListNode* r = reverseList(head);
    for (ListNode* p = r; p; p = p->next) std::cout << p->val << " "; // 3 2 1
    std::cout << "\n";
    return 0;
}
```

易错点/注意: 反转前必须先保存 `cur->next`，否则翻转后丢失后继；面试常考递归版，注意递归返回新的头并传递。

LRU 缓存思路 (list + unordered_map)

概念: LRU (最近最少使用) 缓存淘汰最久未使用的条目。用**双向链表**维护访问顺序 (头部最新)，用**哈希表**实现 $O(1)$ 查找。

要点:

- `list` 存 (key, value)，头部最新、尾部最旧。
- `unordered_map` 存 key -> `list` 迭代器， $O(1)$ 定位。
- `get`: 命中则把节点移到头部；`put`: 更新或淘汰尾部再插头部。

示例:

```
#include <iostream>
#include <list>
#include <unordered_map>
class LRUCache {
    int cap_;
    std::list<std::pair<int, int>> l_; // 头部最新，尾部最旧
    std::unordered_map<int, std::list<std::pair<int, int>>::iterator> m_;
public:
    explicit LRUCache(int c) : cap_(c) {}
    int get(int key) {
        auto it = m_.find(key);
        if (it == m_.end()) return -1;
    }
};
```

```

        l_.splice(l_.begin(), l_, it->second);           // 移到头部
        return it->second->second;
    }
    void put(int key, int value) {
        if (m_.count(key)) {                             // 已存在，更新并移到头
            l_.splice(l_.begin(), l_, m_[key]);
            m_[key]->second = value;
            return;
        }
        if ((int)l_.size() == cap_) {                     // 淘汰最旧（尾部）
            m_.erase(l_.back().first);
            l_.pop_back();
        }
        l_.push_front({key, value});                     // 新节点插头部
        m_[key] = l_.begin();
    }
};
int main() {
    LRUCache c(2);
    c.put(1, 1); c.put(2, 2);
    std::cout << c.get(1) << "\n"; // 1
    c.put(3, 3); // 淘汰 key=2
    std::cout << c.get(2) << "\n"; // -1, 已淘汰
    return 0;
}

```

易错点/注意： `splice` 直接转移节点，迭代器保持有效，是 $O(1)$ 移头的关键；淘汰时记得同时 `m_.erase`，否则哈希表残留失效迭代器。

复习与刷题建议

学习路线、经典书籍与在线练习平台

概念： C++ 学习应从语言基础 → 标准库 → 进阶特性 → 工程实践循序渐进；配合经典书籍和在线平台刷题巩固。

要点：

- **学习路线：** 语法与内存模型 → 类/模板/STL → 移动语义与智能指针 → 并发 → 设计模式与工程 (CMake/gdb)。
- **经典书籍：** 《C++ Primer (第5版)》(入门)、《Effective C++》(规范)、《Effective Modern C++》(C++11/14)、《STL 源码剖析》(底层)。
- **在线平台：** LeetCode (算法面试题)、牛客网 (国内面经)、cppreference.com (权威参考)、Compiler Explorer (在线编译实验)。
- 刷题时优先手写核心数据结构和算法 (本部分手写题)，理解而非背诵。

示例：

```
// 学习路线示例：先掌握 STL 算法与 lambda，再进阶并发
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{5, 2, 8, 1, 9};
    std::sort(v.begin(), v.end());           // STL 算法
    int cnt = std::count_if(v.begin(), v.end(), // lambda 配合算法
        [](int n) { return n > 5; });
    std::cout << "大于5的个数: " << cnt << "\n"; // 2
    return 0;
}
```

易错点/注意：不要只看书不写码——每个知识点都要在本地编译运行；遇到 UB 用 `-fsanitize` 复现，比死记更有效。

本部分小结

易忘点	说明
数组越界是 UB	<code>operator[]</code> 不检查， <code>at()</code> 抛异常
有符号溢出是 UB	无符号才按模回绕
浮点 <code>==</code> 不可靠	用 <code>fabs(a-b) < eps</code> 近似比较
迭代器失效	扩容/erase 后旧迭代器不可用，用 erase 返回值
返回局部引用/指针	悬垂，按值返回或返回所有权对象
静态初始化顺序	跨文件全局对象顺序未定义，用局部 static 单例
金额用整数分	不要用 double 存钱
<code>&</code> 优先级低于 <code>==</code>	位判断加括号
成员初始化顺序 = 声明顺序	初始化列表顺序不一致会踩坑
most vexing parse	<code>T obj();</code> 是函数声明，用 <code>{}</code>
避免拷贝	传 <code>const&</code> 、移动、reserve、emplace
智能指针	shared_ptr 引用计数、unique_ptr 独占
二分边界	左闭右开 <code>[lo, hi)</code> 、 <code>hi = mid</code>
LRU	list 维护顺序 + unordered_map 定位，splice 移头

- 核心心法：**优先用标准库**（智能指针、容器、算法）规避大部分陷阱；**手为题重理解**（指针、所有权、边界）；**性能优化先测量再优化**（profile 找到热点），避免过早优化。